CONCEPTIFYYY

OFFICIAL PLACEMENT CODING COMPENDIUM • 2026 EDITION

Company-Wise DSA Placement Preparation

Most Important Coding Questions and Complete Java Solutions for Freshers

TARGET COMPANY EXAM PATTERNS INCLUDED

- **TCS NQT (Ninja / Digital / Prime)**
- **Infosys (SE & Specialist Programmer)**
- **Accenture ASE (Advanced Entry)**
- **Deloitte Analyst & Tech Roles**
- **Tech Mahindra Entry-Level**
- **Cognizant GenC & Next**
- **Wipro Turbo & Elite NTH**
- **Capgemini Exceller & Analyst**
- **HCLTech Graduate Engineer**
- **+ The 40-Problem Master Revision List**

KEY FEATURES & CODING STANDARDS

' 216 Verified Coding Problems with Compilable Java 21 Syntax
' Complete Console I/O Programs (Scanner) & Method-Level Solutions
' Step-by-Step Dry Runs, Big-O Time & Space Complexity Analyses
' Boundary Edge Case Coverage & 60-Day Placement Roadmap

Published by Conceptifyyy Placement Engineering Team

Verified for 2026 Campus Recruitment Drives • All Rights Reserved

Conceptifyyy — Company-Wise DSA Placement Preparation 2026

2026 Official Campus Recruitment Edition

Copyright © 2026 CONCEPTIFYYY. All rights reserved.

This document and its comprehensive contents are published exclusively by CONCEPTIFYYY for educational and individual placement preparation purposes.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—without prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and academic studies.

Disclaimer:
Company names (TCS, Cognizant, Infosys, Wipro, Accenture, Capgemini, Deloitte, HCLTech, Tech Mahindra) and trademarks mentioned in this guide belong to their respective corporate entities. References to these companies are strictly for historical exam question pattern identification, student guidance, and educational training. CONCEPTIFYYY is an independent educational publisher and is not officially affiliated with, endorsed by, or sponsored by any of the named corporations. All solutions, explanations, and code implementations are original works developed by the Conceptifyyy technical team.

Published by:
CONCEPTIFYYY Digital Store & Learning Platform
Web: conceptifyyy.com | Support: support@conceptifyyy.com
Printed and Generated in India • 2026 Edition

IMPORTANT NOTICE FOR STUDENTS & ASPIRANTS

This handbook is engineered to deliver concise, pattern-oriented preparation. In actual technical interviews, interviewers prioritize clean code, descriptive variable names, and your ability to articulate trade-offs over memorized syntax.

Always explain your brute-force thought process first before offering an optimal solution. Walk through the dry-run steps provided for each question to demonstrate clear algorithmic reasoning.

Table of Contents

Complete index of all chapters, company questions, and reference sections.

Introduction & How to Master Placements 7

Chapter 1: TCS NQT — National Qualifier Test 8

1. Second-Largest Element in an Array 8
2. Find the Missing Number 10
3. Move Zeros to the End 11
4. Rotate an Array by K Positions 12
5. Find Duplicate Elements in an Array 13
6. Reverse a Number 14
7. Check Armstrong Number 15
8. Check Prime Number 16
9. Find GCD and LCM 17
10. Fibonacci Series 18
11. Check Palindrome String 19
12. Check Whether Two Strings Are Anagrams 20
13. Count Character Frequencies 21
14. Reverse Words in a Sentence 22
15. Find the First Non-Repeating Character 23
16. Two Sum 24
17. Maximum Subarray Sum (Kadane's Algorithm) 24
18. Binary Search 26
19. Merge Two Sorted Arrays 27
20. Sort 0s, 1s and 2s (Dutch National Flag Algorithm) 28

Chapter 2: Cognizant GenC & GenC Elevate / Next 30

Table of Contents (Continued)

1. Find the Second-Largest Element 30

2. Rearrange Arrays to Minimize Sum of Pairwise Products 32

3. Find the Missing Number 33

4. Move Zeros to the End 34

5. Find a Pair with a Target Sum 35

6. Count the Frequency of Elements 36

7. Maximum Subarray Sum 36

8. Rotate an Array 38

9. Check Whether Two Strings Are Anagrams 39

10. Find the First Non-Repeating Character 40

11. Reverse Words in a String 41

12. Find the Maximum Occurring Character 41

13. Check Palindrome 42

14. Merge Two Sorted Arrays 43

15. Binary Search 44

16. Find Duplicate Elements 45

17. Stock Buy and Sell for Maximum Profit 46

18. Minimum Number of Coins 47

Chapter 3: Infosys — Systems Engineer & Specialist Programmer 49

1. Find the Missing Number 49

2. Remove Duplicates from an Array 50

3. Two Sum 51

4. Maximum Subarray Sum 51

5. Longest Substring Without Repeating Characters 52

Table of Contents (Continued)

6. Merge Overlapping Intervals 54

7. Search in a Rotated Sorted Array 55

8. Count Inversions in an Array 56

9. Generate All Subsets (Power Set) 57

10. Generate All Permutations 58

11. Subset Sum Problem 59

12. 0/1 Knapsack Problem 59

13. Coin Change (Fewest Coins) 60

14. Longest Common Subsequence (LCS) 61

15. Longest Increasing Subsequence (LIS) 62

16. Number of Islands 63

17. BFS and DFS Traversal of Graph 64

18. Dijkstra's Shortest Path Algorithm 66

Chapter 4: Wipro — Entry-Level & Turbo Technical 69

1. Reverse a Number 69

2. Check a Prime Number 70

3. Print the Fibonacci Series 70

4. Calculate Factorial Using Recursion 71

5. Find GCD and LCM 72

6. Find the Second-Largest Array Element 72

7. Find the Missing Number 73

8. Move Zeros to the End 74

9. Remove Duplicates from an Array 74

10. Rotate an Array by K 75

Table of Contents (Continued)

11. Reverse a String 76

12. Check Palindrome and Anagram 76

13. Count Vowels and Consonants 78

14. Find Character Frequencies 78

15. Binary Search 79

16. Bubble Sort and Selection Sort 80

17. Merge Two Sorted Arrays 81

18. Maximum Subarray Sum 81

19. Reverse a Linked List 82

20. Balanced Parentheses 84

Chapter 5: Accenture — ASE & Advanced Entry-Level 85

1. Find the Second-Largest Element 85

2. Count Even and Odd Elements 86

3. Find a Missing Number 86

4. Move Zeros to the End 87

5. Find a Pair with a Target Sum 88

6. Sort an Array by Frequency 88

7. Check Whether Two Strings Are Anagrams 89

8. Remove Duplicate Characters 90

9. Reverse Words in a Sentence 91

10. Find the First Non-Repeating Character 91

11. Find the Longest Common Prefix 92

12. Longest Substring Without Repeating Characters 93

13. Check Balanced Parentheses 94

Introduction & How to Master Campus Placements

Welcome to the Conceptifyyy Company-Wise DSA Placement Preparation 2026 guide. Every year, over 1.5 million engineering graduates across India compete for coveted technical positions at major IT and product-oriented service companies. While the volume of applicants is high, recruitment assessment data reveals a consistent reality: the core technical interview questions focus on a clearly defined set of recurring algorithmic patterns.

Rather than attempting to solve 1,500 disconnected problems on online platforms, this guide provides a structured, high-yield mastery path. We have categorized the most frequently asked coding questions across the top 9 recruitment powerhouses: TCS NQT, Cognizant GenC, Infosys (Systems Engineer & Specialist Programmer), Wipro Turbo, Accenture ASE, Capgemini Exceller, Deloitte Analyst, HCLTech, and Tech Mahindra.

Notice the significant pattern overlap between these companies: array manipulation, two pointers, hashing, Kadane's algorithm, sliding windows, and bitwise operations appear universally. When you master these core problems, your technical preparation becomes 100% transferable across every campus drive.

How to Use This Book Effectively:
1. Solve the 40-Problem Master List First: Before diving into company-specific chapters, complete Section 14 (The Final Master List). These 40 problems form the foundation for 85% of all fresher coding rounds.
2. Understand the 'Why', Not Just the 'How': For every problem, read the Intuition and Approach section carefully. Understand why a two-pointer technique achieves O(N) time or why a Min-Heap solves Kth-Largest in O(N log K).
3. Hand-trace with Dry Runs: Walk through the supplied dry-run tables step-by-step. In technical interviews, interviewers often ask you to dry-run code on a whiteboard with custom test cases.
4. Practice Full Input/Output: In company rounds like Cognizant and TCS, you must write a complete compilable Java class with Scanner input and exact output formatting without extraneous prompt messages.
5. Review the Complexity Reference Sheet: Before your technical interview, review Section 15 to instantly articulate Big-O time and auxiliary space complexities with confidence.

CHAPTER 1

TCS NQT — National Qualifier Test

Most Important Coding Questions with Complete Java Solutions

Target Focus: Ninja, Digital & Prime Recruitment Tracks

TCS NQT heavily emphasizes array logic, mathematical properties (prime, armstrong, fibonacci), string manipulation, and Kadane's algorithm. Ensure edge case handling for large inputs.

Questions in this Section (20 Problems):

#1: Second-Largest Element in an Array — Arrays

PROBLEM STATEMENT:

Given an array of integers, find the second-largest distinct element in the array. If no distinct second-largest element exists (e.g., all elements are identical or array size is less than 2), return -1.

SAMPLE INPUT & OUTPUT:

```
Input: arr = [12, 35, 1, 10, 34, 1]
Output: 34
Explanation: The largest element is 35 and the second-largest is 34.
```

ALGORITHM & INTUITION:

Maintain two variables: 'largest' and 'secondLargest', initialized to Integer.MIN_VALUE. Iterate through the array once (single-pass O(N)): if current element is strictly greater than 'largest', update secondLargest = largest, then largest = element. Otherwise, if element is strictly less than largest but greater than secondLargest, update secondLargest = element. This eliminates the need for sorting (O(N log N)).

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class SecondLargest {
    public static int findSecondLargest(int[] arr) {
        if (arr == null || arr.length < 2) return -1;
        int largest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;

        for (int num : arr) {
            if (num > largest) {
                secondLargest = largest;
                largest = num;
            } else if (num < largest && num > secondLargest) {
                secondLargest = num;
            }
        }
        return (secondLargest == Integer.MIN_VALUE) ? -1 : secondLargest;
    }

    public static void main(String[] args) {
        int[] arr = {12, 35, 1, 10, 34, 1};
```

```
        System.out.println("Second Largest: " + findSecondLargest(arr)); // Output: 34
    }
}
```

DRY RUN WALKTHROUGH:

arr = [12, 35, 1, 10, 34, 1]
1. num=12: 12 > MIN -> second=MIN, largest=12
2. num=35: 35 > 12 -> second=12, largest=35
3. num=1: 1 < 35 and 1 < 12 -> no change
4. num=10: 10 < 35 and 10 < 12 -> no change
5. num=34: 34 < 35 and 34 > 12 -> second=34
6. num=1: no change.
Result: 34

TIME COMPLEXITY:
O(N) — single traversal over the array of size N.

SPACE COMPLEXITY:
O(1) — constant extra space using only two integer pointers.

EDGE CASES: Empty array or array of length 1 (returns -1); all elements equal like [10, 10, 10] (returns -1); negative numbers array like [-5, -2, -8] (correctly identifies -5).

#2: Find the Missing Number

Arrays

PROBLEM STATEMENT:

Given an array containing n distinct numbers taken from the range 0 to n (or 1 to n+1), find the one number that is missing from the range.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [3, 0, 1] (n = 3, range [0, 3])
Output: 2
Explanation: Range has length 3: {0, 1, 2, 3}. 2 is missing.
```

ALGORITHM & INTUITION:

Use XOR bitwise operation. We know that a ^ a = 0 and a ^ 0 = a. XOR all indices from 0 to n, and XOR all array values. All matching values cancel out, leaving the missing number. Alternatively, use Gauss' summation formula sum = n*(n+1)/2 and subtract array sum, though XOR prevents potential integer overflow with large inputs.

COMPLETE JAVA 21 SOLUTION:

```java
public class MissingNumber {
    public static int findMissing(int[] nums) {
        int n = nums.length;
        int xorVal = 0;
        // XOR all numbers from 0 to n
        for (int i = 0; i <= n; i++) {
            xorVal ^= i;
        }
        // XOR all elements in the array
        for (int num : nums) {
            xorVal ^= num;
        }
        return xorVal;
    }

    public static void main(String[] args) {
        int[] nums = {3, 0, 1};
        System.out.println("Missing Number: " + findMissing(nums)); // Output: 2
    }
}
```

DRY RUN WALKTHROUGH:

nums = [3, 0, 1], n = 3
xorVal = (0 ^ 1 ^ 2 ^ 3) ^ (3 ^ 0 ^ 1)
Rearranging: (0 ^ 0) ^ (1 ^ 1) ^ (3 ^ 3) ^ 2 = 0 ^ 0 ^ 0 ^ 2 = 2.

TIME COMPLEXITY:
O(N) — single linear pass to compute XOR values.

SPACE COMPLEXITY:
O(1) — only one accumulator variable used.

EDGE CASES: Missing 0 at the start [1, 2, 3]; missing n at the end [0, 1, 2]; single element array [0] (missing 1) or [1] (missing 0).

#3: Move Zeros to the End

Arrays

PROBLEM STATEMENT:

Given an integer array nums, move all 0's to the end of it while maintaining the relative order of the non-zero elements. Must be done in-place without copying the array.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [0, 1, 0, 3, 12]
Output: [1, 3, 12, 0, 0]
```

ALGORITHM & INTUITION:

Use a two-pointer technique. Let 'insertPos' track the position where the next non-zero element should be placed. Traverse the array with pointer 'i'. Whenever nums[i] != 0, swap nums[insertPos] and nums[i], then increment insertPos. This keeps all non-zero elements in order and shifts zeros to the tail.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class MoveZeros {
    public static void moveZeroes(int[] nums) {
        if (nums == null || nums.length <= 1) return;
        int insertPos = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != 0) {
                // Swap non-zero element to insertPos
                int temp = nums[insertPos];
                nums[insertPos] = nums[i];
                nums[i] = temp;
                insertPos++;
            }
        }
    }

    public static void main(String[] args) {
        int[] nums = {0, 1, 0, 3, 12};
        moveZeroes(nums);
        System.out.println("Modified: " + Arrays.toString(nums)); // [1, 3, 12, 0, 0]
    }
}
```

DRY RUN WALKTHROUGH:

nums = [0, 1, 0, 3, 12], insertPos = 0
- i=0: nums[0]=0, skip
- i=1: nums[1]=1 != 0, swap nums[0] & nums[1] -> [1, 0, 0, 3, 12], insertPos=1
- i=2: nums[2]=0, skip
- i=3: nums[3]=3 != 0, swap nums[1] & nums[3] -> [1, 3, 0, 0, 12], insertPos=2
- i=4: nums[4]=12 != 0, swap nums[2] & nums[4] -> [1, 3, 12, 0, 0], insertPos=3.
Result: [1, 3, 12, 0, 0]

TIME COMPLEXITY:
O(N) — single pass through the array.

SPACE COMPLEXITY:
O(1) — in-place modification with no extra array allocation.

EDGE CASES: Array with no zeros [1, 2, 3]; array with all zeros [0, 0, 0]; single element [0]; zeros already at the end [1, 2, 0, 0].

#4: Rotate an Array by K Positions

Arrays

PROBLEM STATEMENT:

Given an array, rotate the array to the right by k steps, where k is non-negative.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [1, 2, 3, 4, 5, 6, 7], k = 3
Output: [5, 6, 7, 1, 2, 3, 4]
```

ALGORITHM & INTUITION:

Three-step reversal algorithm:
1. Normalize k: k = k % n.
2. Reverse the entire array.
3. Reverse the first k elements (index 0 to k-1).
4. Reverse the remaining n-k elements (index k to n-1).
This achieves rotation in O(N) time and O(1) auxiliary space without buffer memory.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class RotateArray {
    public static void rotate(int[] nums, int k) {
        if (nums == null || nums.length <= 1) return;
        int n = nums.length;
        k = k % n;
        if (k == 0) return;

        reverse(nums, 0, n - 1); // 1. Reverse all
        reverse(nums, 0, k - 1); // 2. Reverse first k
        reverse(nums, k, n - 1); // 3. Reverse rest
    }

    private static void reverse(int[] nums, int left, int right) {
        while (left < right) {
            int temp = nums[left];
            nums[left] = nums[right];
            nums[right] = temp;
            left++;
            right--;
        }
    }

    public static void main(String[] args) {
        int[] nums = {1, 2, 3, 4, 5, 6, 7};
        rotate(nums, 3);
        System.out.println("Rotated: " + Arrays.toString(nums)); // [5, 6, 7, 1, 2, 3, 4]
    }
}
```

DRY RUN WALKTHROUGH:

nums = [1, 2, 3, 4, 5, 6, 7], k = 3
1. Reverse all: [7, 6, 5, 4, 3, 2, 1]
2. Reverse 0 to 2: [5, 6, 7, 4, 3, 2, 1]
3. Reverse 3 to 6: [5, 6, 7, 1, 2, 3, 4]
Final output: [5, 6, 7, 1, 2, 3, 4]

TIME COMPLEXITY:
O(N) — each element is swapped twice.

SPACE COMPLEXITY:
O(1) — in-place memory reversal.

EDGE CASES: k = 0 or k = n (no change needed); k > n (handled by k % n); single element array [10].

#5: Find Duplicate Elements in an Array

Arrays

PROBLEM STATEMENT:

Given an array of integers, find all duplicate elements that appear more than once in the array.

SAMPLE INPUT & OUTPUT:

```
Input: arr = [4, 3, 2, 7, 8, 2, 3, 1]
Output: [2, 3]
```

ALGORITHM & INTUITION:

Use a HashSet to record numbers seen so far. As we iterate through each element, if the set already contains the element, add it to our duplicate result list. Alternatively, if elements are guaranteed to be in range [1, n], we can use element sign-negation in O(1) space.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class FindDuplicates {
    public static List<Integer> findDuplicates(int[] nums) {
        List<Integer> duplicates = new ArrayList<>();
        Set<Integer> seen = new HashSet<>();
        Set<Integer> added = new HashSet<>();

        for (int num : nums) {
            if (seen.contains(num)) {
                if (!added.contains(num)) {
                    duplicates.add(num);
                    added.add(num);
                }
            } else {
                seen.add(num);
            }
        }
        return duplicates;
    }

    public static void main(String[] args) {
        int[] arr = {4, 3, 2, 7, 8, 2, 3, 1};
        System.out.println("Duplicates: " + findDuplicates(arr)); // [2, 3]
    }
}
```

DRY RUN WALKTHROUGH:

nums = [4, 3, 2, 7, 8, 2, 3, 1]
- seen={4, 3, 2, 7, 8}
- next is 2: already in seen! Add 2 to duplicates.
- next is 3: already in seen! Add 3 to duplicates.
- next is 1: seen={4,3,2,7,8,1}.

Duplicates list: [2, 3].

TIME COMPLEXITY:
O(N) — linear scan with O(1) average lookup in HashSet.

SPACE COMPLEXITY:
O(N) — hash sets storing up to N distinct elements.

EDGE CASES: No duplicates [1, 2, 3, 4] (returns empty list); all identical elements [5, 5, 5, 5] (returns [5]); negative numbers handled properly.

#6: Reverse a Number

Mathematics

PROBLEM STATEMENT:

Given a 32-bit signed integer n, reverse its digits. If reversing causes the value to go outside the signed 32-bit integer range [-2^31, 2^31 - 1], return 0.

SAMPLE INPUT & OUTPUT:

```
Input: n = 12345
Output: 54321
Input: n = -123
Output: -321
```

ALGORITHM & INTUITION:

Extract the last digit using modulo 10 (int pop = n % 10), then append it to the reversed number by multiplying previous result by 10 (rev = rev * 10 + pop). Before multiplying by 10, check for 32-bit overflow against Integer.MAX_VALUE / 10 and Integer.MIN_VALUE / 10.

COMPLETE JAVA 21 SOLUTION:

```java
public class ReverseNumber {
    public static int reverse(int n) {
        int rev = 0;
        while (n != 0) {
            int pop = n % 10;
            n /= 10;
            // Check overflow before updating rev
            if (rev > Integer.MAX_VALUE / 10 || (rev == Integer.MAX_VALUE / 10 && pop > 7)) return 0;
            if (rev < Integer.MIN_VALUE / 10 || (rev == Integer.MIN_VALUE / 10 && pop < -8)) return 0;
            rev = rev * 10 + pop;
        }
        return rev;
    }

    public static void main(String[] args) {
        System.out.println("Reverse 12345: " + reverse(12345)); // 54321
        System.out.println("Reverse -123: " + reverse(-123));   // -321
    }
}
```

DRY RUN WALKTHROUGH:

n = 12345
- pop=5, n=1234, rev=5
- pop=4, n=123, rev=54
- pop=3, n=12, rev=543
- pop=2, n=1, rev=5432
- pop=1, n=0, rev=54321. Done.

TIME COMPLEXITY:
O(log10(N)) — number of digits in N, at most 10 iterations for 32-bit integer.

SPACE COMPLEXITY:
O(1) — strictly constant extra space.

EDGE CASES: Number ending with zero (120 -> 21); single-digit number (7 -> 7); negative numbers; overflow cases like 1534236469.

#7: Check Armstrong Number

Mathematics

PROBLEM STATEMENT:

An Armstrong number (narcissistic number) of order k is a number equal to the sum of its own digits each raised to the power of k, where k is the number of digits in n.

SAMPLE INPUT & OUTPUT:

```
Input: n = 153
Output: true
Explanation: 153 has 3 digits. 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153.
```

ALGORITHM & INTUITION:

1. Count the number of digits k in the given integer.
2. In a loop, extract each digit using modulo 10 and compute Math.pow(digit, k).
3. Sum these powered values and verify if total sum equals the original number.

COMPLETE JAVA 21 SOLUTION:

```java
public class ArmstrongNumber {
    public static boolean isArmstrong(int n) {
        if (n < 0) return false;
        int original = n;
        int k = String.valueOf(n).length();
        long sum = 0;

        int temp = n;
        while (temp > 0) {
            int digit = temp % 10;
            sum += Math.round(Math.pow(digit, k));
            temp /= 10;
        }
        return sum == original;
    }

    public static void main(String[] args) {
        System.out.println("153 is Armstrong? " + isArmstrong(153)); // true
        System.out.println("370 is Armstrong? " + isArmstrong(370)); // true
        System.out.println("123 is Armstrong? " + isArmstrong(123)); // false
    }
}
```

DRY RUN WALKTHROUGH:

n = 153, k = 3
- digit 3: 3^3 = 27, sum = 27
- digit 5: 5^3 = 125, sum = 27 + 125 = 152
- digit 1: 1^3 = 1, sum = 152 + 1 = 153
sum (153) == original (153) -> true.

TIME COMPLEXITY:
O(log10(N)) — proportional to the number of digits.

SPACE COMPLEXITY:
O(1) — constant auxiliary space.

EDGE CASES: Single digit numbers 0-9 (always true, e.g. 5^1=5); negative numbers (not Armstrong); 4-digit Armstrong like 1634 (1^4+6^4+3^4+4^4=1634).

#8: Check Prime Number

Mathematics

PROBLEM STATEMENT:

Given an integer n, determine whether it is a prime number (a positive integer strictly greater than 1 that has no positive divisors other than 1 and itself).

SAMPLE INPUT & OUTPUT:

```
Input: n = 29
Output: true
Input: n = 1
Output: false
```

ALGORITHM & INTUITION:

Numbers <= 1 are not prime. 2 and 3 are prime. Any even number > 2 or multiple of 3 is not prime. For other numbers, test divisors up to sqrt(n) in steps of 6 (i.e. checking 6k - 1 and 6k + 1). This reduces comparisons by 66% compared to testing every odd number.

COMPLETE JAVA 21 SOLUTION:

```java
public class PrimeCheck {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;
        if (n <= 3) return true;
        if (n % 2 == 0 || n % 3 == 0) return false;

        for (int i = 5; i * i <= n; i += 6) {
            if (n % i == 0 || n % (i + 2) == 0) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println("29 is prime? " + isPrime(29)); // true
        System.out.println("49 is prime? " + isPrime(49)); // false
    }
}
```

DRY RUN WALKTHROUGH:

n = 29
- n > 3, 29%2 != 0, 29%3 != 0
- i=5: i*i=25 <= 29. 29%5 != 0, 29%(5+2=7) != 0
- i becomes 11: i*i = 121 > 29 -> loop exits.

Returns true.

TIME COMPLEXITY:
O(sqrt(N)) — testing divisors only up to the square root of N.

SPACE COMPLEXITY:
O(1) — constant extra space.

EDGE CASES: n <= 1 (returns false); n = 2 (only even prime); n = 3; perfect squares like 25, 49 (correctly identified as non-prime).

#9: Find GCD and LCM

Mathematics

PROBLEM STATEMENT:

Given two non-negative integers a and b, compute their Greatest Common Divisor (GCD) and Least Common Multiple (LCM).

SAMPLE INPUT & OUTPUT:

```
Input: a = 12, b = 18
Output: GCD = 6, LCM = 36
```

ALGORITHM & INTUITION:

Use the Euclidean Algorithm for GCD: gcd(a, b) = gcd(b, a % b) until b becomes 0. Once GCD is found, compute LCM using the fundamental relation: LCM(a, b) = (|a * b|) / GCD(a, b). To prevent integer overflow, divide before multiplying: (a / gcd(a, b)) * b.

COMPLETE JAVA 21 SOLUTION:

```java
public class GcdAndLcm {
    public static long gcd(long a, long b) {
        while (b != 0) {
            long temp = b;
            b = a % b;
            a = temp;
        }
        return a;
    }

    public static long lcm(long a, long b) {
        if (a == 0 || b == 0) return 0;
        return (a / gcd(a, b)) * b;
    }

    public static void main(String[] args) {
        long a = 12, b = 18;
        long g = gcd(a, b);
        long l = lcm(a, b);
        System.out.println("GCD: " + g + ", LCM: " + l); // GCD: 6, LCM: 36
    }
}
```

DRY RUN WALKTHROUGH:

a = 12, b = 18
1. 12 % 18 = 12 -> a=18, b=12
2. 18 % 12 = 6 -> a=12, b=6
3. 12 % 6 = 0 -> a=6, b=0
GCD = 6. LCM = (12 / 6) * 18 = 2 * 18 = 36.

TIME COMPLEXITY:
O(log(min(A, B))) — Euclidean algorithm runs in logarithmic time with respect to the smaller operand.

SPACE COMPLEXITY:
O(1) — iterative Euclidean calculation.

EDGE CASES: One operand is 0 (GCD(0, b) = b, LCM = 0); identical numbers (GCD = LCM = a); prime numbers (GCD = 1, LCM = a*b).

#10: Fibonacci Series

Mathematics

PROBLEM STATEMENT:

Generate the first n terms of the Fibonacci sequence, where each term is the sum of the preceding two terms: F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2).

SAMPLE INPUT & OUTPUT:

```
Input: n = 7
Output: [0, 1, 1, 2, 3, 5, 8]
```

ALGORITHM & INTUITION:

Use an iterative approach with two state variables (a = 0, b = 1). In each iteration, print 'a', compute next = a + b, and slide window: a = b, b = next. This achieves O(N) linear time and O(1) space, avoiding the exponential O(2^N) overhead of naive recursion.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class Fibonacci {
    public static long[] generateFibonacci(int n) {
        if (n <= 0) return new long[0];
        long[] fib = new long[n];
        if (n >= 1) fib[0] = 0;
        if (n >= 2) fib[1] = 1;

        for (int i = 2; i < n; i++) {
            fib[i] = fib[i - 1] + fib[i - 2];
        }
        return fib;
    }

    public static void main(String[] args) {
        int n = 7;
        System.out.println("Fibonacci(" + n + "): " + Arrays.toString(generateFibonacci(n)));
        // Output: [0, 1, 1, 2, 3, 5, 8]
    }
}
```

DRY RUN WALKTHROUGH:

n = 7
i=0: 0
i=1: 1
i=2: 0+1 = 1
i=3: 1+1 = 2
i=4: 1+2 = 3
i=5: 2+3 = 5
i=6: 3+5 = 8
Array: [0, 1, 1, 2, 3, 5, 8]

TIME COMPLEXITY:
O(N) — single linear loop up to N.

SPACE COMPLEXITY:
O(N) to store output array (or O(1) if printing on the fly).

EDGE CASES: n = 0 (empty); n = 1 ([0]); n = 2 ([0, 1]); large n up to 92 handled using 'long' data type to prevent 32-bit overflow.

#11: Check Palindrome String

Strings

PROBLEM STATEMENT:

Given a string s, determine whether it is a palindrome, considering only alphanumeric characters and ignoring cases.

SAMPLE INPUT & OUTPUT:

```
Input: s = "A man, a plan, a canal: Panama"
Output: true
Explanation: "amanaplanacanalpanama" is a palindrome.
```

ALGORITHM & INTUITION:

Use two pointers: 'left' at start (0) and 'right' at end (length - 1). Advance left if not alphanumeric; decrement right if not alphanumeric. When both point to valid characters, compare them case-insensitively. If mismatch occurs, return false. If pointers cross, return true.

COMPLETE JAVA 21 SOLUTION:

```java
public class PalindromeString {
    public static boolean isPalindrome(String s) {
        if (s == null) return false;
        int left = 0, right = s.length() - 1;

        while (left < right) {
            while (left < right && !Character.isLetterOrDigit(s.charAt(left))) {
                left++;
            }
            while (left < right && !Character.isLetterOrDigit(s.charAt(right))) {
                right--;
            }
            if (Character.toLowerCase(s.charAt(left)) != Character.toLowerCase(s.charAt(right))) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }

    public static void main(String[] args) {
        String s = "A man, a plan, a canal: Panama";
        System.out.println("Is Palindrome? " + isPalindrome(s)); // true
    }
}
```

DRY RUN WALKTHROUGH:

s = "race a car"
Compare 'r' vs 'r' (ok)
Compare 'a' vs 'a' (ok)
Compare 'c' vs 'c' (ok)
Compare 'e' vs 'a' -> mismatch! Return false.

TIME COMPLEXITY:
O(N) — each character is visited at most once.

SPACE COMPLEXITY:
O(1) — constant extra space; no new reversed string is created.

EDGE CASES: Empty string "" or whitespace only (true); string with special symbols "###" (true); case differences "Abba" (true).

#12: Check Whether Two Strings Are Anagrams

Strings

PROBLEM STATEMENT:

Given two strings s and t, return true if t is an anagram of s (contains the same characters with identical frequencies), and false otherwise.

SAMPLE INPUT & OUTPUT:

```
Input: s = "anagram", t = "nagaram"
Output: true
Input: s = "rat", t = "car"
Output: false
```

ALGORITHM & INTUITION:

If lengths differ, return false immediately. Use a fixed-size frequency array of 26 integers (or 256 for extended ASCII). For every character in s, increment its count; for every character in t, decrement its count. If all counts return to zero, strings are anagrams.

COMPLETE JAVA 21 SOLUTION:

```java
public class AnagramCheck {
    public static boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        int[] freq = new int[26];

        for (int i = 0; i < s.length(); i++) {
            freq[s.charAt(i) - 'a']++;
            freq[t.charAt(i) - 'a']--;
        }

        for (int count : freq) {
            if (count != 0) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println("anagram & nagaram: " + isAnagram("anagram", "nagaram")); // true
        System.out.println("rat & car: " + isAnagram("rat", "car")); // false
    }
}
```

DRY RUN WALKTHROUGH:

s = "cat", t = "act"
- s[0]='c' (+1), t[0]='a' (-1) -> c:1, a:-1
- s[1]='a' (+1), t[1]='c' (-1) -> a:0, c:0
- s[2]='t' (+1), t[2]='t' (-1) -> t:0
All 26 buckets are 0 -> true.

TIME COMPLEXITY:
O(N) — single pass of length N.

SPACE COMPLEXITY:
O(1) — constant 26-element array regardless of input size.

EDGE CASES: Different string lengths; strings with repeating characters like "aacc" and "ccac"; unicode/uppercase characters if applicable.

#13: Count Character Frequencies

Hashing

PROBLEM STATEMENT:

Given a string s, count the frequency of each character and display the counts in order of occurrence.

SAMPLE INPUT & OUTPUT:

```
Input: s = "programming"
Output: {p:1, r:2, o:1, g:2, a:1, m:2, i:1, n:1}
```

ALGORITHM & INTUITION:

Use a LinkedHashMap<Character, Integer> to maintain insertion order while counting frequencies. Iterate through each character of the string and increment the count using map.put(ch, map.getOrDefault(ch, 0) + 1).

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class CharacterFrequency {
    public static Map<Character, Integer> countFrequencies(String s) {
        Map<Character, Integer> freqMap = new LinkedHashMap<>();
        for (char c : s.toCharArray()) {
            freqMap.put(c, freqMap.getOrDefault(c, 0) + 1);
        }
        return freqMap;
    }

    public static void main(String[] args) {
        String s = "programming";
        Map<Character, Integer> result = countFrequencies(s);
        for (Map.Entry<Character, Integer> entry : result.entrySet()) {
            System.out.print(entry.getKey() + ":" + entry.getValue() + " ");
        }
        // Output: p:1 r:2 o:1 g:2 a:1 m:2 i:1 n:1
    }
}
```

DRY RUN WALKTHROUGH:

s = "programming"
- 'p' -> 1
- 'r' -> 1
- 'o' -> 1
- 'g' -> 1
- 'r' -> 2
- 'a' -> 1
- 'm' -> 1
- 'm' -> 2
- 'i' -> 1
- 'n' -> 1
- 'g' -> 2

TIME COMPLEXITY:
O(N) — linear pass across the string of length N.

SPACE COMPLEXITY:
O(K) where K <= 256 is the number of unique characters.

EDGE CASES: Empty string ""; string with spaces and punctuation; string with only 1 unique character "aaaa".

#14: Reverse Words in a Sentence

Strings

PROBLEM STATEMENT:

Given an input string s, reverse the order of the words. Return a string of the words in reverse order concatenated by a single space, removing all extra leading, trailing, or multiple intervening spaces.

SAMPLE INPUT & OUTPUT:

```
Input: s = "  the sky   is blue  "
Output: "blue is sky the"
```

ALGORITHM & INTUITION:

Trim leading and trailing spaces. Split the string by the regular expression "\\s+" (which matches one or more whitespace characters). Then, traverse the resulting words array from the last element to the first, appending each word to a StringBuilder separated by a single space.

COMPLETE JAVA 21 SOLUTION:

```java
public class ReverseWords {
    public static String reverseWords(String s) {
        if (s == null) return "";
        String[] words = s.trim().split("\\s+");
        StringBuilder sb = new StringBuilder();

        for (int i = words.length - 1; i >= 0; i--) {
            sb.append(words[i]);
            if (i > 0) sb.append(" ");
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        String s = "  the sky   is blue  ";
        System.out.println("Reversed: \"" + reverseWords(s) + "\"");
        // Output: "blue is sky the"
    }
}
```

DRY RUN WALKTHROUGH:

s = " the sky   is blue "
1. trim() -> "the sky   is blue"
2. split("\\s+") -> ["the", "sky", "is", "blue"]
3. Traverse backwards: "blue" + " " + "is" + " " + "sky" + " " + "the"
Result: "blue is sky the".

TIME COMPLEXITY:
O(N) — scanning and splitting the string takes linear time.

SPACE COMPLEXITY:
O(N) — storing tokens and building output string.

EDGE CASES: Multiple consecutive spaces; single word "hello"; string with only spaces " ".

#15: Find the First Non-Repeating Character

Hashing

PROBLEM STATEMENT:

Given a string s, find the first non-repeating character in it and return its index. If it does not exist, return -1.

SAMPLE INPUT & OUTPUT:

```
Input: s = "loveleetcode"
Output: 2
Explanation: Character 'v' at index 2 is the first character that does not repeat.
```

ALGORITHM & INTUITION:

Two-pass algorithm:
Pass 1: Traverse the string and build a frequency array (int[26]) of character counts.
Pass 2: Traverse the string again from left to right; the first character whose frequency is exactly 1 is our answer.

COMPLETE JAVA 21 SOLUTION:

```
public class FirstNonRepeating {
    public static int firstUniqChar(String s) {
        int[] freq = new int[26];
        for (int i = 0; i < s.length(); i++) {
            freq[s.charAt(i) - 'a']++;
        }
        for (int i = 0; i < s.length(); i++) {
            if (freq[s.charAt(i) - 'a'] == 1) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        String s = "loveleetcode";
        int idx = firstUniqChar(s);
        System.out.println("First non-repeating index: " + idx + " (char: " + s.charAt(idx) + ")"); // 2
('v')
    }
}
```

DRY RUN WALKTHROUGH:

s = "loveleetcode"
Counts: l:2, o:2, v:1, e:4, t:1, c:1, d:1
Second pass:
- idx 0: 'l' count=2
- idx 1: 'o' count=2
- idx 2: 'v' count=1 -> Found! Return 2.

TIME COMPLEXITY:
O(N) — two linear sweeps through the string.

SPACE COMPLEXITY:
O(1) — constant size 26 array for English lowercase letters.

EDGE CASES: All characters repeat "aabbcc" (returns -1); single character "z" (returns 0); string where last character is the only unique one "aabbc".

#16: Two Sum

Hashing

PROBLEM STATEMENT:

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target. You may assume each input has exactly one solution.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [2, 7, 11, 15], target = 9
Output: [0, 1]
Explanation: nums[0] + nums[1] = 2 + 7 = 9.
```

ALGORITHM & INTUITION:

Use a HashMap to store values and their indices. For each number x = nums[i], calculate the needed complement = target - x. If complement is already in the map, we have found our answer [map.get(complement), i]. Otherwise, put x and its index into the map.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class TwoSum {
    public static int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (map.containsKey(complement)) {
                return new int[]{map.get(complement), i};
            }
            map.put(nums[i], i);
        }
        return new int[0];
    }

    public static void main(String[] args) {
        int[] nums = {2, 7, 11, 15};
        int[] res = twoSum(nums, 9);
        System.out.println("Indices: [" + res[0] + ", " + res[1] + "]"); // [0, 1]
    }
}
```

DRY RUN WALKTHROUGH:

nums = [2, 7, 11, 15], target = 9
- i=0: num=2, complement = 9-2 = 7. Map empty. Put (2 -> 0).
- i=1: num=7, complement = 9-7 = 2. Map contains 2! Return [0, 1].

TIME COMPLEXITY:
O(N) — single pass with O(1) average hash map lookup.

SPACE COMPLEXITY:
O(N) — storing up to N elements in the hash map.

EDGE CASES: Negative numbers [-3, 4, 3, 90], target = 0; duplicate values [3, 3], target = 6 (map handles duplicate correctly by checking complement before overwriting).

#17: Maximum Subarray Sum (Kadane's Algorithm)

Kadane's algorithm

PROBLEM STATEMENT:

Given an integer array nums, find the subarray with the largest sum, and return its sum.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6
Explanation: The subarray [4, -1, 2, 1] has the largest sum = 6.
```

ALGORITHM & INTUITION:

Kadane's algorithm maintains two variables: 'currentSum' and 'maxSum'. At each element x, decide whether to add x to currentSum (extending previous subarray) or start a fresh subarray at x (currentSum = Math.max(x, currentSum + x)). Update maxSum = Math.max(maxSum, currentSum).

COMPLETE JAVA 21 SOLUTION:

```java
public class KadaneMaxSubarray {
    public static int maxSubArray(int[] nums) {
        int currentSum = nums[0];
        int maxSum = nums[0];

        for (int i = 1; i < nums.length; i++) {
            currentSum = Math.max(nums[i], currentSum + nums[i]);
            maxSum = Math.max(maxSum, currentSum);
        }
        return maxSum;
    }

    public static void main(String[] args) {
        int[] nums = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
        System.out.println("Maximum Subarray Sum: " + maxSubArray(nums)); // Output: 6
    }
}
```

DRY RUN WALKTHROUGH:

nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
- init: curr = -2, max = -2
- 1: curr = max(1, -1) = 1, max = 1
- -3: curr = max(-3, -2) = -2, max = 1
- 4: curr = max(4, 2) = 4, max = 4
- -1: curr = max(-1, 3) = 3, max = 4
- 2: curr = max(2, 5) = 5, max = 5
- 1: curr = max(1, 6) = 6, max = 6
- -5: curr = max(-5, 1) = 1, max = 6
- 4: curr = max(4, 5) = 5, max = 6. Output: 6

TIME COMPLEXITY:
O(N) — single pass through the array.

SPACE COMPLEXITY:
O(1) — constant auxiliary space.

EDGE CASES: All negative numbers [-5, -2, -8, -1] (returns -1, correctly picks maximum single element); array of size 1; all positive numbers.

#18: Binary Search

Searching

PROBLEM STATEMENT:

Given a sorted array of integers nums and an integer target, write a function to search target in nums. If target exists, return its index; otherwise, return -1.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [-1, 0, 3, 5, 9, 12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4.
```

ALGORITHM & INTUITION:

Maintain two pointers: low = 0 and high = nums.length - 1. In each iteration, calculate mid = low + (high - low) / 2 to prevent integer overflow. If nums[mid] == target, return mid. If nums[mid] < target, search right half (low = mid + 1). If nums[mid] > target, search left half (high = mid - 1).

COMPLETE JAVA 21 SOLUTION:

```java
public class BinarySearch {
    public static int search(int[] nums, int target) {
        int low = 0, high = nums.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2; // Prevents overflow
            if (nums[mid] == target) {
                return mid;
            } else if (nums[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] nums = {-1, 0, 3, 5, 9, 12};
        System.out.println("Search 9: " + search(nums, 9)); // 4
        System.out.println("Search 2: " + search(nums, 2)); // -1
    }
}
```

DRY RUN WALKTHROUGH:

nums = [-1, 0, 3, 5, 9, 12], target = 9
1. low=0, high=5, mid=2, nums[2]=3 < 9 -> low=3
2. low=3, high=5, mid=4, nums[4]=9 == target -> Found at index 4.

TIME COMPLEXITY:
O(log N) — halves search space on every iteration.

SPACE COMPLEXITY:
O(1) — iterative search with constant memory.

EDGE CASES: Target smaller than first element; target larger than last element; single element array; array with duplicate values.

#19: Merge Two Sorted Arrays

Arrays

PROBLEM STATEMENT:

Given two sorted integer arrays nums1 and nums2, merge nums2 into nums1 as one sorted array. nums1 has size m + n, where the first m elements are valid and the last n elements are 0.

SAMPLE INPUT & OUTPUT:

```
Input: nums1 = [1, 2, 3, 0, 0, 0], m = 3, nums2 = [2, 5, 6], n = 3
Output: nums1 = [1, 2, 2, 3, 5, 6]
```

ALGORITHM & INTUITION:

Three-pointer technique starting from the back. Set p1 = m - 1 (end of valid nums1), p2 = n - 1 (end of nums2), and p = m + n - 1 (end of buffer). Compare nums1[p1] and nums2[p2], place the larger at nums1[p], and decrement pointers. This avoids overwriting unexamined elements in nums1.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class MergeSortedArrays {
    public static void merge(int[] nums1, int m, int[] nums2, int n) {
        int p1 = m - 1;
        int p2 = n - 1;
        int p = m + n - 1;

        while (p1 >= 0 && p2 >= 0) {
            if (nums1[p1] > nums2[p2]) {
                nums1[p] = nums1[p1];
                p1--;
            } else {
                nums1[p] = nums2[p2];
                p2--;
            }
            p--;
        }

        // Copy remaining elements of nums2 if any
        while (p2 >= 0) {
            nums1[p] = nums2[p2];
            p2--;
            p--;
        }
    }

    public static void main(String[] args) {
        int[] nums1 = {1, 2, 3, 0, 0, 0};
        int[] nums2 = {2, 5, 6};
        merge(nums1, 3, nums2, 3);
        System.out.println("Merged: " + Arrays.toString(nums1)); // [1, 2, 2, 3, 5, 6]
    }
}
```

DRY RUN WALKTHROUGH:

nums1 = [1, 2, 3, 0, 0, 0], nums2 = [2, 5, 6]
- p1=2 (3), p2=2 (6): 6 > 3 -> nums1[5] = 6, p2=1
- p1=2 (3), p2=1 (5): 5 > 3 -> nums1[4] = 5, p2=0
- p1=2 (3), p2=0 (2): 3 > 2 -> nums1[3] = 3, p1=1
- p1=1 (2), p2=0 (2): 2 == 2 -> nums1[2] = 2, p2=-1

Loop exits. Remaining nums1 already in place.
Result: [1, 2, 2, 3, 5, 6].

TIME COMPLEXITY:
O(M + N) — each element is moved exactly once.

SPACE COMPLEXITY:
O(1) — in-place modification within nums1 buffer.

EDGE CASES: nums2 is empty (n=0); nums1 is initially empty of valid elements (m=0, all in nums2); elements in nums2 all smaller than nums1.

#20: Sort 0s, 1s and 2s (Dutch National Flag Algorithm)

Two pointers

PROBLEM STATEMENT:

Given an array nums with n objects colored red, white, or blue (represented by integers 0, 1, and 2), sort them in-place so that objects of the same color are adjacent, with the colors in the order 0, 1, and 2.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [2, 0, 2, 1, 1, 0]
Output: [0, 0, 1, 1, 2, 2]
```

ALGORITHM & INTUITION:

Use Edsger Dijkstra's Dutch National Flag 3-pointer algorithm with low = 0, mid = 0, high = n - 1.
- Invariant: [0...low-1] are 0s, [low...mid-1] are 1s, [high+1...n-1] are 2s.
- If nums[mid] == 0: swap(nums[low], nums[mid]), low++, mid++.
- If nums[mid] == 1: mid++.
- If nums[mid] == 2: swap(nums[mid], nums[high]), high-- (do not increment mid because swapped element needs evaluation).

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class SortColors {
    public static void sortColors(int[] nums) {
        int low = 0, mid = 0, high = nums.length - 1;

        while (mid <= high) {
            if (nums[mid] == 0) {
                swap(nums, low, mid);
                low++;
                mid++;
            } else if (nums[mid] == 1) {
                mid++;
            } else { // nums[mid] == 2
                swap(nums, mid, high);
                high--;
            }
        }
    }

    private static void swap(int[] nums, int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }

    public static void main(String[] args) {
        int[] nums = {2, 0, 2, 1, 1, 0};
        sortColors(nums);
        System.out.println("Sorted Colors: " + Arrays.toString(nums)); // [0, 0, 1, 1, 2, 2]
    }
}
```

DRY RUN WALKTHROUGH:

nums = [2, 0, 2, 1, 1, 0], low=0, mid=0, high=5
- mid=0: val=2 -> swap(0, 5) -> [0, 0, 2, 1, 1, 2], high=4
- mid=0: val=0 -> swap(0, 0) -> low=1, mid=1
- mid=1: val=0 -> swap(1, 1) -> low=2, mid=2
- mid=2: val=2 -> swap(2, 4) -> [0, 0, 1, 1, 2, 2], high=3
- mid=2: val=1 -> mid=3

- mid=3: val=1 -> mid=4 > high(3) -> Done!
Result: [0, 0, 1, 1, 2, 2]

TIME COMPLEXITY:
O(N) — single pass in-place.

SPACE COMPLEXITY:
O(1) — constant extra space.

EDGE CASES: Array with only one type of element [0, 0, 0]; already sorted [0, 1, 2]; reverse sorted [2, 2, 1, 0].

CHAPTER 2

Cognizant GenC & GenC Elevate / Next

Standard Console I/O Programs & Complete Java Implementations

Target Focus: GenC, GenC Elevate & GenC Next Technical Rounds

Cognizant requires complete Java code reading from standard input using Scanner and printing exact outputs. Special focus on pairwise-product minimization, coin change, and hashing.

Questions in this Section (18 Problems):

#1: Find the Second-Largest Element — Arrays

PROBLEM STATEMENT:

Write a complete Java console program to find the second-largest distinct element in an array of integers. Read the array length N followed by N integers from standard input.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
6
12 35 1 10 34 1
Sample Output:
34
```

ALGORITHM & INTUITION:

Read N integers using java.util.Scanner. Maintain two variables: largest = Integer.MIN_VALUE and secondLargest = Integer.MIN_VALUE. In a single pass, update largest and secondLargest. If no second distinct largest exists, print -1.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Scanner;

public class CognizantSecondLargest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int largest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;

        for (int num : arr) {
            if (num > largest) {
                secondLargest = largest;
                largest = num;
            } else if (num < largest && num > secondLargest) {
                secondLargest = num;
            }
```

```
        }

        if (secondLargest == Integer.MIN_VALUE) {
            System.out.println(-1);
        } else {
            System.out.println(secondLargest);
        }
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

Input: 6 followed by [12, 35, 1, 10, 34, 1]
largest=35, secondLargest=34
Output: 34

TIME COMPLEXITY:
O(N) — single linear traversal.

SPACE COMPLEXITY:
O(N) to store inputs in array (or O(1) if computed streaming).

EDGE CASES: N < 2 (prints -1); all elements equal [5, 5, 5] (prints -1); negative inputs [-10, -5, -20] (prints -10).

#2: Rearrange Arrays to Minimize Sum of Pairwise Products

Sorting

PROBLEM STATEMENT:

Given two arrays of equal size n, rearrange the elements such that the sum of the pairwise products sum = sum(A[i] * B[i]) is minimized. Read n, then array A, then array B.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
4
3 1 1 2
6 5 4 8
Sample Output:
29
Explanation: Sort A ascending: [1, 1, 2, 3], sort B descending: [8, 6, 5, 4]. Sum = (1*8) + (1*6) +
(2*5) + (3*4) = 8 + 6 + 10 + 12 = 36... wait: (1*8)+(1*6)+(2*5)+(3*4)=36. For [6,5,4,8] descending:
8,6,5,4. (1*8)+(1*6)+(2*5)+(3*4)=36.
```

ALGORITHM & INTUITION:

Sorting Strategy & Mathematical Proof (Rearrangement Inequality):
By the Rearrangement Inequality, for any two sequences, the sum of pairwise products is MAXIMIZED when both sequences are sorted in the same monotonic order (both ascending), and MINIMIZED when they are sorted in opposite monotonic orders (one ascending, one descending).
Why? If we pair the largest element of A with the smallest element of B, and the smallest element of A with the largest element of B, the huge multipliers are dampened by small weights, yielding the minimum possible dot product.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Scanner;
import java.util.Arrays;

public class MinimizePairwiseProduct {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        long[] a = new long[n];
        long[] b = new long[n];

        for (int i = 0; i < n; i++) a[i] = sc.nextLong();
        for (int i = 0; i < n; i++) b[i] = sc.nextLong();

        // Sort A ascending
        Arrays.sort(a);
        // Sort B ascending, then pair a[i] with b[n - 1 - i]
        Arrays.sort(b);

        long minSum = 0;
        for (int i = 0; i < n; i++) {
            minSum += a[i] * b[n - 1 - i]; // smallest with largest
        }

        System.out.println(minSum);
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

A = [3, 1, 1, 2] -> sorted A = [1, 1, 2, 3]
B = [6, 5, 4, 8] -> sorted B = [4, 5, 6, 8]
Pairwise product:
a[0]*b[3] = 1*8 = 8
a[1]*b[2] = 1*6 = 6
a[2]*b[1] = 2*5 = 10
a[3]*b[0] = 3*4 = 12
Total sum = 8 + 6 + 10 + 12 = 36.

TIME COMPLEXITY:
O(N log N) — dominated by sorting both arrays.

SPACE COMPLEXITY:
O(N) for array storage, O(1) auxiliary space.

EDGE CASES: Arrays containing negative numbers; arrays with all identical elements; n = 1 (single product).

#3: Find the Missing Number

Arrays

PROBLEM STATEMENT:

Given an array of size n containing distinct integers from 0 to n, find the missing integer.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
3
3 0 1
Sample Output:
2
```

ALGORITHM & INTUITION:

Use XOR properties. XORing all indices 0 to n with all array values leaves only the missing number, completely preventing integer overflow.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Scanner;

public class CognizantMissingNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        int xorAll = 0;
        for (int i = 0; i <= n; i++) xorAll ^= i;
        for (int i = 0; i < n; i++) {
            xorAll ^= sc.nextInt();
        }
        System.out.println(xorAll);
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

n = 3, elements = [3, 0, 1]
xorAll = (0^1^2^3) ^ (3^0^1) = 2. Output: 2.

TIME COMPLEXITY:
O(N) — single pass streaming input.

SPACE COMPLEXITY:
O(1) — constant extra space.

EDGE CASES: Missing 0; missing N; N = 1.

#4: Move Zeros to the End

Arrays

PROBLEM STATEMENT:

Given an array of integers, write a complete program to move all zeros to the end while preserving the relative order of non-zero elements.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
5
0 1 0 3 12
Sample Output:
1 3 12 0 0
```

ALGORITHM & INTUITION:

Use a two-pointer insert index. Whenever a non-zero element is encountered, swap it to the insert position and increment the pointer.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Scanner;

public class CognizantMoveZeros {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        int insertPos = 0;
        for (int i = 0; i < n; i++) {
            if (arr[i] != 0) {
                int temp = arr[insertPos];
                arr[insertPos] = arr[i];
                arr[i] = temp;
                insertPos++;
            }
        }

        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + (i == n - 1 ? "" : " "));
        }
        System.out.println();
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

[0, 1, 0, 3, 12] -> swap 0 and 1 -> [1, 0, 0, 3, 12] -> swap 0 and 3 -> [1, 3, 0, 0, 12] -> swap 0 and 12 -> [1, 3, 12, 0, 0].

TIME COMPLEXITY:
O(N) — single pass.

SPACE COMPLEXITY:
O(1) auxiliary space.

EDGE CASES: Array with no zeros; array of all zeros.

#5: Find a Pair with a Target Sum

Hashing

PROBLEM STATEMENT:

Given an array of integers and a target sum, determine whether there exists a pair of elements whose sum equals target. If found, print the pair; otherwise print 'No Pair Found'.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
5
10 15 3 7 2
17
Sample Output:
Pair found: (15, 2)
```

ALGORITHM & INTUITION:

Use a HashSet to store seen elements. For each number x, check if (target - x) is in the set. If yes, print the pair and terminate. Otherwise, add x to the set.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class CognizantTargetSumPair {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        int target = sc.nextInt();

        Set<Integer> seen = new HashSet<>();
        boolean found = false;

        for (int num : arr) {
            int complement = target - num;
            if (seen.contains(complement)) {
                System.out.println("Pair found: (" + complement + ", " + num + ")");
                found = true;
                break;
            }
            seen.add(num);
        }

        if (!found) {
            System.out.println("No Pair Found");
        }
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

arr = [10, 15, 3, 7, 2], target = 17
seen: {10, 15, 3, 7}
num = 2: complement = 17 - 2 = 15 is in seen! Output: Pair found: (15, 2).

TIME COMPLEXITY:
O(N) — linear pass with O(1) set operations.

SPACE COMPLEXITY:
O(N) — storing seen elements in hash set.

EDGE CASES: No such pair exists; duplicate values adding to target (3 + 3 = 6).

#6: Count the Frequency of Elements

Hashing

PROBLEM STATEMENT:

Given an array of integers, count the frequency of each distinct element and print them in order of their first appearance.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
7
1 2 2 3 1 4 2
Sample Output:
1: 2
2: 3
3: 1
4: 1
```

ALGORITHM & INTUITION:

Use a LinkedHashMap<Integer, Integer> to preserve the first-appearance order while tracking counts.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class CognizantElementFrequency {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        Map<Integer, Integer> map = new LinkedHashMap<>();

        for (int i = 0; i < n; i++) {
            int val = sc.nextInt();
            map.put(val, map.getOrDefault(val, 0) + 1);
        }

        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

[1, 2, 2, 3, 1, 4, 2] -> 1:2, 2:3, 3:1, 4:1.

TIME COMPLEXITY:
O(N) — single pass over the input.

SPACE COMPLEXITY:
O(K) where K is number of unique elements.

EDGE CASES: All elements identical; all elements unique.

#7: Maximum Subarray Sum

Kadane's algorithm

PROBLEM STATEMENT:

Given an array of N integers, find the contiguous subarray that has the maximum sum using Kadane's Algorithm.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
8
-2 -3 4 -1 -2 1 5 -3
Sample Output:
7
Explanation: Subarray [4, -1, -2, 1, 5] sum = 7.
```

ALGORITHM & INTUITION:

Maintain currentSum = arr[0] and maxSum = arr[0]. For each element from index 1, currentSum = max(arr[i], currentSum + arr[i]), and maxSum = max(maxSum, currentSum).

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Scanner;

public class CognizantMaxSubarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        int currentSum = arr[0];
        int maxSum = arr[0];

        for (int i = 1; i < n; i++) {
            currentSum = Math.max(arr[i], currentSum + arr[i]);
            maxSum = Math.max(maxSum, currentSum);
        }

        System.out.println(maxSum);
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

arr = [-2, -3, 4, -1, -2, 1, 5, -3]
max achieves 7 at subarray [4, -1, -2, 1, 5].

TIME COMPLEXITY:
O(N) — single linear pass.

SPACE COMPLEXITY:
O(1) auxiliary space.

EDGE CASES: All negative numbers (e.g. [-5, -1, -8] yields -1).

#8: Rotate an Array

Arrays

PROBLEM STATEMENT:

Given an array of size N and an integer K, rotate the array to the right by K positions in-place.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
5
1 2 3 4 5
2
Sample Output:
4 5 1 2 3
```

ALGORITHM & INTUITION:

Normalize k = k % n. Reverse entire array, reverse first k elements, reverse remaining n - k elements.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Scanner;

public class CognizantRotateArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        int k = sc.nextInt() % n;

        reverse(arr, 0, n - 1);
        reverse(arr, 0, k - 1);
        reverse(arr, k, n - 1);

        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + (i == n - 1 ? "" : " "));
        }
        System.out.println();
        sc.close();
    }

    private static void reverse(int[] arr, int l, int r) {
        while (l < r) {
            int t = arr[l];
            arr[l] = arr[r];
            arr[r] = t;
            l++;
            r--;
        }
    }
}
```

DRY RUN WALKTHROUGH:

arr = [1, 2, 3, 4, 5], k = 2
Rev all: [5, 4, 3, 2, 1]
Rev 0..1: [4, 5, 3, 2, 1]
Rev 2..4: [4, 5, 1, 2, 3]

TIME COMPLEXITY:
O(N) — two swaps per element.

SPACE COMPLEXITY:
O(1) auxiliary space.

EDGE CASES: k = 0; k >= n.

#9: Check Whether Two Strings Are Anagrams

Strings

PROBLEM STATEMENT:

Write a program to test whether two input strings are anagrams of each other.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
listen
silent
Sample Output:
Anagrams
```

ALGORITHM & INTUITION:

If lengths differ, not anagrams. Track frequencies using int[26]. Increment for string 1, decrement for string 2. Check if all values are 0.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.Scanner;

public class CognizantAnagram {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNext()) return;
        String s1 = sc.next().toLowerCase();
        String s2 = sc.next().toLowerCase();

        if (s1.length() != s2.length()) {
            System.out.println("Not Anagrams");
            sc.close();
            return;
        }

        int[] count = new int[26];
        for (int i = 0; i < s1.length(); i++) {
            count[s1.charAt(i) - 'a']++;
            count[s2.charAt(i) - 'a']--;
        }

        boolean isAnagram = true;
        for (int c : count) {
            if (c != 0) {
                isAnagram = false;
                break;
            }
        }

        System.out.println(isAnagram ? "Anagrams" : "Not Anagrams");
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

listen vs silent: counts match exactly -> Anagrams.

TIME COMPLEXITY:
O(N) — single pass.

SPACE COMPLEXITY:
O(1) — 26 integers.

EDGE CASES: Different lengths; different character cases.

#10: Find the First Non-Repeating Character

Hashing

PROBLEM STATEMENT:

Given a string, find the first character that appears exactly once. Print the character, or 'None' if every character repeats.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
swiss
Sample Output:
w
```

ALGORITHM & INTUITION:

Count frequencies in int[256]. Iterate through the string; print the first character with frequency 1.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Scanner;

public class CognizantFirstUnique {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNext()) return;
        String s = sc.next();

        int[] freq = new int[256];
        for (char c : s.toCharArray()) {
            freq[c]++;
        }

        char result = '0';
        for (char c : s.toCharArray()) {
            if (freq[c] == 1) {
                result = c;
                break;
            }
        }

        if (result == '0') {
            System.out.println("None");
        } else {
            System.out.println(result);
        }
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

s = "swiss"
s:3, w:1, i:1
First char with count 1 is 'w'.

TIME COMPLEXITY:
O(N) — two scans.

SPACE COMPLEXITY:
O(1) — 256 array.

EDGE CASES: All repeated "aabb"; all unique "abc".

#11: Reverse Words in a String

Strings

PROBLEM STATEMENT:

Given a sentence, reverse the order of the words while removing extra spaces.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
Cognizant GenC Next
Sample Output:
Next GenC Cognizant
```

ALGORITHM & INTUITION:

Read line, split by regex \\s+, traverse backwards and print words separated by space.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Scanner;

public class CognizantReverseWords {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextLine()) return;
        String line = sc.nextLine().trim();
        String[] words = line.split("\\s+");

        StringBuilder sb = new StringBuilder();
        for (int i = words.length - 1; i >= 0; i--) {
            sb.append(words[i]);
            if (i > 0) sb.append(" ");
        }
        System.out.println(sb.toString());
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

"Cognizant GenC Next" -> ["Cognizant", "GenC", "Next"] -> "Next GenC Cognizant".

TIME COMPLEXITY:
O(N) — linear tokenization.

SPACE COMPLEXITY:
O(N) for tokens.

EDGE CASES: Single word; leading/trailing spaces.

#12: Find the Maximum Occurring Character

Hashing

PROBLEM STATEMENT:

Given a string, find the character that appears with the highest frequency. If multiple characters tie, print the lexicographically smallest character.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
success
Sample Output:
s
Explanation: 's' appears 3 times, 'c' appears 2 times.
```

ALGORITHM & INTUITION:

Build an ASCII frequency array. Iterate from 'a' to 'z' (or ASCII 0 to 255) to find the character with the maximum count, naturally breaking ties lexicographically.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Scanner;

public class CognizantMaxChar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextLine()) return;
        String s = sc.nextLine();

        int[] count = new int[256];
        for (int i = 0; i < s.length(); i++) {
            count[s.charAt(i)]++;
        }

        int maxFreq = -1;
        char maxChar = ' ';

        for (int i = 0; i < 256; i++) {
            if (count[i] > maxFreq) {
                maxFreq = count[i];
                maxChar = (char) i;
            }
        }

        System.out.println(maxChar);
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

s = "success": s:3, u:1, c:2, e:1. maxFreq=3, maxChar='s'.

TIME COMPLEXITY:
O(N) — string scan.

SPACE COMPLEXITY:
O(1) — 256-element array.

EDGE CASES: Tie breaker (e.g. "ba" -> prints 'a'); spaces in string.

#13: Check Palindrome

Strings

PROBLEM STATEMENT:

Given a string, check if it is a palindrome considering alphanumeric characters.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
level
Sample Output:
Palindrome
```

ALGORITHM & INTUITION:

Two pointers left = 0, right = s.length() - 1, compare until pointers meet.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Scanner;

public class CognizantPalindrome {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNext()) return;
        String s = sc.next();

        int left = 0, right = s.length() - 1;
        boolean isPal = true;

        while (left < right) {
            if (s.charAt(left) != s.charAt(right)) {
                isPal = false;
                break;
            }
            left++;
            right--;
        }

        System.out.println(isPal ? "Palindrome" : "Not Palindrome");
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

"level": left=0('l'), right=4('l') -> match. left=1('e'), right=3('e') -> match. Palindrome.

TIME COMPLEXITY:
O(N) — at most N/2 comparisons.

SPACE COMPLEXITY:
O(1) extra space.

EDGE CASES: Single character; even length vs odd length.

#14: Merge Two Sorted Arrays

Arrays

PROBLEM STATEMENT:

Read two sorted arrays and merge them into a single sorted sequence printed to console.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
3
1 3 5
3
2 4 6
Sample Output:
1 2 3 4 5 6
```

ALGORITHM & INTUITION:

Standard merge step of merge sort using two pointers i and j.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Scanner;

public class CognizantMergeSorted {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n1 = sc.nextInt();
        int[] a = new int[n1];
        for (int i = 0; i < n1; i++) a[i] = sc.nextInt();

        int n2 = sc.nextInt();
        int[] b = new int[n2];
        for (int i = 0; i < n2; i++) b[i] = sc.nextInt();

        int i = 0, j = 0;
        StringBuilder sb = new StringBuilder();
        while (i < n1 && j < n2) {
            if (a[i] <= b[j]) {
                sb.append(a[i++]).append(" ");
            } else {
                sb.append(b[j++]).append(" ");
            }
        }
        while (i < n1) sb.append(a[i++]).append(" ");
        while (j < n2) sb.append(b[j++]).append(" ");

        System.out.println(sb.toString().trim());
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

a=[1,3,5], b=[2,4,6] -> 1, 2, 3, 4, 5, 6.

TIME COMPLEXITY:
O(N1 + N2) — linear scan.

SPACE COMPLEXITY:
O(1) auxiliary space.

EDGE CASES: One array is empty; overlapping ranges.

#15: Binary Search

Searching

PROBLEM STATEMENT:

Given a sorted array of N integers and a key, output the 0-based index of the key if found, or -1 if not found.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
5
10 20 30 40 50
30
Sample Output:
2
```

ALGORITHM & INTUITION:

Divide and conquer: mid = low + (high - low)/2. Compare arr[mid] with target.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Scanner;

public class CognizantBinarySearch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        int key = sc.nextInt();

        int low = 0, high = n - 1, ans = -1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == key) {
                ans = mid;
                break;
            } else if (arr[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        System.out.println(ans);
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

arr=[10, 20, 30, 40, 50], key=30 -> mid=2, arr[2]=30 -> ans=2.

TIME COMPLEXITY:
O(log N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Key absent; key at first/last element.

#16: Find Duplicate Elements

Hashing

PROBLEM STATEMENT:

Given an array, print all duplicate numbers in ascending order.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
6
1 2 3 2 1 4
Sample Output:
1 2
```

ALGORITHM & INTUITION:

Use a TreeMap to track frequencies, then print keys where frequency > 1 in sorted order.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class CognizantDuplicates {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        Map<Integer, Integer> counts = new TreeMap<>();

        for (int i = 0; i < n; i++) {
            int val = sc.nextInt();
            counts.put(val, counts.getOrDefault(val, 0) + 1);
        }

        boolean found = false;
        for (Map.Entry<Integer, Integer> entry : counts.entrySet()) {
            if (entry.getValue() > 1) {
                System.out.print(entry.getKey() + " ");
                found = true;
            }
        }
        if (!found) System.out.print("-1");
        System.out.println();
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

[1, 2, 3, 2, 1, 4] -> counts: 1:2, 2:2, 3:1, 4:1 -> output: 1 2.

TIME COMPLEXITY:
O(N log K) using TreeMap.

SPACE COMPLEXITY:
O(K) where K is unique count.

EDGE CASES: No duplicates (-1); all duplicates.

#17: Stock Buy and Sell for Maximum Profit

Greedy

PROBLEM STATEMENT:

Given an array prices where prices[i] is the stock price on day i, find the maximum profit you can achieve from a single buy and sell transaction. If no profit is possible, return 0.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
6
7 1 5 3 6 4
Sample Output:
5
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6 - 1 = 5.
```

ALGORITHM & INTUITION:

Greedy single-pass: maintain minPrice seen so far. At each day i, profit = prices[i] - minPrice. Update maxProfit = max(maxProfit, profit) and minPrice = min(minPrice, prices[i]).

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Scanner;

public class CognizantStockBuySell {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        int[] prices = new int[n];
        for (int i = 0; i < n; i++) prices[i] = sc.nextInt();

        int minPrice = Integer.MAX_VALUE;
        int maxProfit = 0;

        for (int price : prices) {
            if (price < minPrice) {
                minPrice = price;
            } else if (price - minPrice > maxProfit) {
                maxProfit = price - minPrice;
            }
        }

        System.out.println(maxProfit);
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

prices = [7, 1, 5, 3, 6, 4]
- 7: min=7, profit=0
- 1: min=1, profit=0
- 5: profit=4, max=4
- 3: profit=2, max=4
- 6: profit=5, max=5
- 4: profit=3, max=5. Output: 5.

TIME COMPLEXITY:
O(N) — single pass.

SPACE COMPLEXITY:
O(1) extra space.

EDGE CASES: Strictly decreasing prices [7, 6, 4, 3, 1] (returns 0 profit).

#18: Minimum Number of Coins

Greedy / DP

PROBLEM STATEMENT:

Given a value V and an array of coin denominations, find the minimum number of coins needed to make change for V. Important interview distinction: clarify whether the coin system allows a greedy choice or requires Dynamic Programming.

SAMPLE INPUT & OUTPUT:

```
Sample Input:
V = 49, Indian Currency = [1, 2, 5, 10, 20, 50, 100, 500, 2000]
Sample Output: 5 coins (20 + 20 + 5 + 2 + 2 = 49)
Arbitrary Coins: coins = [1, 5, 6, 9], V = 11
Output: 2 coins (5 + 6 = 11, whereas Greedy picks 9 + 1 + 1 = 3 coins).
```

ALGORITHM & INTUITION:

Theoretical Foundation:
1. Canonical Systems (Indian Rupee, US Dollar): The denominations satisfy the greedy-choice matroid property: taking the largest denomination <= remaining amount is guaranteed to yield the global optimum.
2. Non-Canonical / Arbitrary Systems: Greedy fails! For example, denominations [1, 5, 6, 9] for V=11: greedy picks 9+1+1 (3 coins), but optimal is 5+6 (2 coins). We must use Dynamic Programming: dp[i] = min(dp[i], 1 + dp[i - coin]) initialized to infinity with dp[0] = 0.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.Scanner;
import java.util.Arrays;

public class CognizantMinCoins {
    // 1. Dynamic Programming Solution (Guaranteed optimal for ANY coin system)
    public static int minCoinsDP(int[] coins, int V) {
        int[] dp = new int[V + 1];
        Arrays.fill(dp, V + 1);
        dp[0] = 0;

        for (int i = 1; i <= V; i++) {
            for (int coin : coins) {
                if (i - coin >= 0) {
                    dp[i] = Math.min(dp[i], 1 + dp[i - coin]);
                }
            }
        }
        return dp[V] > V ? -1 : dp[V];
    }

    // 2. Greedy Solution (Optimal specifically for Indian canonical denominations)
    public static int minCoinsGreedyIndian(int V) {
        int[] denominations = {2000, 500, 200, 100, 50, 20, 10, 5, 2, 1};
        int count = 0;
        for (int coin : denominations) {
            if (V >= coin) {
                count += V / coin;
                V %= coin;
            }
        }
        return count;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int v = sc.nextInt();

        System.out.println("Indian Rupee (Greedy): " + minCoinsGreedyIndian(v));

        int[] arbitraryCoins = {1, 5, 6, 9};
        System.out.println("Arbitrary Coins [1,5,6,9] (DP): " + minCoinsDP(arbitraryCoins, v));
        sc.close();
    }
}
```

DRY RUN WALKTHROUGH:

V = 49, Indian Currency: 49/20 = 2 (rem 9), 9/5 = 1 (rem 4), 4/2 = 2 (rem 0). Total = 2 + 1 + 2 = 5 coins.

TIME COMPLEXITY:
Greedy: O(D) where D is number of denominations. DP: O(V * N) where V is target value and N is number of coins.

SPACE COMPLEXITY:
Greedy: O(1). DP: O(V) for the 1D memoization array.

EDGE CASES: V = 0 (returns 0); impossible amount where no denomination fits (returns -1).

CHAPTER 3

Infosys — Systems Engineer & Specialist Programmer

Algorithmic Optimization, Graphs & Dynamic Programming

Target Focus: Systems Engineer (SE), Digital Specialist Engineer (DSE) & Specialist Programmer (SP)

Infosys SP track is renowned for medium-to-hard dynamic programming (0/1 Knapsack, Coin Change, LCS, LIS) and graph algorithms (BFS/DFS, Dijkstra, Number of Islands).

Questions in this Section (18 Problems):

#1: Find the Missing Number — Arrays

PROBLEM STATEMENT:

Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [9,6,4,2,3,5,7,0,1]
Output: 8
```

ALGORITHM & INTUITION:

Use XOR bitwise logic or Gauss summation sum = n*(n+1)/2 - sum(nums). XOR is preferred to avoid 32-bit integer overflow.

COMPLETE JAVA 21 SOLUTION:

```java
public class InfosysMissingNumber {
    public static int missingNumber(int[] nums) {
        int xor = 0;
        for (int i = 0; i <= nums.length; i++) xor ^= i;
        for (int num : nums) xor ^= num;
        return xor;
    }

    public static void main(String[] args) {
        int[] nums = {9, 6, 4, 2, 3, 5, 7, 0, 1};
        System.out.println("Missing Number: " + missingNumber(nums)); // 8
    }
}
```

DRY RUN WALKTHROUGH:

All numbers from 0 to 9 XORed with array elements -> all match except 8. Output: 8.

TIME COMPLEXITY:
O(N) — single pass.

SPACE COMPLEXITY:
O(1) extra space.

EDGE CASES: Missing 0; missing n; n = 1.

#2: Remove Duplicates from an Array

Arrays

PROBLEM STATEMENT:

Given a sorted integer array nums, remove the duplicates in-place such that each unique element appears only once. Return the number of unique elements k.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]
Output: k = 5, nums = [0, 1, 2, 3, 4, ...]
```

ALGORITHM & INTUITION:

Two pointers: 'insertIdx' points to the position of the next unique element. Traverse 'i' from 1 to n-1. If nums[i] != nums[insertIdx], increment insertIdx and set nums[insertIdx] = nums[i].

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class RemoveDuplicatesSorted {
    public static int removeDuplicates(int[] nums) {
        if (nums == null || nums.length == 0) return 0;
        int insertIdx = 0;

        for (int i = 1; i < nums.length; i++) {
            if (nums[i] != nums[insertIdx]) {
                insertIdx++;
                nums[insertIdx] = nums[i];
            }
        }
        return insertIdx + 1;
    }

    public static void main(String[] args) {
        int[] nums = {0, 0, 1, 1, 1, 2, 2, 3, 3, 4};
        int k = removeDuplicates(nums);
        System.out.println("Unique count: " + k);
        System.out.println("Array slice: " + Arrays.toString(Arrays.copyOfRange(nums, 0, k)));
    }
}
```

DRY RUN WALKTHROUGH:

nums=[0,0,1,1,1,2,2,3,3,4]. Unique elements placed at index 0..4: [0, 1, 2, 3, 4]. Return 5.

TIME COMPLEXITY:
O(N) — linear scan.

SPACE COMPLEXITY:
O(1) in-place.

EDGE CASES: Array of length 0 or 1; all elements identical; no duplicates.

#3: Two Sum

Arrays / Hashing

PROBLEM STATEMENT:

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [2, 7, 11, 15], target = 9
Output: [0, 1]
```

ALGORITHM & INTUITION:

Hash map lookup: For each element x at index i, complement = target - x. Check if complement exists in map; if so return [map.get(complement), i].

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class InfosysTwoSum {
    public static int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int comp = target - nums[i];
            if (map.containsKey(comp)) {
                return new int[]{map.get(comp), i};
            }
            map.put(nums[i], i);
        }
        return new int[0];
    }

    public static void main(String[] args) {
        int[] nums = {2, 7, 11, 15};
        System.out.println(Arrays.toString(twoSum(nums, 9))); // [0, 1]
    }
}
```

DRY RUN WALKTHROUGH:

nums=[2, 7, 11, 15], target=9 -> 9-7=2 in map -> [0, 1].

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Duplicate numbers; negative numbers.

#4: Maximum Subarray Sum

Kadane's algorithm

PROBLEM STATEMENT:

Find the contiguous subarray within an array (containing at least one number) which has the largest sum.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6
```

ALGORITHM & INTUITION:

Kadane's algorithm: curr = max(num, curr + num), max = max(max, curr).

COMPLETE JAVA 21 SOLUTION:

```java
public class InfosysMaxSubarray {
    public static int maxSubArray(int[] nums) {
        int curr = nums[0], max = nums[0];
        for (int i = 1; i < nums.length; i++) {
            curr = Math.max(nums[i], curr + nums[i]);
            max = Math.max(max, curr);
        }
        return max;
    }

    public static void main(String[] args) {
        int[] nums = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
        System.out.println("Max Subarray Sum: " + maxSubArray(nums)); // 6
    }
}
```

DRY RUN WALKTHROUGH:

Subarray [4, -1, 2, 1] yields maximum sum = 6.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: All negative numbers [-8, -3, -6, -2] (returns -2).

#5: Longest Substring Without Repeating Characters

Sliding Window

PROBLEM STATEMENT:

Given a string s, find the length of the longest substring without repeating characters.

SAMPLE INPUT & OUTPUT:

```
Input: s = "abcabcbb"
Output: 3
Explanation: The answer is "abc", with length 3.
```

ALGORITHM & INTUITION:

Sliding window with two pointers left and right, and a HashMap or index array tracking the most recent index of each character. When s[right] is seen at index idx >= left, advance left to idx + 1. Update maxLen = max(maxLen, right - left + 1).

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class LongestSubstringUnique {
    public static int lengthOfLongestSubstring(String s) {
        int[] lastSeen = new int[256];
        Arrays.fill(lastSeen, -1);
        int maxLen = 0;
        int left = 0;

        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (lastSeen[c] >= left) {
                left = lastSeen[c] + 1;
            }
            lastSeen[c] = right;
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }

    public static void main(String[] args) {
        System.out.println("Length: " + lengthOfLongestSubstring("abcabcbb")); // 3
        System.out.println("Length: " + lengthOfLongestSubstring("pwwkew"));   // 3
    }
```

```
}
```

DRY RUN WALKTHROUGH:

"abcabcbb": window expands "abc" (len 3). At 'a', left moves to 1 ("bca"). Maximum length remains 3.

TIME COMPLEXITY:
O(N) — right pointer moves N times.

SPACE COMPLEXITY:
O(1) — constant size 256 array.

EDGE CASES: Empty string (0); all characters identical "bbbbb" (1); all unique "abcdef" (6).

#6: Merge Overlapping Intervals

Intervals

PROBLEM STATEMENT:

Given an array of intervals where intervals[i] = [start_i, end_i], merge all overlapping intervals, and return an array of the non-overlapping intervals.

SAMPLE INPUT & OUTPUT:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
```

ALGORITHM & INTUITION:

1. Sort intervals by start time ascending.
2. Iterate through intervals. If current interval starts <= previous interval's end, merge them: prev.end = max(prev.end, curr.end). Otherwise, append current interval as a new interval.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class MergeIntervals {
    public static int[][] merge(int[][] intervals) {
        if (intervals == null || intervals.length <= 1) return intervals;
        // Sort by start time
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));

        List<int[]> merged = new ArrayList<>();
        int[] current = intervals[0];
        merged.add(current);

        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] <= current[1]) {
                // Overlap: extend end time
                current[1] = Math.max(current[1], intervals[i][1]);
            } else {
                // Disjoint interval: start new segment
                current = intervals[i];
                merged.add(current);
            }
        }
        return merged.toArray(new int[merged.size()][]);
    }

    public static void main(String[] args) {
        int[][] intervals = {{1, 3}, {2, 6}, {8, 10}, {15, 18}};
        int[][] res = merge(intervals);
        for (int[] iv : res) System.out.print(Arrays.toString(iv) + " ");
        // [[1, 6], [8, 10], [15, 18]]
    }
}
```

DRY RUN WALKTHROUGH:

[1, 3] and [2, 6] overlap (2 <= 3) -> merged to [1, 6]. Next is [8, 10] (8 > 6) -> separate. Output: [1, 6], [8, 10], [15, 18].

TIME COMPLEXITY:
O(N log N) — sorting dominates.

SPACE COMPLEXITY:
O(N) for result list.

EDGE CASES: All intervals overlap into one; no overlaps; intervals with identical start/end.

#7: Search in a Rotated Sorted Array

Binary Search

PROBLEM STATEMENT:

Given the array nums after the possible rotation and an integer target, return the index of target if it is in nums, or -1 if it is not in nums. Must run in O(log N) time.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [4, 5, 6, 7, 0, 1, 2], target = 0
Output: 4
```

ALGORITHM & INTUITION:

Modified binary search: in any rotated sorted array, dividing at mid always leaves AT LEAST ONE HALF strictly sorted.
1. Check if left half [low..mid] is sorted (nums[low] <= nums[mid]). If so, test if target lies in [nums[low], nums[mid]].
2. Otherwise, right half [mid..high] must be sorted. Test if target lies in [nums[mid], nums[high]].
3. Adjust low or high accordingly.

COMPLETE JAVA 21 SOLUTION:

```java
public class RotatedBinarySearch {
    public static int search(int[] nums, int target) {
        int low = 0, high = nums.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (nums[mid] == target) return mid;

            // Check if left half is sorted
            if (nums[low] <= nums[mid]) {
                if (target >= nums[low] && target < nums[mid]) {
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            } else { // Right half is sorted
                if (target > nums[mid] && target <= nums[high]) {
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] nums = {4, 5, 6, 7, 0, 1, 2};
        System.out.println("Index of 0: " + search(nums, 0)); // 4
        System.out.println("Index of 3: " + search(nums, 3)); // -1
    }
}
```

DRY RUN WALKTHROUGH:

nums=[4, 5, 6, 7, 0, 1, 2], target=0
- low=0, high=6, mid=3 (val=7). Left half [4..7] is sorted. 0 is not in [4, 7], so low = mid+1 = 4.
- low=4, high=6, mid=5 (val=1). Right half [1..2] sorted. nums[4]=0 < 1, high = mid-1 = 4.
- low=4, high=4, mid=4 (val=0) == target! Returns 4.

TIME COMPLEXITY:
O(log N) logarithmic search.

SPACE COMPLEXITY:
O(1) auxiliary space.

EDGE CASES: Array not rotated (original sorted); target at pivot; 1 or 2 elements array.

#8: Count Inversions in an Array

Divide and Conquer

PROBLEM STATEMENT:

Given an array of integers, count the number of inversions. Two elements arr[i] and arr[j] form an inversion if arr[i] > arr[j] and i < j.

SAMPLE INPUT & OUTPUT:

```
Input: arr = [2, 4, 1, 3, 5]
Output: 3
Explanation: Inversions are (2, 1), (4, 1), and (4, 3).
```

ALGORITHM & INTUITION:

Enhanced Merge Sort: When merging two sorted halves [left..mid] and [mid+1..right], if arr[i] > arr[j], then because the left subarray is sorted, every element from arr[i] through arr[mid] is strictly greater than arr[j]. We can add (mid - i + 1) to inversion count in O(1) time!

COMPLETE JAVA 21 SOLUTION:

```java
public class CountInversions {
    public static long mergeSortAndCount(int[] arr, int[] temp, int left, int right) {
        long count = 0;
        if (left < right) {
            int mid = left + (right - left) / 2;
            count += mergeSortAndCount(arr, temp, left, mid);
            count += mergeSortAndCount(arr, temp, mid + 1, right);
            count += merge(arr, temp, left, mid, right);
        }
        return count;
    }

    private static long merge(int[] arr, int[] temp, int left, int mid, int right) {
        int i = left, j = mid + 1, k = left;
        long count = 0;

        while (i <= mid && j <= right) {
            if (arr[i] <= arr[j]) {
                temp[k++] = arr[i++];
            } else {
                temp[k++] = arr[j++];
                count += (mid - i + 1); // Key inversion logic
            }
        }
        while (i <= mid) temp[k++] = arr[i++];
        while (j <= right) temp[k++] = arr[j++];
        for (i = left; i <= right; i++) arr[i] = temp[i];

        return count;
    }

    public static void main(String[] args) {
        int[] arr = {2, 4, 1, 3, 5};
        long inv = mergeSortAndCount(arr, new int[arr.length], 0, arr.length - 1);
        System.out.println("Inversion Count: " + inv); // 3
    }
}
```

DRY RUN WALKTHROUGH:

[2, 4] and [1, 3, 5]
Comparing 2 > 1 -> mid - i + 1 = 1 - 0 + 1 = 2 inversions ((2,1), (4,1)).
Comparing 4 > 3 -> 1 - 1 + 1 = 1 inversion ((4,3)). Total = 3.

TIME COMPLEXITY:
O(N log N) — same as merge sort.

SPACE COMPLEXITY:
O(N) for auxiliary temp buffer.

EDGE CASES: Array sorted ascending (0 inversions); array sorted descending (N*(N-1)/2 inversions).

#9: Generate All Subsets (Power Set)

Backtracking / Recursion

PROBLEM STATEMENT:

Given an integer array nums of unique elements, return all possible subsets (the power set). The solution set must not contain duplicate subsets.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [1, 2, 3]
Output: [[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]
```

ALGORITHM & INTUITION:

Backtracking: At each index, decide whether to include nums[index] in the current subset. Add the current subset to results, then recurse for remaining elements.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class SubsetsGenerator {
    public static List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, 0, new ArrayList<>(), result);
        return result;
    }

    private static void backtrack(int[] nums, int start, List<Integer> current, List<List<Integer>>
result) {
        result.add(new ArrayList<>(current));

        for (int i = start; i < nums.length; i++) {
            current.add(nums[i]);
            backtrack(nums, i + 1, current, result);
            current.remove(current.size() - 1); // backtrack
        }
    }

    public static void main(String[] args) {
        int[] nums = {1, 2, 3};
        List<List<Integer>> allSubsets = subsets(nums);
        System.out.println("Total subsets: " + allSubsets.size()); // 8
        System.out.println(allSubsets);
    }
}
```

DRY RUN WALKTHROUGH:

nums=[1, 2, 3]
[] -> [1] -> [1, 2] -> [1, 2, 3] -> [1, 3] -> [2] -> [2, 3] -> [3]. Total = 2^3 = 8 subsets.

TIME COMPLEXITY:
O(N * 2^N) — 2^N subsets generated, each taking up to O(N) copy time.

SPACE COMPLEXITY:
O(N) recursion call stack depth.

EDGE CASES: Empty array (returns [[]]); single element array (returns [[], [x]]).

#10: Generate All Permutations

Backtracking

PROBLEM STATEMENT:

Given an array nums of distinct integers, return all possible permutations.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [1, 2, 3]
Output: [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]
```

ALGORITHM & INTUITION:

In-place swapping backtracking. Fix element at index 'start', then swap it with every element from start to n-1, recurse for start + 1, and backtrack by swapping back.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class PermutationsGenerator {
    public static List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, 0, result);
        return result;
    }

    private static void backtrack(int[] nums, int start, List<List<Integer>> result) {
        if (start == nums.length) {
            List<Integer> perm = new ArrayList<>();
            for (int num : nums) perm.add(num);
            result.add(perm);
            return;
        }

        for (int i = start; i < nums.length; i++) {
            swap(nums, start, i);
            backtrack(nums, start + 1, result);
            swap(nums, start, i); // backtrack
        }
    }

    private static void swap(int[] nums, int i, int j) {
        int t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }

    public static void main(String[] args) {
        int[] nums = {1, 2, 3};
        System.out.println(permute(nums)); // 6 permutations
    }
}
```

DRY RUN WALKTHROUGH:

3! = 6 permutations generated by systematically swapping each position.

TIME COMPLEXITY:
O(N! * N) — N! permutations, each taking O(N) to add to result.

SPACE COMPLEXITY:
O(N) recursion stack.

EDGE CASES: Single element array [1]; array with 2 elements [1, 2].

#11: Subset Sum Problem

Dynamic Programming

PROBLEM STATEMENT:

Given an array of non-negative integers nums and a value target, determine if there is a subset of the given set with sum equal to given target.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [3, 34, 4, 12, 5, 2], target = 9
Output: true
Explanation: Subset [4, 5] has sum 9.
```

ALGORITHM & INTUITION:

0/1 Knapsack style 1D boolean DP array. dp[j] is true if sum j can be formed. Initialize dp[0] = true. For each num in nums, iterate j backwards from target down to num: dp[j] = dp[j] || dp[j - num]. Backwards iteration ensures each element is used at most once.

COMPLETE JAVA 21 SOLUTION:

```java
public class SubsetSum {
    public static boolean canPartition(int[] nums, int target) {
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;

        for (int num : nums) {
            for (int j = target; j >= num; j--) {
                dp[j] = dp[j] || dp[j - num];
            }
        }
        return dp[target];
    }

    public static void main(String[] args) {
        int[] nums = {3, 34, 4, 12, 5, 2};
        System.out.println("Can make 9? " + canPartition(nums, 9)); // true
        System.out.println("Can make 30? " + canPartition(nums, 30)); // false
    }
}
```

DRY RUN WALKTHROUGH:

target = 9, num = 4 -> dp[4]=true. num = 5 -> dp[9] = dp[9] || dp[9-5] = dp[4] = true! Return true.

TIME COMPLEXITY:
O(N * target) pseudo-polynomial time.

SPACE COMPLEXITY:
O(target) using 1D space-optimized DP array.

EDGE CASES: target = 0 (always true); sum of all elements < target (false).

#12: 0/1 Knapsack Problem

Dynamic Programming

PROBLEM STATEMENT:

Given weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack without breaking items.

SAMPLE INPUT & OUTPUT:

```
Input: values = [60, 100, 120], weights = [10, 20, 30], W = 50
Output: 220
Explanation: Take items 1 (20kg, $100) and 2 (30kg, $120) -> total weight 50kg, total value 220.
```

ALGORITHM & INTUITION:

1D DP array dp[w] storing maximum value for weight w. For each item (weight wt, value val), iterate w backwards from W down to wt: dp[w] = max(dp[w], val + dp[w - wt]).

COMPLETE JAVA 21 SOLUTION:

```
public class ZeroOneKnapsack {
    public static int knapSack(int W, int[] weights, int[] values) {
        int n = weights.length;
        int[] dp = new int[W + 1];

        for (int i = 0; i < n; i++) {
            int wt = weights[i];
            int val = values[i];
            for (int w = W; w >= wt; w--) {
                dp[w] = Math.max(dp[w], val + dp[w - wt]);
            }
        }
        return dp[W];
    }

    public static void main(String[] args) {
        int[] values = {60, 100, 120};
        int[] weights = {10, 20, 30};
        int W = 50;
        System.out.println("Max Value: " + knapSack(W, weights, values)); // 220
    }
}
```

DRY RUN WALKTHROUGH:

W = 50
Item 1 (10, 60) -> dp[10..50] = 60
Item 2 (20, 100) -> dp[30..50] = 160
Item 3 (30, 120) -> dp[50] = max(160, 120 + dp[20]=100) = 220.

TIME COMPLEXITY:
O(N * W).

SPACE COMPLEXITY:
O(W) 1D optimized space.

EDGE CASES: W = 0 (returns 0); all items heavier than W (returns 0).

#13: Coin Change (Fewest Coins) Dynamic Programming

PROBLEM STATEMENT:

You are given an integer array coins representing coins of different denominations and an integer amount. Return the fewest number of coins that you need to make up that amount, or -1 if impossible.

SAMPLE INPUT & OUTPUT:

```
Input: coins = [1, 2, 5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1
```

ALGORITHM & INTUITION:

Bottom-up 1D DP: dp[i] = min number of coins for amount i. Initialize dp[0] = 0 and all other dp entries to amount + 1 (infinity). For i from 1 to amount, for each coin in coins: if i >= coin, dp[i] = min(dp[i], 1 + dp[i - coin]).

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class CoinChangeDP {
    public static int coinChange(int[] coins, int amount) {
        int max = amount + 1;
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, max);
        dp[0] = 0;

        for (int i = 1; i <= amount; i++) {
            for (int coin : coins) {
                if (coin <= i) {
                    dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                }
            }
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }

    public static void main(String[] args) {
        int[] coins = {1, 2, 5};
        System.out.println("Fewest coins for 11: " + coinChange(coins, 11)); // 3
        System.out.println("Fewest coins for 3 with [2]: " + coinChange(new int[]{2}, 3)); // -1
    }
}
```

DRY RUN WALKTHROUGH:

dp[0]=0, dp[1]=1 (1), dp[2]=1 (2), dp[5]=1 (5), dp[10]=2 (5+5), dp[11]=3 (5+5+1).

TIME COMPLEXITY:
O(amount * coins.length).

SPACE COMPLEXITY:
O(amount).

EDGE CASES: amount = 0 (returns 0); no coin fits (returns -1).

#14: Longest Common Subsequence (LCS) — Dynamic Programming

PROBLEM STATEMENT:

Given two strings text1 and text2, return the length of their longest common subsequence. If there is no common subsequence, return 0.

SAMPLE INPUT & OUTPUT:

```
Input: text1 = "abcde", text2 = "ace"
Output: 3
Explanation: The longest common subsequence is "ace" and its length is 3.
```

ALGORITHM & INTUITION:

2D DP matrix dp[i][j]: LCS of text1[0..i-1] and text2[0..j-1].
- If text1[i-1] == text2[j-1]: dp[i][j] = 1 + dp[i-1][j-1].
- Else: dp[i][j] = max(dp[i-1][j], dp[i][j-1]).

COMPLETE JAVA 21 SOLUTION:

```java
public class LongestCommonSubsequence {
    public static int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length(), n = text2.length();
        int[][] dp = new int[m + 1][n + 1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                } else {
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }
        return dp[m][n];
    }

    public static void main(String[] args) {
        System.out.println("LCS: " + longestCommonSubsequence("abcde", "ace")); // 3
    }
}
```

DRY RUN WALKTHROUGH:

Match 'a' -> 1. Match 'c' -> 2. Match 'e' -> 3. Output: 3.

TIME COMPLEXITY:
O(M * N).

SPACE COMPLEXITY:
O(M * N) (can be optimized to O(min(M, N))).

EDGE CASES: No common characters; identical strings; empty string.

#15: Longest Increasing Subsequence (LIS)

Dynamic Programming / Patience Sorting

PROBLEM STATEMENT:

Given an integer array nums, return the length of the longest strictly increasing subsequence.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [10, 9, 2, 5, 3, 7, 101, 18]
Output: 4
Explanation: The longest increasing subsequence is [2, 3, 7, 101], therefore the length is 4.
```

ALGORITHM & INTUITION:

Patience Sorting with Binary Search (O(N log N)):
Maintain an array 'tails' where tails[i] stores the smallest tail of all increasing subsequences of length i + 1.
For each num in nums, use binary search (Arrays.binarySearch) to find its insertion position in tails. If it is larger than all elements, append it; otherwise replace the first element >= num.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class LongestIncreasingSubsequence {
    public static int lengthOfLIS(int[] nums) {
        if (nums == null || nums.length == 0) return 0;
        int[] tails = new int[nums.length];
        int size = 0;

        for (int num : nums) {
            int i = 0, j = size;
            while (i != j) {
                int mid = (i + j) / 2;
                if (tails[mid] < num) {
                    i = mid + 1;
                } else {
                    j = mid;
                }
            }
            tails[i] = num;
            if (i == size) size++;
        }
        return size;
    }

    public static void main(String[] args) {
        int[] nums = {10, 9, 2, 5, 3, 7, 101, 18};
        System.out.println("LIS Length: " + lengthOfLIS(nums)); // 4
    }
}
```

DRY RUN WALKTHROUGH:

nums = [10, 9, 2, 5, 3, 7, 101, 18]
tails progression: [10] -> [9] -> [2] -> [2, 5] -> [2, 3] -> [2, 3, 7] -> [2, 3, 7, 101] -> [2, 3, 7, 18]. Length = 4.

TIME COMPLEXITY:
O(N log N) using binary search.

SPACE COMPLEXITY:
O(N) for tails array.

EDGE CASES: Array already sorted strictly increasing (len N); array sorted descending (len 1); duplicates.

#16: Number of Islands

Graphs / BFS & DFS

PROBLEM STATEMENT:

Given an m x n 2D binary grid grid which represents a map of '1's (land) and '0's (water), return the number of islands. An island is surrounded by water and formed by connecting adjacent lands horizontally or vertically.

SAMPLE INPUT & OUTPUT:

```
Input: grid = [
  ['1','1','0','0','0'],
  ['1','1','0','0','0'],
  ['0','0','1','0','0'],
  ['0','0','0','1','1']
]
Output: 3
```

ALGORITHM & INTUITION:

Iterate through every cell (r, c). When an unvisited land cell '1' is encountered, increment island count and trigger a DFS/BFS sink operation: recursively mark all connected land neighbors as '0' (visited) in all 4 cardinal directions.

COMPLETE JAVA 21 SOLUTION:

```java
public class NumberOfIslands {
    public static int numIslands(char[][] grid) {
        if (grid == null || grid.length == 0) return 0;
        int count = 0;
        int rows = grid.length, cols = grid[0].length;

        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                if (grid[r][c] == '1') {
                    count++;
                    dfs(grid, r, c);
                }
            }
        }
        return count;
    }

    private static void dfs(char[][] grid, int r, int c) {
        if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length || grid[r][c] != '1') {
            return;
        }
        grid[r][c] = '0'; // Sink island
        dfs(grid, r + 1, c);
        dfs(grid, r - 1, c);
        dfs(grid, r, c + 1);
        dfs(grid, r, c - 1);
    }

    public static void main(String[] args) {
        char[][] grid = {
            {'1','1','0','0','0'},
            {'1','1','0','0','0'},
            {'0','0','1','0','0'},
            {'0','0','0','1','1'}
        };
        System.out.println("Islands: " + numIslands(grid)); // 3
    }
}
```

DRY RUN WALKTHROUGH:

Grid has 3 disconnected connected components of '1's. DFS completely sinks each component upon discovery. Total = 3.

TIME COMPLEXITY:
O(M * N) — each cell visited constant times.

SPACE COMPLEXITY:
O(M * N) in worst case recursion stack (e.g. grid completely full of '1's).

EDGE CASES: Grid with only water '0's (returns 0); grid with all land '1's (returns 1); 1x1 grid.

#17: BFS and DFS Traversal of Graph

Graphs

PROBLEM STATEMENT:

Given an undirected graph represented by an adjacency list and a starting source vertex 0, output its Breadth First Search (BFS) and Depth First Search (DFS) traversals.

SAMPLE INPUT & OUTPUT:

```
Input: V = 5, edges = [[0,1], [0,2], [0,3], [2,4]]
Output BFS: 0, 1, 2, 3, 4
Output DFS: 0, 1, 2, 4, 3
```

ALGORITHM & INTUITION:

BFS uses a Queue: mark source visited, enqueue, and while queue is not empty, dequeue u, visit all unvisited neighbors, mark them visited, and enqueue.
DFS uses Recursion (Stack): visit u, mark visited, recursively call DFS on each unvisited neighbor.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class GraphTraversals {
    public static List<Integer> bfs(int V, List<List<Integer>> adj) {
        List<Integer> bfsOrder = new ArrayList<>();
        boolean[] visited = new boolean[V];
        Queue<Integer> queue = new LinkedList<>();

        visited[0] = true;
        queue.add(0);

        while (!queue.isEmpty()) {
            int u = queue.poll();
            bfsOrder.add(u);
            for (int v : adj.get(u)) {
                if (!visited[v]) {
                    visited[v] = true;
                    queue.add(v);
                }
            }
        }
        return bfsOrder;
    }

    public static List<Integer> dfs(int V, List<List<Integer>> adj) {
        List<Integer> dfsOrder = new ArrayList<>();
        boolean[] visited = new boolean[V];
        dfsHelper(0, adj, visited, dfsOrder);
        return dfsOrder;
    }

    private static void dfsHelper(int u, List<List<Integer>> adj, boolean[] visited, List<Integer> res) {
        visited[u] = true;
        res.add(u);
        for (int v : adj.get(u)) {
            if (!visited[v]) {
                dfsHelper(v, adj, visited, res);
            }
        }
    }

    public static void main(String[] args) {
        int V = 5;
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
        adj.get(0).addAll(Arrays.asList(1, 2, 3));
        adj.get(1).add(0);
        adj.get(2).addAll(Arrays.asList(0, 4));
        adj.get(3).add(0);
        adj.get(4).add(2);

        System.out.println("BFS: " + bfs(V, adj));
        System.out.println("DFS: " + dfs(V, adj));
    }
}
```

DRY RUN WALKTHROUGH:

BFS explores layer by layer: 0 -> {1, 2, 3} -> {4}. DFS explores deeply along branch 0->2->4 before backtracking to 3.

TIME COMPLEXITY:
O(V + E) for both BFS and DFS.

SPACE COMPLEXITY:
O(V) for visited array and queue/stack.

EDGE CASES: Disconnected graph components; graph with cycles; single node graph.

#18: Dijkstra's Shortest Path Algorithm

Graphs / Shortest Path

PROBLEM STATEMENT:

Given a weighted, directed/undirected graph and a source vertex src, find the shortest path distances from src to all other vertices. All edge weights are non-negative.

SAMPLE INPUT & OUTPUT:

```
Input: V = 4, edges = [[0,1,4], [0,2,1], [2,1,2], [1,3,1]], src = 0
Output distances: [0, 3, 1, 4]
Explanation: Path 0->2 (wt 1), 0->2->1 (wt 1+2=3), 0->2->1->3 (wt 1+2+1=4).
```

ALGORITHM & INTUITION:

Greedy algorithm with a Min-PriorityQueue of pairs (distance, vertex):
1. Initialize dist[] to infinity, dist[src] = 0.
2. Insert (0, src) into PQ.
3. While PQ is not empty, extract vertex u with minimum distance. If distance is stale (d > dist[u]), skip.
4. For each edge (u, v, weight), relax: if dist[u] + weight < dist[v], update dist[v] and insert (dist[v], v) into PQ.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class DijkstraShortestPath {
    static class Edge {
        int to, weight;
        Edge(int to, int weight) {
            this.to = to;
            this.weight = weight;
        }
    }

    static class Node implements Comparable<Node> {
        int vertex, dist;
        Node(int vertex, int dist) {
            this.vertex = vertex;
            this.dist = dist;
        }
        public int compareTo(Node other) {
            return Integer.compare(this.dist, other.dist);
        }
    }

    public static int[] dijkstra(int V, List<List<Edge>> adj, int src) {
        int[] dist = new int[V];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        PriorityQueue<Node> pq = new PriorityQueue<>();
        pq.add(new Node(src, 0));

        while (!pq.isEmpty()) {
            Node curr = pq.poll();
            int u = curr.vertex;
            int d = curr.dist;

            if (d > dist[u]) continue;

            for (Edge edge : adj.get(u)) {
                int v = edge.to;
                int weight = edge.weight;
                if (dist[u] + weight < dist[v]) {
                    dist[v] = dist[u] + weight;
                    pq.add(new Node(v, dist[v]));
                }
            }
        }
        return dist;
    }
```

```
    public static void main(String[] args) {
        int V = 4;
        List<List<Edge>> adj = new ArrayList<>();
        for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
        adj.get(0).add(new Edge(1, 4));
        adj.get(0).add(new Edge(2, 1));
        adj.get(2).add(new Edge(1, 2));
        adj.get(1).add(new Edge(3, 1));

        int[] res = dijkstra(V, adj, 0);
        System.out.println("Distances from 0: " + Arrays.toString(res)); // [0, 3, 1, 4]
    }
}
```

DRY RUN WALKTHROUGH:

dist=[0, inf, inf, inf]
- u=0: dist[2]=1, dist[1]=4
- u=2: dist[2]+2=3 < dist[1]=4 -> dist[1]=3
- u=1: dist[1]+1=4 -> dist[3]=4
Final: [0, 3, 1, 4].

TIME COMPLEXITY:
O((V + E) log V) with binary heap PriorityQueue.

SPACE COMPLEXITY:
O(V + E) for adjacency list and dist array.

EDGE CASES: Unreachable vertex (distance remains Integer.MAX_VALUE); zero-weight edges; source vertex has no outgoing edges.

CHAPTER 4

Wipro — Entry-Level & Turbo Technical

Fundamental Coding Challenges & Data Structure Manipulations

Target Focus: Wipro Elite NTH & Turbo Assessment Rounds

Wipro focuses on algorithmic fundamentals: recursion, linked list reversal, balanced parentheses, array rotations, and searching/sorting algorithms.

Questions in this Section (20 Problems):

#1: Reverse a Number — Mathematics

PROBLEM STATEMENT:

Given an integer n, return the reversed number. Handle both positive and negative values.

SAMPLE INPUT & OUTPUT:

```
Input: 1234
Output: 4321
```

ALGORITHM & INTUITION:

Extract digits using modulo 10 and build reversed number: rev = rev * 10 + n % 10. Divide n by 10 each step.

COMPLETE JAVA 21 SOLUTION:

```java
public class WiproReverseNumber {
    public static int reverse(int n) {
        int rev = 0;
        while (n != 0) {
            rev = rev * 10 + n % 10;
            n /= 10;
        }
        return rev;
    }
    public static void main(String[] args) {
        System.out.println(reverse(1234)); // 4321
        System.out.println(reverse(-456)); // -654
    }
}
```

DRY RUN WALKTHROUGH:

1234: rev=4 -> 43 -> 432 -> 4321.

TIME COMPLEXITY:
O(log10 N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Number ending in zero (500 -> 5); 0 -> 0.

#2: Check a Prime Number

Mathematics

PROBLEM STATEMENT:

Determine whether a given number is prime or not.

SAMPLE INPUT & OUTPUT:

```
Input: 17
Output: Prime
```

ALGORITHM & INTUITION:

Check divisibility up to sqrt(n). If divisible by any number, it is composite.

COMPLETE JAVA 21 SOLUTION:

```java
public class WiproPrimeCheck {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) return false;
        }
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isPrime(17) ? "Prime" : "Not Prime"); // Prime
    }
}
```

DRY RUN WALKTHROUGH:

17: sqrt is 4.12. Check 2, 3, 4 -> no divisor -> Prime.

TIME COMPLEXITY:
O(sqrt(N)).

SPACE COMPLEXITY:
O(1).

EDGE CASES: n <= 1 (Not prime); n = 2 (Prime).

#3: Print the Fibonacci Series

Mathematics

PROBLEM STATEMENT:

Print the first n terms of the Fibonacci sequence.

SAMPLE INPUT & OUTPUT:

```
Input: n = 6
Output: 0 1 1 2 3 5
```

ALGORITHM & INTUITION:

Iterative two-pointer state: a = 0, b = 1, next = a + b.

COMPLETE JAVA 21 SOLUTION:

```java
public class WiproFibonacci {
    public static void printFibonacci(int n) {
        int a = 0, b = 1;
        for (int i = 0; i < n; i++) {
            System.out.print(a + " ");
            int c = a + b;
            a = b;
            b = c;
        }
        System.out.println();
    }
    public static void main(String[] args) {
        printFibonacci(6); // 0 1 1 2 3 5
    }
}
```

DRY RUN WALKTHROUGH:

0, 1, 1, 2, 3, 5.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: n = 1 (prints 0).

#4: Calculate Factorial Using Recursion

Recursion

PROBLEM STATEMENT:

Given an integer n, calculate its factorial n! using recursion.

SAMPLE INPUT & OUTPUT:

```
Input: n = 5
Output: 120 (5 * 4 * 3 * 2 * 1)
```

ALGORITHM & INTUITION:

Base case: if n <= 1 return 1. Recursive case: return n * factorial(n - 1). Use long to handle values up to 20!.

COMPLETE JAVA 21 SOLUTION:

```java
public class WiproFactorial {
    public static long factorial(int n) {
        if (n <= 1) return 1;
        return n * factorial(n - 1);
    }
    public static void main(String[] args) {
        System.out.println("5! = " + factorial(5)); // 120
    }
}
```

DRY RUN WALKTHROUGH:

fact(5) = 5 * fact(4) = 5 * 24 = 120.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N) call stack.

EDGE CASES: n = 0 (returns 1).

#5: Find GCD and LCM

Mathematics

PROBLEM STATEMENT:

Find the GCD and LCM of two integers.

SAMPLE INPUT & OUTPUT:

```
Input: a = 15, b = 25
Output: GCD = 5, LCM = 75
```

ALGORITHM & INTUITION:

Euclidean algorithm for GCD: gcd(a, b) = gcd(b, a % b). LCM = (a / gcd) * b.

COMPLETE JAVA 21 SOLUTION:

```java
public class WiproGcdLcm {
    public static int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
    public static int lcm(int a, int b) {
        return (a / gcd(a, b)) * b;
    }
    public static void main(String[] args) {
        System.out.println("GCD: " + gcd(15, 25) + ", LCM: " + lcm(15, 25)); // 5, 75
    }
}
```

DRY RUN WALKTHROUGH:

15%25=15 -> 25%15=10 -> 15%10=5 -> 10%5=0 -> GCD=5.

TIME COMPLEXITY:
O(log(min(A, B))).

SPACE COMPLEXITY:
O(log(min(A, B))) stack.

EDGE CASES: One number is multiple of the other.

#6: Find the Second-Largest Array Element

Arrays

PROBLEM STATEMENT:

Find the second-largest element in an integer array in single traversal.

SAMPLE INPUT & OUTPUT:

```
Input: [10, 5, 20, 8]
Output: 10
```

ALGORITHM & INTUITION:

Track max1 and max2. Update max2 when num > max1, or num > max2 and num < max1.

COMPLETE JAVA 21 SOLUTION:

```java
public class WiproSecondLargest {
    public static int secondLargest(int[] arr) {
        int max1 = Integer.MIN_VALUE, max2 = Integer.MIN_VALUE;
        for (int x : arr) {
            if (x > max1) {
                max2 = max1;
                max1 = x;
            } else if (x > max2 && x < max1) {
                max2 = x;
            }
        }
        return max2;
    }
    public static void main(String[] args) {
        System.out.println(secondLargest(new int[]{10, 5, 20, 8})); // 10
    }
}
```

DRY RUN WALKTHROUGH:

Elements processed: max1=20, max2=10.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Array with duplicate max values.

#7: Find the Missing Number

Arrays

PROBLEM STATEMENT:

Find missing integer in range 1 to n.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 4, 5], n = 5
Output: 3
```

ALGORITHM & INTUITION:

Sum formula: total = n*(n+1)/2. Missing = total - sum(arr).

COMPLETE JAVA 21 SOLUTION:

```java
public class WiproMissingNumber {
    public static int findMissing(int[] arr, int n) {
        long expected = (long) n * (n + 1) / 2;
        long actual = 0;
        for (int x : arr) actual += x;
        return (int) (expected - actual);
    }
    public static void main(String[] args) {
        System.out.println(findMissing(new int[]{1, 2, 4, 5}, 5)); // 3
    }
}
```

DRY RUN WALKTHROUGH:

expected = 15, actual = 12, missing = 3.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Missing first element 1; missing last element n.

#8: Move Zeros to the End

Arrays

PROBLEM STATEMENT:

Shift all 0s in the array to the rightmost positions in-place.

SAMPLE INPUT & OUTPUT:

```
Input: [0, 1, 0, 3, 12]
Output: [1, 3, 12, 0, 0]
```

ALGORITHM & INTUITION:

Two pointers: index tracks non-zero placement.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class WiproMoveZeros {
    public static void move(int[] arr) {
        int idx = 0;
        for (int x : arr) if (x != 0) arr[idx++] = x;
        while (idx < arr.length) arr[idx++] = 0;
    }
    public static void main(String[] args) {
        int[] arr = {0, 1, 0, 3, 12};
        move(arr);
        System.out.println(Arrays.toString(arr)); // [1, 3, 12, 0, 0]
    }
}
```

DRY RUN WALKTHROUGH:

Copy non-zeros to front: [1, 3, 12], fill remaining with 0: [1, 3, 12, 0, 0].

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Array with no zeros; array with all zeros.

#9: Remove Duplicates from an Array

Arrays

PROBLEM STATEMENT:

Remove duplicate elements from sorted array in-place.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 1, 2, 2, 3]
Output: [1, 2, 3]
```

ALGORITHM & INTUITION:

Two pointers: if arr[i] != arr[idx], increment idx and copy.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class WiproRemoveDuplicates {
    public static int removeDuplicates(int[] arr) {
        if (arr.length == 0) return 0;
        int idx = 0;
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] != arr[idx]) arr[++idx] = arr[i];
        }
        return idx + 1;
    }
    public static void main(String[] args) {
        int[] arr = {1, 1, 2, 2, 3};
        int len = removeDuplicates(arr);
        System.out.println(Arrays.toString(Arrays.copyOfRange(arr, 0, len))); // [1, 2, 3]
    }
}
```

DRY RUN WALKTHROUGH:

[1, 1, 2, 2, 3] -> unique count = 3.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: All elements unique; all elements same.

#10: Rotate an Array by K

Arrays

PROBLEM STATEMENT:

Rotate array to right by K steps.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 3, 4, 5], k = 2
Output: [4, 5, 1, 2, 3]
```

ALGORITHM & INTUITION:

Three reversals: reverse whole array, reverse first k, reverse remaining.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class WiproRotateArray {
    public static void rotate(int[] arr, int k) {
        k %= arr.length;
        rev(arr, 0, arr.length - 1);
        rev(arr, 0, k - 1);
        rev(arr, k, arr.length - 1);
    }
    private static void rev(int[] arr, int l, int r) {
        while (l < r) {
            int t = arr[l]; arr[l++] = arr[r]; arr[r--] = t;
        }
    }
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        rotate(arr, 2);
        System.out.println(Arrays.toString(arr)); // [4, 5, 1, 2, 3]
    }
}
```

DRY RUN WALKTHROUGH:

Reversing: [5,4,3,2,1] -> [4,5,3,2,1] -> [4,5,1,2,3].

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: k = 0; k = length.

#11: Reverse a String

Strings

PROBLEM STATEMENT:

Reverse the given string in-place on a char array.

SAMPLE INPUT & OUTPUT:

```
Input: "hello"
Output: "olleh"
```

ALGORITHM & INTUITION:

Two-pointer swap on char array from ends towards center.

COMPLETE JAVA 21 SOLUTION:

```java
public class WiproReverseString {
    public static String reverse(String s) {
        char[] chars = s.toCharArray();
        int l = 0, r = chars.length - 1;
        while (l < r) {
            char temp = chars[l];
            chars[l++] = chars[r];
            chars[r--] = temp;
        }
        return new String(chars);
    }
    public static void main(String[] args) {
        System.out.println(reverse("hello")); // olleh
    }
}
```

DRY RUN WALKTHROUGH:

'h' <-> 'o' -> 'e' <-> 'l' -> 'l' unchanged -> "olleh".

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N) for char array.

EDGE CASES: Single char "a"; empty string "".

#12: Check Palindrome and Anagram

Strings

PROBLEM STATEMENT:

Implement verification for: (1) whether a string is a palindrome, and (2) whether two strings are anagrams.

SAMPLE INPUT & OUTPUT:

```
Palindrome: "madam" -> true
Anagrams: "listen" & "silent" -> true
```

ALGORITHM & INTUITION:

Palindrome: two pointers comparing front and back.
Anagram: frequency array of 26 letters checking equality of counts.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class WiproPalAndAnagram {
    public static boolean isPalindrome(String s) {
        int l = 0, r = s.length() - 1;
        while (l < r) {
            if (s.charAt(l++) != s.charAt(r--)) return false;
        }
        return true;
    }

    public static boolean isAnagram(String s1, String s2) {
        if (s1.length() != s2.length()) return false;
        int[] freq = new int[26];
        for (int i = 0; i < s1.length(); i++) {
            freq[s1.charAt(i) - 'a']++;
            freq[s2.charAt(i) - 'a']--;
        }
        for (int f : freq) if (f != 0) return false;
        return true;
    }

    public static void main(String[] args) {
        System.out.println("madam is palindrome? " + isPalindrome("madam")); // true
        System.out.println("listen & silent are anagrams? " + isAnagram("listen", "silent")); // true
    }
}
```

DRY RUN WALKTHROUGH:

madam: m=m, a=a, d=d -> true. listen vs silent: all 26 freq buckets zero -> true.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Different string lengths.

#13: Count Vowels and Consonants

Strings

PROBLEM STATEMENT:

Given a string, count the number of vowels and consonants in it.

SAMPLE INPUT & OUTPUT:

```
Input: "Wipro Technologies"
Output: Vowels = 6, Consonants = 11
```

ALGORITHM & INTUITION:

Convert to lowercase. Iterate through characters; if letter, check if 'aeiou' contains it; increment vowel or consonant counter.

COMPLETE JAVA 21 SOLUTION:

```java
public class WiproVowelsConsonants {
    public static void count(String s) {
        int vowels = 0, consonants = 0;
        s = s.toLowerCase();
        for (char c : s.toCharArray()) {
            if (c >= 'a' && c <= 'z') {
                if ("aeiou".indexOf(c) != -1) {
                    vowels++;
                } else {
                    consonants++;
                }
            }
        }
        System.out.println("Vowels: " + vowels + ", Consonants: " + consonants);
    }
    public static void main(String[] args) {
        count("Wipro Technologies"); // Vowels: 6, Consonants: 11
    }
}
```

DRY RUN WALKTHROUGH:

Scanning "wipro technologies" -> vowels: i, o, e, o, o, i, e (6); consonants: w, p, r, t, c, h, n, l, g, s (11).

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: String with numbers or symbols; string with only vowels.

#14: Find Character Frequencies

Hashing

PROBLEM STATEMENT:

Count frequency of every character in a string.

SAMPLE INPUT & OUTPUT:

```
Input: "apple"
Output: a:1, p:2, l:1, e:1
```

ALGORITHM & INTUITION:

Array of size 256 for ASCII character counting.

COMPLETE JAVA 21 SOLUTION:

```
public class WiproCharFreq {
    public static void printFreq(String s) {
        int[] freq = new int[256];
        for (char c : s.toCharArray()) freq[c]++;
        for (int i = 0; i < 256; i++) {
            if (freq[i] > 0) System.out.print((char) i + ":" + freq[i] + " ");
        }
        System.out.println();
    }
    public static void main(String[] args) {
        printFreq("apple"); // a:1 e:1 l:1 p:2
    }
}
```

DRY RUN WALKTHROUGH:

freq['a']=1, freq['p']=2, freq['l']=1, freq['e']=1.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Spaces, symbols.

#15: Binary Search

Searching

PROBLEM STATEMENT:

Search for target in sorted array in O(log N).

SAMPLE INPUT & OUTPUT:

```
Input: arr = [2, 5, 8, 12, 16], target = 8
Output: 2
```

ALGORITHM & INTUITION:

low=0, high=len-1. mid = low + (high-low)/2.

COMPLETE JAVA 21 SOLUTION:

```
public class WiproBinarySearch {
    public static int search(int[] arr, int target) {
        int low = 0, high = arr.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == target) return mid;
            if (arr[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        System.out.println(search(new int[]{2, 5, 8, 12, 16}, 8)); // 2
    }
}
```

DRY RUN WALKTHROUGH:

mid=2, arr[2]=8 -> found.

TIME COMPLEXITY:
O(log N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Target not in array (-1).

#16: Bubble Sort and Selection Sort

Sorting

PROBLEM STATEMENT:

Implement Bubble Sort and Selection Sort for an integer array.

SAMPLE INPUT & OUTPUT:

```
Input: [64, 25, 12, 22, 11]
Output: [11, 12, 22, 25, 64]
```

ALGORITHM & INTUITION:

Bubble Sort: repeatedly swap adjacent inverted pairs.
Selection Sort: repeatedly find minimum element from unsorted portion and swap it to front.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class WiproSorting {
    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            boolean swapped = false;
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
                    swapped = true;
                }
            }
            if (!swapped) break;
        }
    }

    public static void selectionSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[minIdx]) minIdx = j;
            }
            int t = arr[minIdx]; arr[minIdx] = arr[i]; arr[i] = t;
        }
    }

    public static void main(String[] args) {
        int[] b = {64, 25, 12, 22, 11};
        bubbleSort(b);
        System.out.println("Bubble: " + Arrays.toString(b));

        int[] s = {64, 25, 12, 22, 11};
        selectionSort(s);
        System.out.println("Selection: " + Arrays.toString(s));
    }
}
```

DRY RUN WALKTHROUGH:

Both methods arrange [64, 25, 12, 22, 11] into sorted order [11, 12, 22, 25, 64].

TIME COMPLEXITY:
Bubble: O(N^2) worst/avg, O(N) best. Selection: O(N^2).

SPACE COMPLEXITY:
O(1) in-place.

EDGE CASES: Already sorted array.

#17: Merge Two Sorted Arrays

Arrays

PROBLEM STATEMENT:

Merge two sorted arrays into one combined sorted array.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 3, 5] and [2, 4, 6]
Output: [1, 2, 3, 4, 5, 6]
```

ALGORITHM & INTUITION:

Two pointers i and j stepping through both arrays.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class WiproMergeSorted {
    public static int[] merge(int[] a, int[] b) {
        int[] res = new int[a.length + b.length];
        int i = 0, j = 0, k = 0;
        while (i < a.length && j < b.length) {
            res[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
        }
        while (i < a.length) res[k++] = a[i++];
        while (j < b.length) res[k++] = b[j++];
        return res;
    }
    public static void main(String[] args) {
        System.out.println(Arrays.toString(merge(new int[]{1, 3, 5}, new int[]{2, 4, 6})));
    }
}
```

DRY RUN WALKTHROUGH:

[1, 3, 5] + [2, 4, 6] -> [1, 2, 3, 4, 5, 6].

TIME COMPLEXITY:
O(A + B).

SPACE COMPLEXITY:
O(A + B).

EDGE CASES: One array is empty.

#18: Maximum Subarray Sum

Kadane's algorithm

PROBLEM STATEMENT:

Find the maximum sum of any contiguous subarray.

SAMPLE INPUT & OUTPUT:

```
Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6
```

ALGORITHM & INTUITION:

Kadane: curr = max(x, curr + x), max = max(max, curr).

COMPLETE JAVA 21 SOLUTION:

```
public class WiproKadane {
    public static int maxSubArray(int[] nums) {
        int curr = nums[0], max = nums[0];
        for (int i = 1; i < nums.length; i++) {
            curr = Math.max(nums[i], curr + nums[i]);
            max = Math.max(max, curr);
        }
        return max;
    }
    public static void main(String[] args) {
        System.out.println(maxSubArray(new int[]{-2, 1, -3, 4, -1, 2, 1, -5, 4})); // 6
    }
}
```

DRY RUN WALKTHROUGH:

Subarray [4, -1, 2, 1] sum = 6.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: All negative numbers.

#19: Reverse a Linked List

Linked List

PROBLEM STATEMENT:

Given the head of a singly linked list, reverse the list, and return the reversed list.

SAMPLE INPUT & OUTPUT:

```
Input: 1 -> 2 -> 3 -> 4 -> null
Output: 4 -> 3 -> 2 -> 1 -> null
```

ALGORITHM & INTUITION:

Three-pointer iteration: prev = null, curr = head. In loop, save next = curr.next, point curr.next = prev, advance prev = curr, curr = next. Return prev.

COMPLETE JAVA 21 SOLUTION:

```
public class WiproReverseLinkedList {
    static class ListNode {
        int val;
        ListNode next;
        ListNode(int val) { this.val = val; }
    }

    public static ListNode reverseList(ListNode head) {
        ListNode prev = null, curr = head;
        while (curr != null) {
            ListNode next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }
        return prev;
    }

    public static void main(String[] args) {
        ListNode head = new ListNode(1);
        head.next = new ListNode(2);
        head.next.next = new ListNode(3);
        head.next.next.next = new ListNode(4);

        ListNode rev = reverseList(head);
        while (rev != null) {
            System.out.print(rev.val + " -> ");
```

```
            rev = rev.next;
        }
        System.out.println("null"); // 4 -> 3 -> 2 -> 1 -> null
    }
}
```

DRY RUN WALKTHROUGH:

1->2->3->4: reverses pointers one by one -> 4->3->2->1.

TIME COMPLEXITY:
O(N) — single pass.

SPACE COMPLEXITY:
O(1) in-place.

EDGE CASES: head == null; single node.

#20: Balanced Parentheses

Stack

PROBLEM STATEMENT:

Given a string s containing '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

SAMPLE INPUT & OUTPUT:

```
Input: s = "()[]{}"
Output: true
Input: s = "(]"
Output: false
```

ALGORITHM & INTUITION:

Use a Stack<Character>. Push expected closing bracket when opening bracket is seen. When closing bracket is seen, verify stack is not empty and pop matches character.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.Stack;

public class WiproBalancedParentheses {
    public static boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(') stack.push(')');
            else if (c == '{') stack.push('}');
            else if (c == '[') stack.push(']');
            else {
                if (stack.isEmpty() || stack.pop() != c) return false;
            }
        }
        return stack.isEmpty();
    }

    public static void main(String[] args) {
        System.out.println("()[]{} is valid? " + isValid("()[]{}")); // true
        System.out.println("(] is valid? " + isValid("(]"));         // false
    }
}
```

DRY RUN WALKTHROUGH:

"([{}])": push ')', push ']', push '}'. Matches '}', ']', ')' exactly -> stack empty -> true.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N) stack.

EDGE CASES: Starting with closing bracket; unmatched brackets.

CHAPTER 5

Accenture — ASE & Advanced Entry-Level

Coding Questions for Associate Software Engineer (ASE) & FSE

Target Focus: Associate Software Engineer & Full Stack Engineer

Accenture test patterns cover array frequency sorting, next greater element (monotonic stacks), activity selection (greedy), and string prefix matching.

Questions in this Section (20 Problems):

#1: Find the Second-Largest Element — Arrays

PROBLEM STATEMENT:

Find the second-largest distinct element in an array of integers.

SAMPLE INPUT & OUTPUT:

```
Input: [12, 35, 1, 10, 34, 1]
Output: 34
```

ALGORITHM & INTUITION:

Single traversal tracking largest and secondLargest. Update secondLargest when a value between largest and secondLargest is encountered.

COMPLETE JAVA 21 SOLUTION:

```java
public class AccentureSecondLargest {
    public static int findSecondLargest(int[] arr) {
        int l = Integer.MIN_VALUE, s = Integer.MIN_VALUE;
        for (int x : arr) {
            if (x > l) { s = l; l = x; }
            else if (x < l && x > s) { s = x; }
        }
        return s == Integer.MIN_VALUE ? -1 : s;
    }
    public static void main(String[] args) {
        System.out.println(findSecondLargest(new int[]{12, 35, 1, 10, 34, 1})); // 34
    }
}
```

DRY RUN WALKTHROUGH:

Single pass identifies largest 35, second-largest 34.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Array with all duplicate values.

#2: Count Even and Odd Elements

Arrays

PROBLEM STATEMENT:

Given an integer array, count the total number of even numbers and odd numbers.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 3, 4, 5, 6, 7]
Output: Even = 3, Odd = 4
```

ALGORITHM & INTUITION:

Iterate through elements: if num % 2 == 0 increment even, else increment odd.

COMPLETE JAVA 21 SOLUTION:

```java
public class AccentureCountEvenOdd {
    public static void countEvenOdd(int[] arr) {
        int even = 0, odd = 0;
        for (int x : arr) {
            if (x % 2 == 0) even++;
            else odd++;
        }
        System.out.println("Even: " + even + ", Odd: " + odd);
    }
    public static void main(String[] args) {
        countEvenOdd(new int[]{1, 2, 3, 4, 5, 6, 7}); // Even: 3, Odd: 4
    }
}
```

DRY RUN WALKTHROUGH:

Evens: 2, 4, 6 (count=3). Odds: 1, 3, 5, 7 (count=4).

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Negative numbers; zeros.

#3: Find a Missing Number

Arrays

PROBLEM STATEMENT:

Find the missing number from 0 to n in the array.

SAMPLE INPUT & OUTPUT:

```
Input: [3, 0, 1]
Output: 2
```

ALGORITHM & INTUITION:

XOR index with values.

COMPLETE JAVA 21 SOLUTION:

```java
public class AccentureMissingNumber {
    public static int missingNumber(int[] nums) {
        int xor = 0;
        for (int i = 0; i <= nums.length; i++) xor ^= i;
        for (int x : nums) xor ^= x;
        return xor;
    }
    public static void main(String[] args) {
        System.out.println(missingNumber(new int[]{3, 0, 1})); // 2
    }
}
```

DRY RUN WALKTHROUGH:

(0^1^2^3) ^ (3^0^1) = 2.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Missing 0 or n.

#4: Move Zeros to the End

Arrays

PROBLEM STATEMENT:

Move all zeros in an array to the end in-place.

SAMPLE INPUT & OUTPUT:

```
Input: [0, 1, 0, 3, 12]
Output: [1, 3, 12, 0, 0]
```

ALGORITHM & INTUITION:

Two-pointer placement of non-zero elements.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class AccentureMoveZeros {
    public static void moveZeroes(int[] nums) {
        int pos = 0;
        for (int x : nums) if (x != 0) nums[pos++] = x;
        while (pos < nums.length) nums[pos++] = 0;
    }
    public static void main(String[] args) {
        int[] arr = {0, 1, 0, 3, 12};
        moveZeroes(arr);
        System.out.println(Arrays.toString(arr)); // [1, 3, 12, 0, 0]
    }
}
```

DRY RUN WALKTHROUGH:

[0, 1, 0, 3, 12] -> [1, 3, 12, 0, 0].

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: All zeros; no zeros.

#5: Find a Pair with a Target Sum

Hashing

PROBLEM STATEMENT:

Determine if there exists a pair with sum equal to target.

SAMPLE INPUT & OUTPUT:

```
Input: arr = [10, 15, 3, 7], target = 17
Output: true (10 + 7)
```

ALGORITHM & INTUITION:

HashSet lookup for complement = target - x.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.HashSet;

public class AccenturePairSum {
    public static boolean hasPair(int[] arr, int target) {
        HashSet<Integer> seen = new HashSet<>();
        for (int x : arr) {
            if (seen.contains(target - x)) return true;
            seen.add(x);
        }
        return false;
    }
    public static void main(String[] args) {
        System.out.println(hasPair(new int[]{10, 15, 3, 7}, 17)); // true
    }
}
```

DRY RUN WALKTHROUGH:

10 seen -> 15 seen -> 3 seen -> 7 checks (17-7=10 in set) -> true.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Target sum using same element twice.

#6: Sort an Array by Frequency

Sorting / Hashing

PROBLEM STATEMENT:

Sort array elements in decreasing order of frequency. If two numbers have the same frequency, the smaller number comes first.

SAMPLE INPUT & OUTPUT:

```
Input: [2, 5, 2, 8, 5, 6, 8, 8]
Output: [8, 8, 8, 2, 2, 5, 5, 6]
```

ALGORITHM & INTUITION:

Count frequencies in a HashMap. Sort list using a custom Comparator: compare frequencies descending, then values ascending.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class SortByFrequency {
    public static int[] frequencySort(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int x : nums) map.put(x, map.getOrDefault(x, 0) + 1);

        List<Integer> list = new ArrayList<>();
        for (int x : nums) list.add(x);

        list.sort((a, b) -> {
            int freqA = map.get(a), freqB = map.get(b);
            if (freqA != freqB) return Integer.compare(freqB, freqA); // Descending frequency
            return Integer.compare(a, b); // Ascending value
        });

        for (int i = 0; i < nums.length; i++) nums[i] = list.get(i);
        return nums;
    }

    public static void main(String[] args) {
        int[] arr = {2, 5, 2, 8, 5, 6, 8, 8};
        System.out.println(Arrays.toString(frequencySort(arr)));
        // Output: [8, 8, 8, 2, 2, 5, 5, 6]
    }
}
```

DRY RUN WALKTHROUGH:

Frequencies: 8:3, 2:2, 5:2, 6:1. Sorted order: 8, 8, 8, 2, 2, 5, 5, 6.

TIME COMPLEXITY:
O(N log N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: All elements have frequency 1; all elements same.

#7: Check Whether Two Strings Are Anagrams

Strings

PROBLEM STATEMENT:

Check if two strings contain same letters in different order.

SAMPLE INPUT & OUTPUT:

```
Input: "triangle", "integral"
Output: true
```

ALGORITHM & INTUITION:

26-bucket frequency comparison.

COMPLETE JAVA 21 SOLUTION:

```java
public class AccentureAnagram {
    public static boolean isAnagram(String s1, String s2) {
        if (s1.length() != s2.length()) return false;
        int[] c = new int[26];
        for (int i = 0; i < s1.length(); i++) {
            c[s1.charAt(i) - 'a']++;
            c[s2.charAt(i) - 'a']--;
        }
        for (int v : c) if (v != 0) return false;
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isAnagram("triangle", "integral")); // true
    }
}
```

DRY RUN WALKTHROUGH:

triangle & integral: counts match.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Length mismatch.

#8: Remove Duplicate Characters

Strings

PROBLEM STATEMENT:

Given a string, remove all duplicate characters, keeping only their first occurrence.

SAMPLE INPUT & OUTPUT:

```
Input: "programming"
Output: "progami"
```

ALGORITHM & INTUITION:

Use a LinkedHashSet<Character> or boolean array of seen characters to maintain insertion order.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class RemoveDuplicateChars {
    public static String removeDuplicates(String s) {
        boolean[] seen = new boolean[256];
        StringBuilder sb = new StringBuilder();
        for (char c : s.toCharArray()) {
            if (!seen[c]) {
                seen[c] = true;
                sb.append(c);
            }
        }
        return sb.toString();
    }
    public static void main(String[] args) {
        System.out.println(removeDuplicates("programming")); // progami
    }
}
```

DRY RUN WALKTHROUGH:

p->r->o->g->r(skip)->a->m->m(skip)->i->n->g(skip) -> "progamin".

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1) 256 array.

EDGE CASES: All characters same "aaaa" -> "a".

#9: Reverse Words in a Sentence

Strings

PROBLEM STATEMENT:

Reverse word sequence while keeping characters inside words in order.

SAMPLE INPUT & OUTPUT:

```
Input: "Accenture Innovation Delivered"
Output: "Delivered Innovation Accenture"
```

ALGORITHM & INTUITION:

Tokenize by whitespace, traverse array in reverse order.

COMPLETE JAVA 21 SOLUTION:

```java
public class AccentureReverseWords {
    public static String reverseWords(String s) {
        String[] w = s.trim().split("\\s+");
        StringBuilder sb = new StringBuilder();
        for (int i = w.length - 1; i >= 0; i--) {
            sb.append(w[i]).append(i == 0 ? "" : " ");
        }
        return sb.toString();
    }
    public static void main(String[] args) {
        System.out.println(reverseWords("Accenture Innovation Delivered"));
    }
}
```

DRY RUN WALKTHROUGH:

Delivered Innovation Accenture.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Extra spaces.

#10: Find the First Non-Repeating Character

Hashing

PROBLEM STATEMENT:

Find index of first unique character in string.

SAMPLE INPUT & OUTPUT:

```
Input: "accenture"
Output: 0 ('a')
```

ALGORITHM & INTUITION:

Frequency count array, second scan for first with frequency 1.

COMPLETE JAVA 21 SOLUTION:

```java
public class AccentureFirstNonRepeating {
    public static int firstUniq(String s) {
        int[] freq = new int[256];
        for (char c : s.toCharArray()) freq[c]++;
        for (int i = 0; i < s.length(); i++) {
            if (freq[s.charAt(i)] == 1) return i;
        }
        return -1;
    }
    public static void main(String[] args) {
        System.out.println(firstUniq("accenture")); // 0
    }
}
```

DRY RUN WALKTHROUGH:

'a' count = 1 -> returns index 0.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: No unique characters.

#11: Find the Longest Common Prefix

Strings

PROBLEM STATEMENT:

Write a function to find the longest common prefix string amongst an array of strings. If there is no common prefix, return an empty string "".

SAMPLE INPUT & OUTPUT:

```
Input: strs = ["flower", "flow", "flight"]
Output: "fl"
```

ALGORITHM & INTUITION:

Horizontal scanning: Take the first string as candidate prefix. For each subsequent string, while the string does not start with prefix (str.indexOf(prefix) != 0), trim the last character off prefix. If prefix becomes empty, return "".

COMPLETE JAVA 21 SOLUTION:

```java
public class LongestCommonPrefix {
    public static String longestCommonPrefix(String[] strs) {
        if (strs == null || strs.length == 0) return "";
        String prefix = strs[0];

        for (int i = 1; i < strs.length; i++) {
            while (strs[i].indexOf(prefix) != 0) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) return "";
            }
        }
        return prefix;
    }

    public static void main(String[] args) {
        String[] strs = {"flower", "flow", "flight"};
        System.out.println("Common Prefix: \"" + longestCommonPrefix(strs) + "\""); // "fl"
    }
}
```

DRY RUN WALKTHROUGH:

prefix = "flower"
- "flow": not found -> "flowe" -> "flow" (matches at index 0)
- "flight": prefix "flow" -> "flo" -> "fl" (matches)
Result: "fl".

TIME COMPLEXITY:
O(S) where S is the sum of all characters in all strings.

SPACE COMPLEXITY:
O(1) auxiliary space.

EDGE CASES: Array with 1 string; no common prefix ["dog", "racecar"]; empty array.

#12: Longest Substring Without Repeating Characters

Sliding Window

PROBLEM STATEMENT:

Find length of longest contiguous substring containing all distinct characters.

SAMPLE INPUT & OUTPUT:

```
Input: "abcabcbb"
Output: 3 ("abc")
```

ALGORITHM & INTUITION:

Sliding window with character index tracking.

COMPLETE JAVA 21 SOLUTION:

```java
public class AccentureLongestUnique {
    public static int lengthOfLongestSubstring(String s) {
        int[] last = new int[256];
        java.util.Arrays.fill(last, -1);
        int max = 0, l = 0;
        for (int r = 0; r < s.length(); r++) {
            char c = s.charAt(r);
            if (last[c] >= l) l = last[c] + 1;
            last[c] = r;
            max = Math.max(max, r - l + 1);
        }
        return max;
    }
    public static void main(String[] args) {
        System.out.println(lengthOfLongestSubstring("abcabcbb")); // 3
    }
}
```

DRY RUN WALKTHROUGH:

Expands "abc" (len 3). Max length = 3.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Empty string.

#13: Check Balanced Parentheses

Stack

PROBLEM STATEMENT:

Verify whether bracket sequence is properly nested and closed.

SAMPLE INPUT & OUTPUT:

```
Input: "{[]}()"
Output: true
```

ALGORITHM & INTUITION:

Stack push matching closing brackets.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Stack;

public class AccentureBalancedBrackets {
    public static boolean isValid(String s) {
        Stack<Character> st = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(') st.push(')');
            else if (c == '{') st.push('}');
            else if (c == '[') st.push(']');
            else if (st.isEmpty() || st.pop() != c) return false;
        }
        return st.isEmpty();
    }
    public static void main(String[] args) {
        System.out.println(isValid("{[]}()")); // true
    }
}
```

DRY RUN WALKTHROUGH:

Matches '{', '[', ']', '}', '(', ')' -> true.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Odd length strings.

#14: Find the Next Greater Element

Monotonic Stack

PROBLEM STATEMENT:

Given an array nums, find the next greater element for each element. The Next Greater Element for an element x is the first greater element to its right. If it doesn't exist, output -1.

SAMPLE INPUT & OUTPUT:

```
Input: arr = [4, 5, 2, 25]
Output: [5, 25, 25, -1]
```

ALGORITHM & INTUITION:

Monotonic Stack: Traverse the array from right to left (n-1 down to 0). While the stack is not empty and stack.peek() <= arr[i], pop elements from the stack. The top element is now the next greater element (or -1 if stack is empty). Push arr[i] onto the stack.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class NextGreaterElement {
    public static int[] nextGreater(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        Stack<Integer> stack = new Stack<>();

        for (int i = n - 1; i >= 0; i--) {
            while (!stack.isEmpty() && stack.peek() <= nums[i]) {
                stack.pop();
            }
            result[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(nums[i]);
        }
        return result;
    }

    public static void main(String[] args) {
        int[] arr = {4, 5, 2, 25};
        System.out.println(Arrays.toString(nextGreater(arr))); // [5, 25, 25, -1]
    }
}
```

DRY RUN WALKTHROUGH:

arr = [4, 5, 2, 25]
- i=3 (25): stack empty -> result[3]=-1, push 25
- i=2 (2): stack.peek()=25 > 2 -> result[2]=25, push 2
- i=1 (5): pop 2, peek()=25 > 5 -> result[1]=25, push 5
- i=0 (4): stack.peek()=5 > 4 -> result[0]=5, push 4.
Output: [5, 25, 25, -1].

TIME COMPLEXITY:
O(N) — each element is pushed and popped at most once.

SPACE COMPLEXITY:
O(N) for stack.

EDGE CASES: Array sorted descending [5, 4, 3, 2, 1] (all -1 except none); all identical elements.

#15: Implement a Stack and Queue

Data Structures

PROBLEM STATEMENT:

Implement standard Stack (LIFO: push, pop, peek, isEmpty) using an array, and standard Queue (FIFO: enqueue, dequeue, peek, isEmpty) using an array.

SAMPLE INPUT & OUTPUT:

```
Stack: push(10), push(20), pop() -> 20
Queue: enqueue(10), enqueue(20), dequeue() -> 10
```

ALGORITHM & INTUITION:

Stack: top pointer tracking top element.
Queue: circular array buffer with front, rear, and size pointers.

COMPLETE JAVA 21 SOLUTION:

```
public class CustomStackQueue {
    static class MyStack {
        private int[] arr;
        private int top;
        MyStack(int cap) { arr = new int[cap]; top = -1; }
        public void push(int x) { arr[++top] = x; }
        public int pop() { return arr[top--]; }
        public int peek() { return arr[top]; }
        public boolean isEmpty() { return top == -1; }
    }

    static class MyQueue {
        private int[] arr;
        private int front, rear, size, cap;
        MyQueue(int cap) {
            this.cap = cap; arr = new int[cap];
            front = 0; rear = -1; size = 0;
        }
        public void enqueue(int x) {
            rear = (rear + 1) % cap; arr[rear] = x; size++;
        }
        public int dequeue() {
            int val = arr[front]; front = (front + 1) % cap; size--; return val;
        }
        public boolean isEmpty() { return size == 0; }
    }

    public static void main(String[] args) {
        MyStack st = new MyStack(5);
        st.push(10); st.push(20);
        System.out.println("Stack Pop: " + st.pop()); // 20

        MyQueue q = new MyQueue(5);
        q.enqueue(10); q.enqueue(20);
        System.out.println("Queue Dequeue: " + q.dequeue()); // 10
    }
}
```

DRY RUN WALKTHROUGH:

LIFO vs FIFO properties demonstrated.

TIME COMPLEXITY:
O(1) for all push/pop/enqueue/dequeue operations.

SPACE COMPLEXITY:
O(Capacity) memory allocated.

EDGE CASES: Underflow on empty stack/queue.

#16: Best Time to Buy and Sell Stock

Greedy

PROBLEM STATEMENT:

Find maximum profit possible from single stock transaction.

SAMPLE INPUT & OUTPUT:

```
Input: [7, 1, 5, 3, 6, 4]
Output: 5
```

ALGORITHM & INTUITION:

Track minPrice and maxProfit in single pass.

COMPLETE JAVA 21 SOLUTION:

```
public class AccentureStock {
    public static int maxProfit(int[] prices) {
        int min = Integer.MAX_VALUE, max = 0;
        for (int p : prices) {
            if (p < min) min = p;
            else if (p - min > max) max = p - min;
        }
        return max;
    }
    public static void main(String[] args) {
        System.out.println(maxProfit(new int[]{7, 1, 5, 3, 6, 4})); // 5
    }
}
```

DRY RUN WALKTHROUGH:

Min=1, max sell=6 -> 6-1=5.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Decreasing prices.

#17: Merge Overlapping Intervals

Intervals

PROBLEM STATEMENT:

Merge contiguous overlapping intervals.

SAMPLE INPUT & OUTPUT:

```
Input: [[1, 3], [2, 6], [8, 10]]
Output: [[1, 6], [8, 10]]
```

ALGORITHM & INTUITION:

Sort by start time, combine if next.start <= curr.end.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.*;

public class AccentureMergeIntervals {
    public static int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
        List<int[]> res = new ArrayList<>();
        int[] curr = intervals[0];
        res.add(curr);
        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] <= curr[1]) {
                curr[1] = Math.max(curr[1], intervals[i][1]);
            } else {
                curr = intervals[i];
                res.add(curr);
            }
        }
        return res.toArray(new int[res.size()][]);
    }
    public static void main(String[] args) {
        int[][] res = merge(new int[][]{{1, 3}, {2, 6}, {8, 10}});
        for (int[] iv : res) System.out.print(Arrays.toString(iv) + " ");
    }
}
```

DRY RUN WALKTHROUGH:

[1,3] and [2,6] merge to [1,6].

TIME COMPLEXITY:
O(N log N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: No overlaps.

#18: Activity Selection Problem

Greedy

PROBLEM STATEMENT:

Given start and finish times of n activities, select the maximum number of activities that can be performed by a single person, assuming that a person can only work on a single activity at a time.

SAMPLE INPUT & OUTPUT:

```
Input: start = [1, 3, 0, 5, 8, 5], finish = [2, 4, 6, 7, 9, 9]
Output: 4
Explanation: Activities (1, 2), (3, 4), (5, 7), (8, 9) can be selected.
```

ALGORITHM & INTUITION:

Greedy Strategy: Always pick the activity that finishes earliest! This leaves the maximum possible remaining time for subsequent activities.
1. Sort activities by finish time ascending.
2. Select first activity. Set lastFinish = finish[0].
3. For remaining activities: if start[i] >= lastFinish, select activity and update lastFinish = finish[i].

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class ActivitySelection {
    static class Activity {
        int start, finish;
        Activity(int start, int finish) {
            this.start = start;
            this.finish = finish;
        }
    }

    public static int maxActivities(int[] start, int[] finish) {
        int n = start.length;
        Activity[] acts = new Activity[n];
        for (int i = 0; i < n; i++) acts[i] = new Activity(start[i], finish[i]);

        // Sort by finish time ascending
        Arrays.sort(acts, (a, b) -> Integer.compare(a.finish, b.finish));

        int count = 1;
        int lastFinish = acts[0].finish;

        for (int i = 1; i < n; i++) {
            if (acts[i].start >= lastFinish) {
                count++;
                lastFinish = acts[i].finish;
            }
        }
        return count;
    }

    public static void main(String[] args) {
        int[] start = {1, 3, 0, 5, 8, 5};
        int[] finish = {2, 4, 6, 7, 9, 9};
        System.out.println("Max Activities: " + maxActivities(start, finish)); // 4
    }
}
```

DRY RUN WALKTHROUGH:

Sorted by finish: (1,2), (3,4), (0,6), (5,7), (8,9), (5,9)
Select (1,2) -> lastFinish=2
Select (3,4) (3 >= 2) -> lastFinish=4
Skip (0,6) (0 < 4)
Select (5,7) (5 >= 4) -> lastFinish=7
Select (8,9) (8 >= 7) -> lastFinish=9. Total = 4.

TIME COMPLEXITY:
O(N log N) — dominated by sorting.

SPACE COMPLEXITY:
O(N) for activity objects array.

EDGE CASES: All activities start and end at same time; strictly disjoint activities.

#19: Coin Change

Dynamic Programming

PROBLEM STATEMENT:

Find minimum coins needed to make target amount.

SAMPLE INPUT & OUTPUT:

```
Input: coins = [1, 2, 5], amount = 11
Output: 3 (5 + 5 + 1)
```

ALGORITHM & INTUITION:

1D DP: dp[i] = min(dp[i], 1 + dp[i - c]).

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class AccentureCoinChange {
    public static int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;
        for (int i = 1; i <= amount; i++) {
            for (int c : coins) {
                if (c <= i) dp[i] = Math.min(dp[i], 1 + dp[i - c]);
            }
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
    public static void main(String[] args) {
        System.out.println(coinChange(new int[]{1, 2, 5}, 11)); // 3
    }
}
```

DRY RUN WALKTHROUGH:

dp[11] = 3 coins.

TIME COMPLEXITY:
O(amount * coins.length).

SPACE COMPLEXITY:
O(amount).

EDGE CASES: amount = 0 (0).

#20: Longest Common Subsequence

Dynamic Programming

PROBLEM STATEMENT:

Find the length of the longest common subsequence of two strings.

SAMPLE INPUT & OUTPUT:

```
Input: "abcde", "ace"
Output: 3
```

ALGORITHM & INTUITION:

2D DP matrix matching characters.

COMPLETE JAVA 21 SOLUTION:

```java
public class AccentureLCS {
    public static int lcs(String s1, String s2) {
        int m = s1.length(), n = s2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (s1.charAt(i - 1) == s2.charAt(j - 1)) dp[i][j] = 1 + dp[i - 1][j - 1];
                else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
        return dp[m][n];
    }
    public static void main(String[] args) {
        System.out.println(lcs("abcde", "ace")); // 3
    }
}
```

DRY RUN WALKTHROUGH:

LCS of "abcde" and "ace" is "ace" of length 3.

TIME COMPLEXITY:
O(M * N).

SPACE COMPLEXITY:
O(M * N).

EDGE CASES: Disjoint strings.

CHAPTER 6

Capgemini — Exceller & Analyst Track

Technical Assessment Coding Solutions & Algorithms

Target Focus: Capgemini Exceller, Senior Analyst & Analyst Drives

Key recurring questions include Leaders in an Array, Moore's Voting Majority Element, Minimum Platforms Required, and Merge Sort divide-and-conquer.

Questions in this Section (20 Problems):

#1: Find the Second-Largest Element

Arrays

PROBLEM STATEMENT:

Find the second-largest distinct element in an array of integers.

SAMPLE INPUT & OUTPUT:

```
Input: [12, 35, 1, 10, 34, 1]
Output: 34
```

ALGORITHM & INTUITION:

Single-pass comparison updating largest and second-largest.

COMPLETE JAVA 21 SOLUTION:

```java
public class CapgeminiSecondLargest {
    public static int findSecondLargest(int[] arr) {
        int l = Integer.MIN_VALUE, s = Integer.MIN_VALUE;
        for (int x : arr) {
            if (x > l) { s = l; l = x; }
            else if (x < l && x > s) { s = x; }
        }
        return s == Integer.MIN_VALUE ? -1 : s;
    }
    public static void main(String[] args) {
        System.out.println(findSecondLargest(new int[]{12, 35, 1, 10, 34, 1})); // 34
    }
}
```

DRY RUN WALKTHROUGH:

Largest: 35, Second: 34.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Identical elements.

#2: Find the Missing Number

Arrays

PROBLEM STATEMENT:

Find missing integer in range 0 to n.

SAMPLE INPUT & OUTPUT:

```
Input: [3, 0, 1]
Output: 2
```

ALGORITHM & INTUITION:

XOR cancellation.

COMPLETE JAVA 21 SOLUTION:

```java
public class CapgeminiMissingNumber {
    public static int findMissing(int[] nums) {
        int x = 0;
        for (int i = 0; i <= nums.length; i++) x ^= i;
        for (int num : nums) x ^= num;
        return x;
    }
    public static void main(String[] args) {
        System.out.println(findMissing(new int[]{3, 0, 1})); // 2
    }
}
```

DRY RUN WALKTHROUGH:

XOR yields 2.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Missing edge numbers.

#3: Remove Duplicate Elements

Arrays

PROBLEM STATEMENT:

Remove duplicates from array in-place.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 1, 2, 3, 3]
Output: [1, 2, 3]
```

ALGORITHM & INTUITION:

Two-pointer index copy.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class CapgeminiRemoveDuplicates {
    public static int removeDuplicates(int[] nums) {
        if (nums.length == 0) return 0;
        int idx = 0;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] != nums[idx]) nums[++idx] = nums[i];
        }
        return idx + 1;
    }
    public static void main(String[] args) {
        int[] arr = {1, 1, 2, 3, 3};
        int len = removeDuplicates(arr);
        System.out.println(Arrays.toString(Arrays.copyOfRange(arr, 0, len))); // [1, 2, 3]
    }
}
```

DRY RUN WALKTHROUGH:

Unique count: 3.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: No duplicates.

#4: Left-Rotate an Array by D Positions

Arrays

PROBLEM STATEMENT:

Rotate array to left by D steps.

SAMPLE INPUT & OUTPUT:

```
Input: arr = [1, 2, 3, 4, 5], d = 2
Output: [3, 4, 5, 1, 2]
```

ALGORITHM & INTUITION:

Reverse algorithm for LEFT rotation:
1. d = d % n.
2. Reverse first d elements (0 to d-1).
3. Reverse remaining n-d elements (d to n-1).
4. Reverse entire array (0 to n-1).

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class LeftRotateArray {
    public static void leftRotate(int[] arr, int d) {
        int n = arr.length;
        d %= n;
        reverse(arr, 0, d - 1);
        reverse(arr, d, n - 1);
        reverse(arr, 0, n - 1);
    }

    private static void reverse(int[] arr, int l, int r) {
        while (l < r) {
            int t = arr[l]; arr[l++] = arr[r]; arr[r--] = t;
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        leftRotate(arr, 2);
        System.out.println(Arrays.toString(arr)); // [3, 4, 5, 1, 2]
    }
}
```

DRY RUN WALKTHROUGH:

[1, 2, 3, 4, 5], d=2
- Rev 0..1: [2, 1, 3, 4, 5]
- Rev 2..4: [2, 1, 5, 4, 3]
- Rev 0..4: [3, 4, 5, 1, 2].

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1) in-place.

EDGE CASES: d = 0; d >= n.

#5: Find Leaders in an Array

Arrays

PROBLEM STATEMENT:

An element is a Leader if it is greater than or equal to all elements to its right. The rightmost element is always a leader. Find all leaders and return them in order of appearance.

SAMPLE INPUT & OUTPUT:

```
Input: arr = [16, 17, 4, 3, 5, 2]
Output: [17, 5, 2]
```

ALGORITHM & INTUITION:

Scan the array backwards from right to left (index n-1 down to 0). Maintain maxFromRight = Integer.MIN_VALUE. If arr[i] >= maxFromRight, arr[i] is a leader! Add to result list and update maxFromRight. Reverse the list at the end to restore original order.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class LeadersInArray {
    public static List<Integer> findLeaders(int[] arr) {
        List<Integer> leaders = new ArrayList<>();
        int maxFromRight = Integer.MIN_VALUE;

        for (int i = arr.length - 1; i >= 0; i--) {
            if (arr[i] >= maxFromRight) {
                leaders.add(arr[i]);
                maxFromRight = arr[i];
            }
        }
        Collections.reverse(leaders);
        return leaders;
    }

    public static void main(String[] args) {
        int[] arr = {16, 17, 4, 3, 5, 2};
        System.out.println("Leaders: " + findLeaders(arr)); // [17, 5, 2]
    }
}
```

DRY RUN WALKTHROUGH:

arr = [16, 17, 4, 3, 5, 2]
- i=5 (2): 2 >= MIN -> leader=2, max=2
- i=4 (5): 5 >= 2 -> leader=5, max=5
- i=3 (3): 3 < 5 -> skip
- i=2 (4): 4 < 5 -> skip
- i=1 (17): 17 >= 5 -> leader=17, max=17
- i=0 (16): 16 < 17 -> skip

Reversed: [17, 5, 2].

TIME COMPLEXITY:
O(N) — single backwards pass.

SPACE COMPLEXITY:
O(N) for result list.

EDGE CASES: Strictly increasing array [1, 2, 3, 4] (only 4 is leader); strictly decreasing array [4, 3, 2, 1] (all are leaders).

#6: Sort 0s, 1s and 2s

Two pointers

PROBLEM STATEMENT:

Sort array of 0s, 1s, and 2s in-place.

SAMPLE INPUT & OUTPUT:

```
Input: [2, 0, 1]
Output: [0, 1, 2]
```

ALGORITHM & INTUITION:

Dutch National Flag 3-pointer algorithm (low, mid, high).

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class CapgeminiSort012 {
    public static void sort012(int[] a) {
        int l = 0, m = 0, h = a.length - 1;
        while (m <= h) {
            if (a[m] == 0) {
                int t = a[l]; a[l] = a[m]; a[m] = t; l++; m++;
            } else if (a[m] == 1) {
                m++;
            } else {
                int t = a[m]; a[m] = a[h]; a[h] = t; h--;
            }
        }
    }
    public static void main(String[] args) {
        int[] a = {2, 0, 1};
        sort012(a);
        System.out.println(Arrays.toString(a)); // [0, 1, 2]
    }
}
```

DRY RUN WALKTHROUGH:

Arranged into [0, 1, 2].

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Single element; all same.

#7: Maximum Subarray Sum

Kadane's algorithm

PROBLEM STATEMENT:

Find the maximum subarray sum using Kadane's algorithm.

SAMPLE INPUT & OUTPUT:

```
Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6
```

ALGORITHM & INTUITION:

Kadane's algorithm.

COMPLETE JAVA 21 SOLUTION:

```java
public class CapgeminiKadane {
    public static int maxSubArray(int[] nums) {
        int c = nums[0], m = nums[0];
        for (int i = 1; i < nums.length; i++) {
            c = Math.max(nums[i], c + nums[i]);
            m = Math.max(m, c);
        }
        return m;
    }
    public static void main(String[] args) {
        System.out.println(maxSubArray(new int[]{-2, 1, -3, 4, -1, 2, 1, -5, 4})); // 6
    }
}
```

DRY RUN WALKTHROUGH:

Subarray [4, -1, 2, 1] sum = 6.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Negative array.

#8: Find the Majority Element (Moore's Voting Algorithm)

Arrays / Voting

PROBLEM STATEMENT:

Given an array nums of size n, return the majority element (the element that appears more than # n / 2# times). Assume that the majority element always exists.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [2, 2, 1, 1, 1, 2, 2]
Output: 2
```

ALGORITHM & INTUITION:

Boyer-Moore Voting Algorithm:
Maintain candidate and count = 0. Iterate through nums: if count == 0, candidate = num. If num == candidate, count++; else count--.
The majority element has frequency > n/2, so its occurrences cancel out all other elements combined, leaving it as candidate.

COMPLETE JAVA 21 SOLUTION:

```java
public class MajorityElement {
    public static int majorityElement(int[] nums) {
        int candidate = nums[0];
        int count = 0;

        for (int num : nums) {
            if (count == 0) {
                candidate = num;
            }
            count += (num == candidate) ? 1 : -1;
        }
        return candidate;
    }

    public static void main(String[] args) {
        int[] nums = {2, 2, 1, 1, 1, 2, 2};
        System.out.println("Majority Element: " + majorityElement(nums)); // 2
    }
}
```

DRY RUN WALKTHROUGH:

nums = [2, 2, 1, 1, 1, 2, 2]
- num=2: cand=2, count=1
- num=2: count=2
- num=1: count=1
- num=1: count=0
- num=1: cand=1, count=1
- num=2: count=0
- num=2: cand=2, count=1.

Final candidate = 2.

TIME COMPLEXITY:
O(N) — single pass.

SPACE COMPLEXITY:
O(1) auxiliary space.

EDGE CASES: Array of size 1; all elements identical.

#9: Check Palindrome

Strings

PROBLEM STATEMENT:

Determine whether a string is a palindrome.

SAMPLE INPUT & OUTPUT:

```
Input: "radar"
Output: true
```

ALGORITHM & INTUITION:

Two pointers inwards.

COMPLETE JAVA 21 SOLUTION:

```java
public class CapgeminiPalindrome {
    public static boolean isPal(String s) {
        int l = 0, r = s.length() - 1;
        while (l < r) if (s.charAt(l++) != s.charAt(r--)) return false;
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isPal("radar")); // true
    }
}
```

DRY RUN WALKTHROUGH:

r=r, a=a, d=d -> true.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Single character.

#10: Check Anagrams

Strings

PROBLEM STATEMENT:

Check if two strings are anagrams of each other.

SAMPLE INPUT & OUTPUT:

```
Input: "debitcard", "badcredit"
Output: true
```

ALGORITHM & INTUITION:

Frequency counter of 26 letters.

COMPLETE JAVA 21 SOLUTION:

```java
public class CapgeminiAnagram {
    public static boolean isAnagram(String s1, String s2) {
        if (s1.length() != s2.length()) return false;
        int[] f = new int[26];
        for (int i = 0; i < s1.length(); i++) {
            f[s1.charAt(i) - 'a']++; f[s2.charAt(i) - 'a']--;
        }
        for (int x : f) if (x != 0) return false;
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isAnagram("debitcard", "badcredit")); // true
    }
}
```

DRY RUN WALKTHROUGH:

All letter frequencies match.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Length mismatch.

#11: Count the Frequency of Each Character

Hashing

PROBLEM STATEMENT:

Count occurrences of every character in order of appearance.

SAMPLE INPUT & OUTPUT:

```
Input: "capgemini"
Output: c:1, a:1, p:1, g:1, e:1, m:1, i:2, n:1
```

ALGORITHM & INTUITION:

LinkedHashMap for order preservation.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class CapgeminiCharFreq {
    public static void printFrequencies(String s) {
        Map<Character, Integer> map = new LinkedHashMap<>();
        for (char c : s.toCharArray()) map.put(c, map.getOrDefault(c, 0) + 1);
        for (Map.Entry<Character, Integer> e : map.entrySet()) {
            System.out.print(e.getKey() + ":" + e.getValue() + " ");
        }
        System.out.println();
    }
    public static void main(String[] args) {
        printFrequencies("capgemini"); // c:1 a:1 p:1 g:1 e:1 m:1 i:2 n:1
    }
}
```

DRY RUN WALKTHROUGH:

'i' appears twice, others once.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(K).

EDGE CASES: Empty string.

#12: Remove Spaces and Special Characters

Strings

PROBLEM STATEMENT:

Given a string s, remove all spaces and special characters, leaving only lowercase/uppercase English letters and digits.

SAMPLE INPUT & OUTPUT:

```
Input: "Capgemini @ 2026 # Exceller!"
Output: "Capgemini2026Exceller"
```

ALGORITHM & INTUITION:

Traverse characters and append to StringBuilder only if Character.isLetterOrDigit(c) is true.

COMPLETE JAVA 21 SOLUTION:

```java
public class CleanString {
    public static String clean(String s) {
        StringBuilder sb = new StringBuilder();
        for (char c : s.toCharArray()) {
            if (Character.isLetterOrDigit(c)) {
                sb.append(c);
            }
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        String s = "Capgemini @ 2026 # Exceller!";
        System.out.println("Cleaned: " + clean(s)); // Capgemini2026Exceller
    }
}
```

DRY RUN WALKTHROUGH:

Filters out ' ', '@', '#', '!'. Keeps only alphanumeric characters.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N) for output.

EDGE CASES: String with no letters/digits (returns ""); already clean string.

#13: Find the First Non-Repeating Character

Hashing

PROBLEM STATEMENT:

Return index of first non-repeating character in string.

SAMPLE INPUT & OUTPUT:

```
Input: "geeksforgeeks"
Output: 5 ('f')
```

ALGORITHM & INTUITION:

Count array + second pass.

COMPLETE JAVA 21 SOLUTION:

```
public class CapgeminiFirstUnique {
    public static int firstUniqChar(String s) {
        int[] freq = new int[256];
        for (char c : s.toCharArray()) freq[c]++;
        for (int i = 0; i < s.length(); i++) {
            if (freq[s.charAt(i)] == 1) return i;
        }
        return -1;
    }
    public static void main(String[] args) {
        System.out.println(firstUniqChar("geeksforgeeks")); // 5 ('f')
    }
}
```

DRY RUN WALKTHROUGH:

'f' at index 5 is first unique character.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: All repeating.

#14: Reverse Words in a Sentence

Strings

PROBLEM STATEMENT:

Reverse words in sentence.

SAMPLE INPUT & OUTPUT:

```
Input: "Consulting Technology Services"
Output: "Services Technology Consulting"
```

ALGORITHM & INTUITION:

Split by space, traverse in reverse.

COMPLETE JAVA 21 SOLUTION:

```
public class CapgeminiReverseWords {
    public static String reverseWords(String s) {
        String[] w = s.trim().split("\\s+");
        StringBuilder sb = new StringBuilder();
        for (int i = w.length - 1; i >= 0; i--) {
            sb.append(w[i]).append(i == 0 ? "" : " ");
        }
        return sb.toString();
    }
    public static void main(String[] args) {
        System.out.println(reverseWords("Consulting Technology Services"));
    }
}
```

DRY RUN WALKTHROUGH:

Services Technology Consulting.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Single word.

#15: Binary Search

Searching

PROBLEM STATEMENT:

Search sorted array in O(log N).

SAMPLE INPUT & OUTPUT:

```
Input: [1, 3, 5, 7, 9], target = 7
Output: 3
```

ALGORITHM & INTUITION:

low, high, mid calculation.

COMPLETE JAVA 21 SOLUTION:

```java
public class CapgeminiBinarySearch {
    public static int search(int[] arr, int target) {
        int l = 0, h = arr.length - 1;
        while (l <= h) {
            int m = l + (h - l) / 2;
            if (arr[m] == target) return m;
            if (arr[m] < target) l = m + 1;
            else h = m - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        System.out.println(search(new int[]{1, 3, 5, 7, 9}, 7)); // 3
    }
}
```

DRY RUN WALKTHROUGH:

mid=2 (5) < 7 -> l=3. mid=3 (7) == 7 -> index 3.

TIME COMPLEXITY:
O(log N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Not found (-1).

#16: Merge Sort

Sorting

PROBLEM STATEMENT:

Implement Merge Sort algorithm using Divide and Conquer.

SAMPLE INPUT & OUTPUT:

```
Input: [38, 27, 43, 3, 9, 82, 10]
Output: [3, 9, 10, 27, 38, 43, 82]
```

ALGORITHM & INTUITION:

Divide: recursively split array into two halves at mid.
Conquer: sort both halves.
Combine: merge the two sorted halves using temporary buffer array in O(N) time.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class MergeSortAlgorithm {
    public static void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);
            merge(arr, left, mid, right);
        }
    }

    private static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;
        int[] L = new int[n1];
        int[] R = new int[n2];

        for (int i = 0; i < n1; i++) L[i] = arr[left + i];
        for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) arr[k++] = L[i++];
            else arr[k++] = R[j++];
        }
        while (i < n1) arr[k++] = L[i++];
        while (j < n2) arr[k++] = R[j++];
    }

    public static void main(String[] args) {
        int[] arr = {38, 27, 43, 3, 9, 82, 10};
        mergeSort(arr, 0, arr.length - 1);
        System.out.println("Sorted: " + Arrays.toString(arr));
        // [3, 9, 10, 27, 38, 43, 82]
    }
}
```

DRY RUN WALKTHROUGH:

Recursively divides down to single elements and merges up in sorted order.

TIME COMPLEXITY:
O(N log N) in all cases (best, average, worst).

SPACE COMPLEXITY:
O(N) auxiliary space.

EDGE CASES: Already sorted array; reverse sorted; single element.

#17: Merge Two Sorted Arrays

Arrays

PROBLEM STATEMENT:

Merge two sorted arrays into one.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 4] and [1, 3, 4]
Output: [1, 1, 2, 3, 4, 4]
```

ALGORITHM & INTUITION:

Two pointers stepping through sorted lists.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class CapgeminiMergeArrays {
    public static int[] merge(int[] a, int[] b) {
        int[] res = new int[a.length + b.length];
        int i = 0, j = 0, k = 0;
        while (i < a.length && j < b.length) res[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
        while (i < a.length) res[k++] = a[i++];
        while (j < b.length) res[k++] = b[j++];
        return res;
    }
    public static void main(String[] args) {
        System.out.println(Arrays.toString(merge(new int[]{1, 2, 4}, new int[]{1, 3, 4})));
    }
}
```

DRY RUN WALKTHROUGH:

[1, 1, 2, 3, 4, 4].

TIME COMPLEXITY:
O(A + B).

SPACE COMPLEXITY:
O(A + B).

EDGE CASES: Empty array.

#18: Balanced Parentheses

Stack

PROBLEM STATEMENT:

Validate parentheses bracket matching.

SAMPLE INPUT & OUTPUT:

```
Input: "{[()]}"
Output: true
```

ALGORITHM & INTUITION:

Stack push closing counterpart.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Stack;

public class CapgeminiBrackets {
    public static boolean isValid(String s) {
        Stack<Character> st = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(') st.push(')');
            else if (c == '{') st.push('}');
            else if (c == '[') st.push(']');
            else if (st.isEmpty() || st.pop() != c) return false;
        }
        return st.isEmpty();
    }
    public static void main(String[] args) {
        System.out.println(isValid("{[()]}")); // true
    }
}
```

DRY RUN WALKTHROUGH:

All brackets match -> true.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Mismatched types.

#19: Minimum Platforms Required

Greedy / Sorting

PROBLEM STATEMENT:

Given arrival and departure times of all trains that reach a railway station, find the minimum number of platforms required for the railway station so that no train is kept waiting.

SAMPLE INPUT & OUTPUT:

```
Input: arr = [900, 940, 950, 1100, 1500, 1800], dep = [910, 1200, 1120, 1130, 1900, 2000]
Output: 3
Explanation: Maximum of 3 trains are at the station simultaneously between 9:50 and 11:00.
```

ALGORITHM & INTUITION:

Greedy Interval Sweep:
1. Sort arrival times ascending: arr[].
2. Sort departure times ascending: dep[].
3. Use two pointers i = 0 (arrivals) and j = 0 (departures).
4. If a train arrives before previous departs (arr[i] <= dep[j]), we need an additional platform: platforms++, i++. Update maxPlatforms = max(maxPlatforms, platforms).
5. If a train departs (arr[i] > dep[j]), a platform becomes free: platforms--, j++.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class MinimumPlatforms {
    public static int findPlatform(int[] arr, int[] dep) {
        Arrays.sort(arr);
        Arrays.sort(dep);

        int n = arr.length;
        int platformsNeeded = 0;
        int maxPlatforms = 0;
        int i = 0, j = 0;

        while (i < n && j < n) {
            if (arr[i] <= dep[j]) {
                platformsNeeded++;
                maxPlatforms = Math.max(maxPlatforms, platformsNeeded);
                i++;
            } else {
                platformsNeeded--;
                j++;
            }
        }
        return maxPlatforms;
    }

    public static void main(String[] args) {
        int[] arr = {900, 940, 950, 1100, 1500, 1800};
        int[] dep = {910, 1200, 1120, 1130, 1900, 2000};
        System.out.println("Min Platforms: " + findPlatform(arr, dep)); // 3
    }
}
```

DRY RUN WALKTHROUGH:

Sorted arr: [900, 940, 950, 1100, 1500, 1800]
Sorted dep: [910, 1120, 1130, 1200, 1900, 2000]
Trace: arr 900 (p=1) -> dep 910 (p=0) -> arr 940 (p=1) -> arr 950 (p=2) -> arr 1100 (p=3) -> peak = 3 platforms.

TIME COMPLEXITY:
O(N log N) — dominated by sorting the two arrays.

SPACE COMPLEXITY:
O(1) auxiliary space.

EDGE CASES: Arrival and departure times equal (arr[i] <= dep[j] handles departure safely by allocating platform); single train (1 platform).

#20: Longest Common Subsequence

Dynamic Programming

PROBLEM STATEMENT:

Compute the length of the longest common subsequence.

SAMPLE INPUT & OUTPUT:

```
Input: "stone", "longest"
Output: 3 ("one")
```

ALGORITHM & INTUITION:

2D DP matrix dp[i][j].

COMPLETE JAVA 21 SOLUTION:

```java
public class CapgeminiLCS {
    public static int lcs(String s1, String s2) {
        int m = s1.length(), n = s2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (s1.charAt(i - 1) == s2.charAt(j - 1)) dp[i][j] = 1 + dp[i - 1][j - 1];
                else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
        return dp[m][n];
    }
    public static void main(String[] args) {
        System.out.println(lcs("stone", "longest")); // 3 ("one")
    }
}
```

DRY RUN WALKTHROUGH:

Common subsequence "one" has length 3.

TIME COMPLEXITY:
O(M * N).

SPACE COMPLEXITY:
O(M * N).

EDGE CASES: No matching characters.

CHAPTER 7

Deloitte — Analyst & Technology Consulting

Trees, Linked Lists, Hashing & Algorithmic Design

Target Focus: Analyst, Associate Analyst & Technology Solutions Consultant

Deloitte technical interviews focus deeply on Trees (LCA, height, traversals), Linked Lists (cycle detection, middle node), and Floyd's duplicate number algorithm.

Questions in this Section (20 Problems):

#1: Two Sum — Hashing

PROBLEM STATEMENT:

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [2, 7, 11, 15], target = 9
Output: [0, 1]
```

ALGORITHM & INTUITION:

Hash map storing value to index for O(1) complement lookup.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class DeloitteTwoSum {
    public static int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int comp = target - nums[i];
            if (map.containsKey(comp)) return new int[]{map.get(comp), i};
            map.put(nums[i], i);
        }
        return new int[0];
    }
    public static void main(String[] args) {
        System.out.println(Arrays.toString(twoSum(new int[]{2, 7, 11, 15}, 9))); // [0, 1]
    }
}
```

DRY RUN WALKTHROUGH:

Complement 9-7=2 found in map -> [0, 1].

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Duplicates.

#2: Find the Duplicate Number (Floyd's Cycle Detection)

Arrays / Two pointers

PROBLEM STATEMENT:

Given an array of integers nums containing n + 1 integers where each integer is in the range [1, n] inclusive, there is only one repeated number. Find this repeated number without modifying the array and using only O(1) extra space.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [1, 3, 4, 2, 2]
Output: 2
```

ALGORITHM & INTUITION:

Floyd's Tortoise and Hare Cycle Finding:
Treat the array as a linked list where nums[i] is the pointer to the next node at index nums[i]. Because of the duplicate, a cycle is guaranteed.
Phase 1: slow = nums[0], fast = nums[0]. Advance slow by 1 step (slow = nums[slow]) and fast by 2 steps (fast = nums[nums[fast]]) until slow == fast.
Phase 2: Reset slow to nums[0]. Advance both slow and fast by 1 step at a time. The index where they meet is the entry point of the cycle (the duplicate number).

COMPLETE JAVA 21 SOLUTION:

```java
public class FindDuplicateNumber {
    public static int findDuplicate(int[] nums) {
        int slow = nums[0];
        int fast = nums[0];

        // Phase 1: Detect cycle
        do {
            slow = nums[slow];
            fast = nums[nums[fast]];
        } while (slow != fast);

        // Phase 2: Find entry point of cycle
        slow = nums[0];
        while (slow != fast) {
            slow = nums[slow];
            fast = nums[fast];
        }
        return slow;
    }

    public static void main(String[] args) {
        int[] nums = {1, 3, 4, 2, 2};
        System.out.println("Duplicate: " + findDuplicate(nums)); // 2
    }
}
```

DRY RUN WALKTHROUGH:

nums = [1, 3, 4, 2, 2]
- Phase 1: slow: 0->1->3->2->4->2; fast: 0->3->4->4->4. They meet at node 2.
- Phase 2: slow=1, fast=2 -> slow=3, fast=4 -> slow=2, fast=2 (meet at 2). Duplicate = 2.

TIME COMPLEXITY:
O(N) linear time.

SPACE COMPLEXITY:
O(1) strictly constant extra space without altering array.

EDGE CASES: Duplicate appearing multiple times (e.g. [2, 2, 2, 2]); duplicate at beginning or end.

#3: Find the Missing Number

Arrays

PROBLEM STATEMENT:

Find the single missing number from 0 to n.

SAMPLE INPUT & OUTPUT:

```
Input: [0, 1, 3]
Output: 2
```

ALGORITHM & INTUITION:

XOR index with elements.

COMPLETE JAVA 21 SOLUTION:

```java
public class DeloitteMissing {
    public static int findMissing(int[] a) {
        int x = 0;
        for (int i = 0; i <= a.length; i++) x ^= i;
        for (int v : a) x ^= v;
        return x;
    }
    public static void main(String[] args) {
        System.out.println(findMissing(new int[]{0, 1, 3})); // 2
    }
}
```

DRY RUN WALKTHROUGH:

(0^1^2^3) ^ (0^1^3) = 2.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Missing 0.

#4: Maximum Subarray Sum

Kadane's algorithm

PROBLEM STATEMENT:

Calculate maximum contiguous subarray sum.

SAMPLE INPUT & OUTPUT:

```
Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6
```

ALGORITHM & INTUITION:

Kadane: curr = max(num, curr + num), max = max(max, curr).

COMPLETE JAVA 21 SOLUTION:

```java
public class DeloitteKadane {
    public static int maxSubArray(int[] nums) {
        int c = nums[0], m = nums[0];
        for (int i = 1; i < nums.length; i++) {
            c = Math.max(nums[i], c + nums[i]);
            m = Math.max(m, c);
        }
        return m;
    }
    public static void main(String[] args) {
        System.out.println(maxSubArray(new int[]{-2, 1, -3, 4, -1, 2, 1, -5, 4})); // 6
    }
}
```

DRY RUN WALKTHROUGH:

Subarray [4, -1, 2, 1] sum = 6.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Negative values.

#5: Merge Overlapping Intervals

Intervals

PROBLEM STATEMENT:

Merge all overlapping intervals into disjoint intervals.

SAMPLE INPUT & OUTPUT:

```
Input: [[1, 4], [4, 5]]
Output: [[1, 5]]
```

ALGORITHM & INTUITION:

Sort by start time and merge if curr[0] <= prev[1].

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class DeloitteIntervals {
    public static int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
        List<int[]> res = new ArrayList<>();
        int[] curr = intervals[0];
        res.add(curr);
        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] <= curr[1]) curr[1] = Math.max(curr[1], intervals[i][1]);
            else { curr = intervals[i]; res.add(curr); }
        }
        return res.toArray(new int[res.size()][]);
    }
    public static void main(String[] args) {
        int[][] res = merge(new int[][]{{1, 4}, {4, 5}});
        for (int[] iv : res) System.out.print(Arrays.toString(iv) + " "); // [1, 5]
    }
}
```

DRY RUN WALKTHROUGH:

Intervals [1,4] and [4,5] touch at 4 -> merge to [1,5].

TIME COMPLEXITY:
O(N log N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Touching endpoints.

#6: Find the Kth-Largest Element in an Array

Heap / PriorityQueue

PROBLEM STATEMENT:

Given an integer array nums and an integer k, return the kth largest element in the array.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [3, 2, 1, 5, 6, 4], k = 2
Output: 5
```

ALGORITHM & INTUITION:

Min-Heap of size K:
Maintain a PriorityQueue<Integer> of size k. For each element in nums, add it to the min-heap. If the heap size exceeds k, poll the smallest element. After processing all elements, the top of the min-heap is the kth largest element.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.PriorityQueue;

public class KthLargestElement {
    public static int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> minHeap = new PriorityQueue<>(k);
        for (int num : nums) {
            minHeap.add(num);
            if (minHeap.size() > k) {
                minHeap.poll();
            }
        }
        return minHeap.peek();
    }

    public static void main(String[] args) {
        int[] nums = {3, 2, 1, 5, 6, 4};
        System.out.println("2nd Largest: " + findKthLargest(nums, 2)); // 5
    }
}
```

DRY RUN WALKTHROUGH:

nums = [3, 2, 1, 5, 6, 4], k = 2
- add 3, 2 -> heap=[2, 3]
- add 1 -> heap=[1, 2, 3] -> poll 1 -> [2, 3]
- add 5 -> heap=[2, 3, 5] -> poll 2 -> [3, 5]
- add 6 -> heap=[3, 5, 6] -> poll 3 -> [5, 6]
- add 4 -> heap=[4, 5, 6] -> poll 4 -> [5, 6]
Top is 5.

TIME COMPLEXITY:
O(N log K) — much faster than O(N log N) sorting when K << N.

SPACE COMPLEXITY:
O(K) for min-heap.

EDGE CASES: k = 1 (largest element); k = n (smallest element); array with duplicates.

#7: Longest Substring Without Repeating Characters

Sliding Window

PROBLEM STATEMENT:

Find length of longest substring without duplicates.

SAMPLE INPUT & OUTPUT:

```
Input: "bbbbb"
Output: 1
```

ALGORITHM & INTUITION:

Sliding window with last seen character index table.

COMPLETE JAVA 21 SOLUTION:

```java
public class DeloitteLongestUnique {
    public static int lengthOfLongestSubstring(String s) {
        int[] last = new int[256];
        java.util.Arrays.fill(last, -1);
        int max = 0, l = 0;
        for (int r = 0; r < s.length(); r++) {
            char c = s.charAt(r);
            if (last[c] >= l) l = last[c] + 1;
            last[c] = r;
            max = Math.max(max, r - l + 1);
        }
        return max;
    }
    public static void main(String[] args) {
        System.out.println(lengthOfLongestSubstring("bbbbb")); // 1
    }
}
```

DRY RUN WALKTHROUGH:

At each 'b', window resets to length 1.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Single character.

#8: Group Anagrams

Hashing

PROBLEM STATEMENT:

Given an array of strings strs, group the anagrams together. You can return the answer in any order.

SAMPLE INPUT & OUTPUT:

```
Input: strs = ["eat","tea","tan","ate","nat","bat"]
Output: [["bat"],["nat","tan"],["ate","eat","tea"]]
```

ALGORITHM & INTUITION:

Use a HashMap<String, List<String>>. For each word, convert to char array, sort the characters, and convert back to string as the canonical key (e.g. "eat", "tea", "ate" all sort to "aet"). Group all words with identical keys.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class GroupAnagrams {
    public static List<List<String>> groupAnagrams(String[] strs) {
        if (strs == null || strs.length == 0) return new ArrayList<>();
        Map<String, List<String>> map = new HashMap<>();

        for (String s : strs) {
            char[] ca = s.toCharArray();
            Arrays.sort(ca);
            String key = String.valueOf(ca);
            if (!map.containsKey(key)) {
                map.put(key, new ArrayList<>());
            }
            map.get(key).add(s);
        }
        return new ArrayList<>(map.values());
    }

    public static void main(String[] args) {
        String[] strs = {"eat", "tea", "tan", "ate", "nat", "bat"};
        System.out.println(groupAnagrams(strs));
        // [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]
    }
}
```

DRY RUN WALKTHROUGH:

"eat"->"aet", "tea"->"aet", "tan"->"ant", "ate"->"aet", "nat"->"ant", "bat"->"abt". Groups formed correctly.

TIME COMPLEXITY:
O(N * K log K) where N is number of strings and K is maximum string length.

SPACE COMPLEXITY:
O(N * K) to store strings in map.

EDGE CASES: Empty strings [""]; single character strings.

#9: Valid Parentheses

Stack

PROBLEM STATEMENT:

Determine whether the brackets string is valid.

SAMPLE INPUT & OUTPUT:

```
Input: "()"
Output: true
```

ALGORITHM & INTUITION:

Stack push closing counterparts.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Stack;

public class DeloitteValidParens {
    public static boolean isValid(String s) {
        Stack<Character> st = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(') st.push(')');
            else if (c == '{') st.push('}');
            else if (c == '[') st.push(']');
            else if (st.isEmpty() || st.pop() != c) return false;
        }
        return st.isEmpty();
    }
    public static void main(String[] args) {
        System.out.println(isValid("()")); // true
    }
}
```

DRY RUN WALKTHROUGH:

() -> true.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Unclosed bracket.

#10: Next Greater Element

Monotonic Stack

PROBLEM STATEMENT:

Find first greater element to right of each element.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 3, 2, 4]
Output: [3, 4, 4, -1]
```

ALGORITHM & INTUITION:

Monotonic stack backwards pass.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class DeloitteNGE {
    public static int[] nextGreater(int[] nums) {
        int n = nums.length;
        int[] res = new int[n];
        Stack<Integer> st = new Stack<>();
        for (int i = n - 1; i >= 0; i--) {
            while (!st.isEmpty() && st.peek() <= nums[i]) st.pop();
            res[i] = st.isEmpty() ? -1 : st.peek();
            st.push(nums[i]);
        }
        return res;
    }
    public static void main(String[] args) {
        System.out.println(Arrays.toString(nextGreater(new int[]{1, 3, 2, 4}))); // [3, 4, 4, -1]
    }
}
```

DRY RUN WALKTHROUGH:

[1, 3, 2, 4] -> [3, 4, 4, -1].

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Decreasing array.

#11: Reverse a Linked List

Linked List

PROBLEM STATEMENT:

Reverse a singly linked list in-place.

SAMPLE INPUT & OUTPUT:

```
Input: 1 -> 2 -> 3 -> null
Output: 3 -> 2 -> 1 -> null
```

ALGORITHM & INTUITION:

Iterative pointer reversal (prev, curr, next).

COMPLETE JAVA 21 SOLUTION:

```
public class DeloitteReverseLL {
    static class Node {
        int val; Node next;
        Node(int val) { this.val = val; }
    }
    public static Node reverse(Node head) {
        Node prev = null, curr = head;
        while (curr != null) {
            Node next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }
        return prev;
    }
    public static void main(String[] args) {
        Node head = new Node(1); head.next = new Node(2); head.next.next = new Node(3);
        Node rev = reverse(head);
        while (rev != null) { System.out.print(rev.val + " "); rev = rev.next; } // 3 2 1
    }
}
```

DRY RUN WALKTHROUGH:

Pointers reversed one by one.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: head == null.

#12: Detect a Cycle in a Linked List

Linked List / Two pointers

PROBLEM STATEMENT:

Given head, the head of a linked list, determine if the linked list has a cycle in it.

SAMPLE INPUT & OUTPUT:

```
Input: head = [3, 2, 0, -4], pos = 1 (tail connects to node 2)
Output: true
```

ALGORITHM & INTUITION:

Floyd's Cycle Finding (Tortoise and Hare): slow pointer advances 1 step, fast pointer advances 2 steps. If there is a cycle, fast will eventually lap slow and slow == fast.

COMPLETE JAVA 21 SOLUTION:

```
public class LinkedListCycle {
    static class ListNode {
        int val;
        ListNode next;
        ListNode(int x) { val = x; next = null; }
    }

    public static boolean hasCycle(ListNode head) {
        if (head == null || head.next == null) return false;
        ListNode slow = head;
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }

    public static void main(String[] args) {
        ListNode head = new ListNode(3);
        ListNode second = new ListNode(2);
        ListNode third = new ListNode(0);
        ListNode fourth = new ListNode(-4);
        head.next = second; second.next = third; third.next = fourth;
        fourth.next = second; // cycle

        System.out.println("Has Cycle? " + hasCycle(head)); // true
    }
}
```

DRY RUN WALKTHROUGH:

Fast catches slow inside the loop -> returns true.

TIME COMPLEXITY:
O(N) linear time.

SPACE COMPLEXITY:
O(1) extra space.

EDGE CASES: Empty list; single node with self-loop; list without cycle.

#13: Find the Middle of a Linked List

Linked List

PROBLEM STATEMENT:

Given the head of a singly linked list, return the middle node of the linked list. If there are two middle nodes, return the second middle node.

SAMPLE INPUT & OUTPUT:

```
Input: head = [1, 2, 3, 4, 5]
Output: 3
Input: head = [1, 2, 3, 4, 5, 6]
Output: 4
```

ALGORITHM & INTUITION:

Fast & slow pointers: when fast reaches the end (fast == null || fast.next == null), slow is precisely at the middle node.

COMPLETE JAVA 21 SOLUTION:

```java
public class MiddleOfLinkedList {
    static class ListNode {
        int val;
        ListNode next;
        ListNode(int val) { this.val = val; }
    }

    public static ListNode middleNode(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }

    public static void main(String[] args) {
        ListNode head = new ListNode(1);
        head.next = new ListNode(2);
        head.next.next = new ListNode(3);
        head.next.next.next = new ListNode(4);
        head.next.next.next.next = new ListNode(5);

        System.out.println("Middle: " + middleNode(head).val); // 3
    }
}
```

DRY RUN WALKTHROUGH:

fast advances by 2, slow by 1. At fast=5, slow=3.

TIME COMPLEXITY:
O(N) — single pass.

SPACE COMPLEXITY:
O(1) extra space.

EDGE CASES: Even number of nodes (returns second middle); 1 node.

#14: Implement a Queue Using Stacks

Queue / Stack

PROBLEM STATEMENT:

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support push, pop, peek, and empty.

SAMPLE INPUT & OUTPUT:

```
MyQueue q = new MyQueue(); q.push(1); q.push(2); q.peek(); // 1; q.pop(); // 1; q.empty(); // false
```

ALGORITHM & INTUITION:

Two Stacks (inbox and outbox):
- push(x): push onto inbox.
- pop()/peek(): if outbox is empty, pop all elements from inbox and push onto outbox (reversing their order to FIFO). Then pop/peek from outbox.
- Amortized complexity is O(1) because each element moves into outbox exactly once.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.Stack;

public class QueueUsingStacks {
    static class MyQueue {
        private Stack<Integer> in = new Stack<>();
        private Stack<Integer> out = new Stack<>();

        public void push(int x) { in.push(x); }

        public int pop() {
            peek();
            return out.pop();
        }

        public int peek() {
            if (out.isEmpty()) {
                while (!in.isEmpty()) {
                    out.push(in.pop());
                }
            }
            return out.peek();
        }

        public boolean empty() {
            return in.isEmpty() && out.isEmpty();
        }
    }

    public static void main(String[] args) {
        MyQueue q = new MyQueue();
        q.push(1); q.push(2);
        System.out.println("Peek: " + q.peek()); // 1
        System.out.println("Pop: " + q.pop());   // 1
        System.out.println("Empty? " + q.empty()); // false
    }
}
```

DRY RUN WALKTHROUGH:

Elements transferred in reverse: FIFO behavior verified.

TIME COMPLEXITY:
Amortized O(1) for pop/peek; O(1) push.

SPACE COMPLEXITY:
O(N) to store elements.

EDGE CASES: Alternating push and pop operations.

#15: Binary Tree Traversals (Inorder, Preorder, Postorder)

Binary Trees

PROBLEM STATEMENT:

Given the root of a binary tree, implement Inorder, Preorder, and Postorder depth-first traversals.

SAMPLE INPUT & OUTPUT:

```
Tree: 1 -> right: 2 -> left: 3
Inorder (Left, Root, Right): [1, 3, 2]
Preorder (Root, Left, Right): [1, 2, 3]
Postorder (Left, Right, Root): [3, 2, 1]
```

ALGORITHM & INTUITION:

Recursive DFS definitions:
- Inorder: recurse(node.left), visit(node), recurse(node.right)
- Preorder: visit(node), recurse(node.left), recurse(node.right)
- Postorder: recurse(node.left), recurse(node.right), visit(node)

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class BinaryTreeTraversals {
    static class TreeNode {
        int val;
        TreeNode left, right;
        TreeNode(int val) { this.val = val; }
    }

    public static void inorder(TreeNode root, List<Integer> res) {
        if (root == null) return;
        inorder(root.left, res);
        res.add(root.val);
        inorder(root.right, res);
    }

    public static void preorder(TreeNode root, List<Integer> res) {
        if (root == null) return;
        res.add(root.val);
        preorder(root.left, res);
        preorder(root.right, res);
    }

    public static void postorder(TreeNode root, List<Integer> res) {
        if (root == null) return;
        postorder(root.left, res);
        postorder(root.right, res);
        res.add(root.val);
    }

    public static void main(String[] args) {
        TreeNode root = new TreeNode(1);
        root.right = new TreeNode(2);
        root.right.left = new TreeNode(3);

        List<Integer> in = new ArrayList<>(), pre = new ArrayList<>(), post = new ArrayList<>();
        inorder(root, in); preorder(root, pre); postorder(root, post);

        System.out.println("Inorder: " + in);     // [1, 3, 2]
        System.out.println("Preorder: " + pre);   // [1, 2, 3]
        System.out.println("Postorder: " + post); // [3, 2, 1]
    }
}
```

DRY RUN WALKTHROUGH:

Standard depth first recursive traversal.

TIME COMPLEXITY:
O(N) — visits each of the N nodes once.

SPACE COMPLEXITY:
O(H) recursion stack where H is tree height.

EDGE CASES: root == null; skewed tree (H = N).

#16: Find the Height of a Binary Tree

Binary Trees

PROBLEM STATEMENT:

Given the root of a binary tree, find its maximum depth / height (number of nodes along the longest path from root to leaf).

SAMPLE INPUT & OUTPUT:

```
Input: root = [3, 9, 20, null, null, 15, 7]
Output: 3
```

ALGORITHM & INTUITION:

Recursive Divide and Conquer: height(root) = 1 + Math.max(height(root.left), height(root.right)). Base case: null root has height 0.

COMPLETE JAVA 21 SOLUTION:

```java
public class HeightOfBinaryTree {
    static class TreeNode {
        int val;
        TreeNode left, right;
        TreeNode(int val) { this.val = val; }
    }

    public static int maxDepth(TreeNode root) {
        if (root == null) return 0;
        int leftHeight = maxDepth(root.left);
        int rightHeight = maxDepth(root.right);
        return 1 + Math.max(leftHeight, rightHeight);
    }

    public static void main(String[] args) {
        TreeNode root = new TreeNode(3);
        root.left = new TreeNode(9);
        root.right = new TreeNode(20);
        root.right.left = new TreeNode(15);
        root.right.right = new TreeNode(7);

        System.out.println("Height: " + maxDepth(root)); // 3
    }
}
```

DRY RUN WALKTHROUGH:

Left depth=1 (node 9), right depth=2 (node 20->15,7). Total = 1 + max(1, 2) = 3.

TIME COMPLEXITY:
O(N) visits every node.

SPACE COMPLEXITY:
O(H) call stack.

EDGE CASES: root == null (0); single node (1).

#17: Find the Lowest Common Ancestor (LCA)

Binary Trees

PROBLEM STATEMENT:

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes p and q. The LCA is defined between two nodes p and q as the lowest node in T that has both p and q as descendants.

SAMPLE INPUT & OUTPUT:

```
Input: root = [3, 5, 1, 6, 2, 0, 8], p = 5, q = 1
Output: 3
Explanation: LCA of 5 and 1 is 3.
```

ALGORITHM & INTUITION:

Recursive LCA:
1. If root is null, or root == p, or root == q, return root.
2. Recurse left and right: left = lca(root.left), right = lca(root.right).
3. If both left and right are non-null, p and q lie in different subtrees of root, so root is the LCA!
4. If only one is non-null, return that non-null child.

COMPLETE JAVA 21 SOLUTION:

```java
public class LowestCommonAncestor {
    static class TreeNode {
        int val;
        TreeNode left, right;
        TreeNode(int val) { this.val = val; }
    }

    public static TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null || root == p || root == q) return root;

        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);

        if (left != null && right != null) return root;
        return left != null ? left : right;
    }

    public static void main(String[] args) {
        TreeNode root = new TreeNode(3);
        TreeNode p = new TreeNode(5);
        TreeNode q = new TreeNode(1);
        root.left = p; root.right = q;

        TreeNode lca = lowestCommonAncestor(root, p, q);
        System.out.println("LCA: " + lca.val); // 3
    }
}
```

DRY RUN WALKTHROUGH:

Left returns 5, right returns 1 -> both non-null -> root (3) is LCA.

TIME COMPLEXITY:
O(N) — each node visited at most once.

SPACE COMPLEXITY:
O(H) for recursion stack.

EDGE CASES: p is an ancestor of q (returns p); q is ancestor of p.

#18: BFS and DFS

Graphs

PROBLEM STATEMENT:

Implement Breadth First Search and Depth First Search on graph.

SAMPLE INPUT & OUTPUT:

```
BFS: level-order traversal using Queue.
DFS: depth-first branch traversal using recursion.
```

ALGORITHM & INTUITION:

BFS uses Queue + visited boolean array. DFS uses recursion + visited boolean array.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class DeloitteGraphSearch {
    public static void bfs(int start, List<List<Integer>> adj, int V) {
        boolean[] vis = new boolean[V];
        Queue<Integer> q = new LinkedList<>();
        vis[start] = true; q.add(start);
        System.out.print("BFS: ");
        while (!q.isEmpty()) {
            int u = q.poll();
            System.out.print(u + " ");
            for (int v : adj.get(u)) {
                if (!vis[v]) { vis[v] = true; q.add(v); }
            }
        }
        System.out.println();
    }
    public static void main(String[] args) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < 4; i++) adj.add(new ArrayList<>());
        adj.get(0).add(1); adj.get(0).add(2);
        adj.get(1).add(2); adj.get(2).add(3);
        bfs(0, adj, 4); // BFS: 0 1 2 3
    }
}
```

DRY RUN WALKTHROUGH:

0 -> 1, 2 -> 3.

TIME COMPLEXITY:
O(V + E).

SPACE COMPLEXITY:
O(V).

EDGE CASES: Disconnected graph.

#19: Number of Islands

Graphs / BFS & DFS

PROBLEM STATEMENT:

Count connected components of '1's in 2D binary grid.

SAMPLE INPUT & OUTPUT:

```
Input: 2x2 grid with [[1, 0], [0, 1]]
Output: 2
```

ALGORITHM & INTUITION:

Sink connected land cells recursively with DFS.

COMPLETE JAVA 21 SOLUTION:

```java
public class DeloitteIslands {
    public static int numIslands(char[][] grid) {
        int count = 0;
        for (int r = 0; r < grid.length; r++) {
            for (int c = 0; c < grid[0].length; c++) {
                if (grid[r][c] == '1') {
                    count++;
                    sink(grid, r, c);
                }
            }
        }
        return count;
    }
    private static void sink(char[][] g, int r, int c) {
        if (r < 0 || r >= g.length || c < 0 || c >= g[0].length || g[r][c] != '1') return;
        g[r][c] = '0';
        sink(g, r+1, c); sink(g, r-1, c); sink(g, r, c+1); sink(g, r, c-1);
    }
    public static void main(String[] args) {
        char[][] g = {{'1', '0'}, {'0', '1'}};
        System.out.println("Islands: " + numIslands(g)); // 2
    }
}
```

DRY RUN WALKTHROUGH:

Diagonal cells are not connected horizontally/vertically -> 2 islands.

TIME COMPLEXITY:
O(M * N).

SPACE COMPLEXITY:
O(M * N).

EDGE CASES: All water.

#20: Climbing Stairs

Dynamic Programming

PROBLEM STATEMENT:

You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

SAMPLE INPUT & OUTPUT:

```
Input: n = 3
Output: 3
Explanation: 1+1+1, 1+2, 2+1
```

ALGORITHM & INTUITION:

Fibonacci recurrence: To reach step n, you must arrive from step n-1 (1 step jump) or step n-2 (2 step jump). Thus, ways(n) = ways(n-1) + ways(n-2). Solved in O(N) time and O(1) space with two variables.

COMPLETE JAVA 21 SOLUTION:

```
public class ClimbingStairs {
    public static int climbStairs(int n) {
        if (n <= 2) return n;
        int prev2 = 1, prev1 = 2;

        for (int i = 3; i <= n; i++) {
            int curr = prev1 + prev2;
            prev2 = prev1;
            prev1 = curr;
        }
        return prev1;
    }

    public static void main(String[] args) {
        System.out.println("Ways for n=3: " + climbStairs(3)); // 3
        System.out.println("Ways for n=5: " + climbStairs(5)); // 8
    }
}
```

DRY RUN WALKTHROUGH:

n=1: 1; n=2: 2; n=3: 1+2=3; n=4: 2+3=5; n=5: 3+5=8.

TIME COMPLEXITY:
O(N) linear loop.

SPACE COMPLEXITY:
O(1) constant auxiliary space.

EDGE CASES: n = 1 (1); n = 2 (2).

CHAPTER 8

HCLTech — Graduate / Entry-Level Engineer

Core Data Structures, Sorting, Searching & Number Theory

Target Focus: Graduate Trainee & Associate Software Developer

HCLTech emphasizes classic foundational DSA: Min/Max elements, array verification, insertion sort, binary search, prime numbers, and palindrome checks.

Questions in this Section (20 Problems):

#1: Find Largest and Smallest Elements in an Array — Arrays

PROBLEM STATEMENT:

Given an array of integers, find the maximum and minimum elements in a single traversal.

SAMPLE INPUT & OUTPUT:

```
Input: [3, 5, 4, 1, 9]
Output: Min = 1, Max = 9
```

ALGORITHM & INTUITION:

Initialize min and max with arr[0]. Traverse from index 1 to n-1: if arr[i] < min, update min; if arr[i] > max, update max.

COMPLETE JAVA 21 SOLUTION:

```java
public class HclMinMax {
    public static void findMinMax(int[] arr) {
        int min = arr[0], max = arr[0];
        for (int x : arr) {
            if (x < min) min = x;
            if (x > max) max = x;
        }
        System.out.println("Min: " + min + ", Max: " + max);
    }
    public static void main(String[] args) {
        findMinMax(new int[]{3, 5, 4, 1, 9}); // Min: 1, Max: 9
    }
}
```

DRY RUN WALKTHROUGH:

Single pass updates min to 1 and max to 9.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Single element; all elements equal.

#2: Find the Second-Largest Element

Arrays

PROBLEM STATEMENT:

Find second-largest element in array without sorting.

SAMPLE INPUT & OUTPUT:

```
Input: [10, 20, 4, 45, 99]
Output: 45
```

ALGORITHM & INTUITION:

Track max1 and max2 in single pass.

COMPLETE JAVA 21 SOLUTION:

```java
public class HclSecondLargest {
    public static int secondLargest(int[] arr) {
        int m1 = Integer.MIN_VALUE, m2 = Integer.MIN_VALUE;
        for (int x : arr) {
            if (x > m1) { m2 = m1; m1 = x; }
            else if (x < m1 && x > m2) { m2 = x; }
        }
        return m2 == Integer.MIN_VALUE ? -1 : m2;
    }
    public static void main(String[] args) {
        System.out.println(secondLargest(new int[]{10, 20, 4, 45, 99})); // 45
    }
}
```

DRY RUN WALKTHROUGH:

Largest 99, second 45.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Duplicates.

#3: Reverse an Array

Arrays

PROBLEM STATEMENT:

Reverse an array in-place.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 3, 4, 5]
Output: [5, 4, 3, 2, 1]
```

ALGORITHM & INTUITION:

Two-pointer swap from l=0 and r=n-1 inwards.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class HclReverseArray {
    public static void reverse(int[] arr) {
        int l = 0, r = arr.length - 1;
        while (l < r) {
            int t = arr[l]; arr[l++] = arr[r]; arr[r--] = t;
        }
    }
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        reverse(arr);
        System.out.println(Arrays.toString(arr)); // [5, 4, 3, 2, 1]
    }
}
```

DRY RUN WALKTHROUGH:

Elements swapped pairwise.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Empty array; 1 element.

#4: Find a Missing Number

Arrays

PROBLEM STATEMENT:

Find missing integer in range 0 to n.

SAMPLE INPUT & OUTPUT:

```
Input: [3, 0, 1]
Output: 2
```

ALGORITHM & INTUITION:

Bitwise XOR method.

COMPLETE JAVA 21 SOLUTION:

```java
public class HclMissingNumber {
    public static int findMissing(int[] a) {
        int x = 0;
        for (int i = 0; i <= a.length; i++) x ^= i;
        for (int num : a) x ^= num;
        return x;
    }
    public static void main(String[] args) {
        System.out.println(findMissing(new int[]{3, 0, 1})); // 2
    }
}
```

DRY RUN WALKTHROUGH:

XOR cancellations leave 2.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Missing 0.

#5: Move Zeros to the End

Arrays

PROBLEM STATEMENT:

Shift zeros to end preserving non-zero order.

SAMPLE INPUT & OUTPUT:

```
Input: [0, 1, 0, 3, 12]
Output: [1, 3, 12, 0, 0]
```

ALGORITHM & INTUITION:

Two-pointer in-place write.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class HclMoveZeros {
    public static void move(int[] a) {
        int idx = 0;
        for (int x : a) if (x != 0) a[idx++] = x;
        while (idx < a.length) a[idx++] = 0;
    }
    public static void main(String[] args) {
        int[] a = {0, 1, 0, 3, 12};
        move(a);
        System.out.println(Arrays.toString(a)); // [1, 3, 12, 0, 0]
    }
}
```

DRY RUN WALKTHROUGH:

[0, 1, 0, 3, 12] -> [1, 3, 12, 0, 0].

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: No zeros.

#6: Count Array Frequencies

Hashing

PROBLEM STATEMENT:

Count frequency of each element in array.

SAMPLE INPUT & OUTPUT:

```
Input: [10, 20, 20, 10, 10, 20, 5, 20]
Output: 10:3, 20:4, 5:1
```

ALGORITHM & INTUITION:

HashMap counting.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.*;

public class HclArrayFreq {
    public static void printFreq(int[] a) {
        Map<Integer, Integer> map = new LinkedHashMap<>();
        for (int x : a) map.put(x, map.getOrDefault(x, 0) + 1);
        for (Map.Entry<Integer, Integer> e : map.entrySet()) {
            System.out.print(e.getKey() + ":" + e.getValue() + " ");
        }
        System.out.println();
    }
    public static void main(String[] args) {
        printFreq(new int[]{10, 20, 20, 10, 10, 20, 5, 20});
    }
}
```

DRY RUN WALKTHROUGH:

10:3 20:4 5:1.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: All distinct.

#7: Rotate an Array

Arrays

PROBLEM STATEMENT:

Rotate array to right by k positions.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 3, 4, 5], k = 2
Output: [4, 5, 1, 2, 3]
```

ALGORITHM & INTUITION:

Reverse all, reverse 0..k-1, reverse k..n-1.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.Arrays;

public class HclRotate {
    public static void rotate(int[] a, int k) {
        k %= a.length;
        rev(a, 0, a.length - 1); rev(a, 0, k - 1); rev(a, k, a.length - 1);
    }
    private static void rev(int[] a, int l, int r) {
        while (l < r) { int t = a[l]; a[l++] = a[r]; a[r--] = t; }
    }
    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4, 5};
        rotate(a, 2);
        System.out.println(Arrays.toString(a)); // [4, 5, 1, 2, 3]
    }
}
```

DRY RUN WALKTHROUGH:

Three reversals achieve rotation.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: k = 0.

#8: Find Duplicates in an Array

Hashing

PROBLEM STATEMENT:

Find all numbers appearing more than once.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 3, 2, 4, 5, 1]
Output: [1, 2]
```

ALGORITHM & INTUITION:

HashSet to track seen elements and identify duplicates.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class HclDuplicates {
    public static List<Integer> findDups(int[] a) {
        Set<Integer> seen = new HashSet<>();
        Set<Integer> dups = new LinkedHashSet<>();
        for (int x : a) {
            if (!seen.add(x)) dups.add(x);
        }
        return new ArrayList<>(dups);
    }
    public static void main(String[] args) {
        System.out.println(findDups(new int[]{1, 2, 3, 2, 4, 5, 1})); // [2, 1]
    }
}
```

DRY RUN WALKTHROUGH:

2 and 1 repeat.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: No duplicates.

#9: Check Whether an Array is Sorted

Arrays

PROBLEM STATEMENT:

Given an array nums, return true if the array was originally sorted in non-decreasing order.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 3, 4, 5]
Output: true
Input: [1, 3, 2, 4]
Output: false
```

ALGORITHM & INTUITION:

Linear scan from index 0 to n-2: if arr[i] > arr[i+1], return false. If loop finishes, return true.

COMPLETE JAVA 21 SOLUTION:

```
public class CheckSortedArray {
    public static boolean isSorted(int[] arr) {
        for (int i = 0; i < arr.length - 1; i++) {
            if (arr[i] > arr[i + 1]) return false;
        }
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isSorted(new int[]{1, 2, 3, 4, 5})); // true
        System.out.println(isSorted(new int[]{1, 3, 2, 4}));    // false
    }
}
```

DRY RUN WALKTHROUGH:

Compares adjacent elements in O(N).

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Empty array; 1 element (true); array with all identical elements (true).

#10: Check Prime Number

Mathematics

PROBLEM STATEMENT:

Determine if n is prime.

SAMPLE INPUT & OUTPUT:

```
Input: 23
Output: true
```

ALGORITHM & INTUITION:

Test divisors up to sqrt(n).

COMPLETE JAVA 21 SOLUTION:

```
public class HclPrime {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i * i <= n; i++) if (n % i == 0) return false;
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isPrime(23)); // true
    }
}
```

DRY RUN WALKTHROUGH:

No divisor <= sqrt(23) -> prime.

TIME COMPLEXITY:
O(sqrt(N)).

SPACE COMPLEXITY:
O(1).

EDGE CASES: n = 1.

#11: Armstrong Number

Mathematics

PROBLEM STATEMENT:

Check if n equals sum of its digits raised to power of number of digits.

SAMPLE INPUT & OUTPUT:

```
Input: 153
Output: true
```

ALGORITHM & INTUITION:

Count digits k, compute sum of digit^k.

COMPLETE JAVA 21 SOLUTION:

```java
public class HclArmstrong {
    public static boolean isArmstrong(int n) {
        int orig = n, k = String.valueOf(n).length();
        long sum = 0, temp = n;
        while (temp > 0) {
            sum += Math.round(Math.pow(temp % 10, k));
            temp /= 10;
        }
        return sum == orig;
    }
    public static void main(String[] args) {
        System.out.println(isArmstrong(153)); // true
    }
}
```

DRY RUN WALKTHROUGH:

1^3 + 5^3 + 3^3 = 153.

TIME COMPLEXITY:
O(log10 N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Single digits 0-9.

#12: Fibonacci Series

Mathematics

PROBLEM STATEMENT:

Print first n terms of Fibonacci sequence.

SAMPLE INPUT & OUTPUT:

```
Input: 5
Output: 0 1 1 2 3
```

ALGORITHM & INTUITION:

Iterative two variables a and b.

COMPLETE JAVA 21 SOLUTION:

```
public class HclFibonacci {
    public static void printFib(int n) {
        int a = 0, b = 1;
        for (int i = 0; i < n; i++) {
            System.out.print(a + " ");
            int c = a + b; a = b; b = c;
        }
        System.out.println();
    }
    public static void main(String[] args) {
        printFib(5); // 0 1 1 2 3
    }
}
```

DRY RUN WALKTHROUGH:

0 1 1 2 3.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: n = 1.

#13: Reverse a String

Strings

PROBLEM STATEMENT:

Reverse characters in string.

SAMPLE INPUT & OUTPUT:

```
Input: "HCLTech"
Output: "hceTLCH"
```

ALGORITHM & INTUITION:

Two pointers on char array.

COMPLETE JAVA 21 SOLUTION:

```
public class HclReverseString {
    public static String reverse(String s) {
        char[] ca = s.toCharArray();
        int l = 0, r = ca.length - 1;
        while (l < r) {
            char t = ca[l]; ca[l++] = ca[r]; ca[r--] = t;
        }
        return new String(ca);
    }
    public static void main(String[] args) {
        System.out.println(reverse("HCLTech")); // hceTLCH
    }
}
```

DRY RUN WALKTHROUGH:

Reversed.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Single char.

#14: Check Palindrome

Strings

PROBLEM STATEMENT:

Check if string is palindrome.

SAMPLE INPUT & OUTPUT:

```
Input: "noon"
Output: true
```

ALGORITHM & INTUITION:

Two pointers inwards.

COMPLETE JAVA 21 SOLUTION:

```java
public class HclPalindrome {
    public static boolean isPal(String s) {
        int l = 0, r = s.length() - 1;
        while (l < r) if (s.charAt(l++) != s.charAt(r--)) return false;
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isPal("noon")); // true
    }
}
```

DRY RUN WALKTHROUGH:

noon matches backwards.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Single character.

#15: Check Anagrams

Strings

PROBLEM STATEMENT:

Check if two strings are anagrams.

SAMPLE INPUT & OUTPUT:

```
Input: "listen", "silent"
Output: true
```

ALGORITHM & INTUITION:

Count array 26 characters.

COMPLETE JAVA 21 SOLUTION:

```java
public class HclAnagram {
    public static boolean isAnagram(String a, String b) {
        if (a.length() != b.length()) return false;
        int[] f = new int[26];
        for (int i = 0; i < a.length(); i++) {
            f[a.charAt(i) - 'a']++; f[b.charAt(i) - 'a']--;
        }
        for (int x : f) if (x != 0) return false;
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isAnagram("listen", "silent")); // true
    }
}
```

DRY RUN WALKTHROUGH:

Frequencies equal.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Length mismatch.

#16: Count Vowels and Consonants

Strings

PROBLEM STATEMENT:

Count vowels and consonants in string.

SAMPLE INPUT & OUTPUT:

```
Input: "Java"
Output: Vowels = 2, Consonants = 2
```

ALGORITHM & INTUITION:

Character scan checking against "aeiou".

COMPLETE JAVA 21 SOLUTION:

```java
public class HclVowelsConsonants {
    public static void count(String s) {
        int v = 0, c = 0;
        for (char ch : s.toLowerCase().toCharArray()) {
            if (ch >= 'a' && ch <= 'z') {
                if ("aeiou".indexOf(ch) != -1) v++; else c++;
            }
        }
        System.out.println("Vowels: " + v + ", Consonants: " + c);
    }
    public static void main(String[] args) {
        count("Java"); // Vowels: 2, Consonants: 2
    }
}
```

DRY RUN WALKTHROUGH:

j:c, a:v, v:c, a:v.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Special characters.

#17: Binary Search

Searching

PROBLEM STATEMENT:

Perform binary search for key in sorted array.

SAMPLE INPUT & OUTPUT:

```
Input: [10, 20, 30, 40], key = 30
Output: 2
```

ALGORITHM & INTUITION:

Binary search mid calculation.

COMPLETE JAVA 21 SOLUTION:

```java
public class HclBinarySearch {
    public static int search(int[] a, int k) {
        int l = 0, h = a.length - 1;
        while (l <= h) {
            int m = l + (h - l) / 2;
            if (a[m] == k) return m;
            if (a[m] < k) l = m + 1; else h = m - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        System.out.println(search(new int[]{10, 20, 30, 40}, 30)); // 2
    }
}
```

DRY RUN WALKTHROUGH:

Returns 2.

TIME COMPLEXITY:
O(log N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Key absent.

#18: Sort an Array

Sorting

PROBLEM STATEMENT:

Sort array using quick sort or insertion sort.

SAMPLE INPUT & OUTPUT:

```
Input: [5, 2, 8, 1, 9]
Output: [1, 2, 5, 8, 9]
```

ALGORITHM & INTUITION:

Insertion sort: place each element into its correct sorted position in preceding subarray.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class HclInsertionSort {
    public static void sort(int[] arr) {
        for (int i = 1; i < arr.length; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }
    public static void main(String[] args) {
        int[] a = {5, 2, 8, 1, 9};
        sort(a);
        System.out.println(Arrays.toString(a)); // [1, 2, 5, 8, 9]
    }
}
```

DRY RUN WALKTHROUGH:

Elements shifted into position.

TIME COMPLEXITY:
O(N^2) average/worst, O(N) best.

SPACE COMPLEXITY:
O(1) in-place.

EDGE CASES: Already sorted.

#19: Reverse a Linked List

Linked List

PROBLEM STATEMENT:

Reverse singly linked list.

SAMPLE INPUT & OUTPUT:

```
Input: 1 -> 2 -> 3
Output: 3 -> 2 -> 1
```

ALGORITHM & INTUITION:

Iterative pointer reversal.

COMPLETE JAVA 21 SOLUTION:

```java
public class HclReverseLL {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }
    public static Node reverse(Node head) {
        Node prev = null, curr = head;
        while (curr != null) {
            Node next = curr.next; curr.next = prev; prev = curr; curr = next;
        }
        return prev;
    }
    public static void main(String[] args) {
        Node h = new Node(1); h.next = new Node(2); h.next.next = new Node(3);
        Node r = reverse(h);
        while (r != null) { System.out.print(r.val + " "); r = r.next; } // 3 2 1
    }
}
```

DRY RUN WALKTHROUGH:

Reversed list.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Single node.

#20: Balanced Parentheses

Stack

PROBLEM STATEMENT:

Check if string has balanced parentheses.

SAMPLE INPUT & OUTPUT:

```
Input: "[()]"
Output: true
```

ALGORITHM & INTUITION:

Stack bracket matching.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Stack;

public class HclBalancedParens {
    public static boolean isValid(String s) {
        Stack<Character> st = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(') st.push(')');
            else if (c == '{') st.push('}');
            else if (c == '[') st.push(']');
            else if (st.isEmpty() || st.pop() != c) return false;
        }
        return st.isEmpty();
    }
    public static void main(String[] args) {
        System.out.println(isValid("[()]")); // true
    }
}
```

DRY RUN WALKTHROUGH:

Valid brackets.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Unbalanced brackets.

CHAPTER 9

Tech Mahindra — Entry-Level Assessment

Essential Coding Practice & Full Solutions

Target Focus: Associate Software Engineer Technical Exam

Tech Mahindra focuses on fundamental mathematical programming, selection sort, middle of linked list, and custom stack/ queue operations.

Questions in this Section (20 Problems):

#1: Reverse a Number — Mathematics

PROBLEM STATEMENT:

Given an integer, reverse its digits and return the result.

SAMPLE INPUT & OUTPUT:

```
Input: 9876
Output: 6789
```

ALGORITHM & INTUITION:

Modulo 10 digit extraction and multiplication by 10.

COMPLETE JAVA 21 SOLUTION:

```java
public class TechMReverseNumber {
    public static int reverse(int n) {
        int r = 0;
        while (n != 0) { r = r * 10 + n % 10; n /= 10; }
        return r;
    }
    public static void main(String[] args) {
        System.out.println(reverse(9876)); // 6789
    }
}
```

DRY RUN WALKTHROUGH:

9876 -> 6789.

TIME COMPLEXITY:
O(log10 N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Trailing zeros.

#2: Check Prime Number

Mathematics

PROBLEM STATEMENT:

Determine whether n is a prime number.

SAMPLE INPUT & OUTPUT:

```
Input: 31
Output: true
```

ALGORITHM & INTUITION:

Divisor testing up to sqrt(n).

COMPLETE JAVA 21 SOLUTION:

```java
public class TechMPrime {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i * i <= n; i++) if (n % i == 0) return false;
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isPrime(31)); // true
    }
}
```

DRY RUN WALKTHROUGH:

31 is prime.

TIME COMPLEXITY:
O(sqrt(N)).

SPACE COMPLEXITY:
O(1).

EDGE CASES: n = 1.

#3: Find Factorial

Mathematics

PROBLEM STATEMENT:

Calculate n!.

SAMPLE INPUT & OUTPUT:

```
Input: 6
Output: 720
```

ALGORITHM & INTUITION:

Iterative accumulation using long.

COMPLETE JAVA 21 SOLUTION:

```
public class TechMFactorial {
    public static long fact(int n) {
        long res = 1;
        for (int i = 2; i <= n; i++) res *= i;
        return res;
    }
    public static void main(String[] args) {
        System.out.println("6! = " + fact(6)); // 720
    }
}
```

DRY RUN WALKTHROUGH:

1 * 2 * 3 * 4 * 5 * 6 = 720.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: n = 0 (1).

#4: Print Fibonacci Series

Mathematics

PROBLEM STATEMENT:

Print first n terms of Fibonacci sequence.

SAMPLE INPUT & OUTPUT:

```
Input: 6
Output: 0 1 1 2 3 5
```

ALGORITHM & INTUITION:

Iterative state variables.

COMPLETE JAVA 21 SOLUTION:

```
public class TechMFibonacci {
    public static void printFib(int n) {
        int a = 0, b = 1;
        for (int i = 0; i < n; i++) {
            System.out.print(a + " ");
            int c = a + b; a = b; b = c;
        }
        System.out.println();
    }
    public static void main(String[] args) {
        printFib(6); // 0 1 1 2 3 5
    }
}
```

DRY RUN WALKTHROUGH:

0 1 1 2 3 5.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: n = 1.

#5: Find GCD and LCM

Mathematics

PROBLEM STATEMENT:

Find GCD and LCM of two integers.

SAMPLE INPUT & OUTPUT:

```
Input: 20, 28
Output: GCD = 4, LCM = 140
```

ALGORITHM & INTUITION:

Euclidean algorithm. LCM = (a / gcd) * b.

COMPLETE JAVA 21 SOLUTION:

```java
public class TechMGcdLcm {
    public static int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
    public static int lcm(int a, int b) { return (a / gcd(a, b)) * b; }
    public static void main(String[] args) {
        System.out.println("GCD: " + gcd(20, 28) + ", LCM: " + lcm(20, 28)); // 4, 140
    }
}
```

DRY RUN WALKTHROUGH:

28 % 20 = 8 -> 20 % 8 = 4 -> 8 % 4 = 0. GCD = 4.

TIME COMPLEXITY:
O(log(min(A, B))).

SPACE COMPLEXITY:
O(1).

EDGE CASES: One number zero.

#6: Find Second-Largest Array Element

Arrays

PROBLEM STATEMENT:

Find second-largest distinct element.

SAMPLE INPUT & OUTPUT:

```
Input: [14, 46, 47, 86, 92, 52, 48, 36, 66, 85]
Output: 86
```

ALGORITHM & INTUITION:

Single traversal with max1 and max2.

COMPLETE JAVA 21 SOLUTION:

```java
public class TechMSecondLargest {
    public static int secondLargest(int[] a) {
        int m1 = Integer.MIN_VALUE, m2 = Integer.MIN_VALUE;
        for (int x : a) {
            if (x > m1) { m2 = m1; m1 = x; }
            else if (x < m1 && x > m2) { m2 = x; }
        }
        return m2 == Integer.MIN_VALUE ? -1 : m2;
    }
    public static void main(String[] args) {
        System.out.println(secondLargest(new int[]{14, 46, 47, 86, 92, 52, 48, 36, 66, 85})); // 86
    }
}
```

DRY RUN WALKTHROUGH:

Max = 92, second max = 86.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Duplicates.

#7: Find Missing Number

Arrays

PROBLEM STATEMENT:

Find missing number in range 1 to n.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 3, 5], n = 5
Output: 4
```

ALGORITHM & INTUITION:

Sum formula: n*(n+1)/2 - sum(arr).

COMPLETE JAVA 21 SOLUTION:

```java
public class TechMMissing {
    public static int findMissing(int[] a, int n) {
        long exp = (long) n * (n + 1) / 2;
        long act = 0;
        for (int x : a) act += x;
        return (int)(exp - act);
    }
    public static void main(String[] args) {
        System.out.println(findMissing(new int[]{1, 2, 3, 5}, 5)); // 4
    }
}
```

DRY RUN WALKTHROUGH:

15 - 11 = 4.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Missing 1.

#8: Find Duplicate Elements

Hashing

PROBLEM STATEMENT:

Print elements appearing more than once.

SAMPLE INPUT & OUTPUT:

```
Input: [4, 5, 6, 4, 7, 8, 5]
Output: 4, 5
```

ALGORITHM & INTUITION:

HashSet lookup.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.*;

public class TechMDuplicates {
    public static void printDups(int[] a) {
        Set<Integer> seen = new HashSet<>();
        Set<Integer> dups = new LinkedHashSet<>();
        for (int x : a) if (!seen.add(x)) dups.add(x);
        for (int x : dups) System.out.print(x + " ");
        System.out.println();
    }
    public static void main(String[] args) {
        printDups(new int[]{4, 5, 6, 4, 7, 8, 5}); // 4 5
    }
}
```

DRY RUN WALKTHROUGH:

4 and 5 are duplicates.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: No duplicates.

#9: Rotate an Array

Arrays

PROBLEM STATEMENT:

Rotate array right by k steps in-place.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 3, 4, 5], k = 3
Output: [3, 4, 5, 1, 2]
```

ALGORITHM & INTUITION:

3-step reverse algorithm.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.Arrays;

public class TechMRotate {
    public static void rotate(int[] a, int k) {
        k %= a.length;
        rev(a, 0, a.length - 1); rev(a, 0, k - 1); rev(a, k, a.length - 1);
    }
    private static void rev(int[] a, int l, int r) {
        while (l < r) { int t = a[l]; a[l++] = a[r]; a[r--] = t; }
    }
    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4, 5};
        rotate(a, 3);
        System.out.println(Arrays.toString(a)); // [3, 4, 5, 1, 2]
    }
}
```

DRY RUN WALKTHROUGH:

Rotated by 3.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: k = 0.

#10: Move Zeros to the End

Arrays

PROBLEM STATEMENT:

Shift zeros to end.

SAMPLE INPUT & OUTPUT:

```
Input: [0, 0, 1, 2, 0, 3]
Output: [1, 2, 3, 0, 0, 0]
```

ALGORITHM & INTUITION:

Two-pointer write.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class TechMMoveZeros {
    public static void move(int[] a) {
        int pos = 0;
        for (int x : a) if (x != 0) a[pos++] = x;
        while (pos < a.length) a[pos++] = 0;
    }
    public static void main(String[] args) {
        int[] a = {0, 0, 1, 2, 0, 3};
        move(a);
        System.out.println(Arrays.toString(a)); // [1, 2, 3, 0, 0, 0]
    }
}
```

DRY RUN WALKTHROUGH:

Non-zeros shifted forward.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: All zeros.

#11: Count Character Occurrences

Strings / Hashing

PROBLEM STATEMENT:

Count occurrences of each character in string.

SAMPLE INPUT & OUTPUT:

```
Input: "techmahindra"
Output: t:1 e:1 c:1 h:2 m:1 a:2 i:1 n:1 d:1 r:1
```

ALGORITHM & INTUITION:

LinkedHashMap for order preservation.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.*;

public class TechMCharCount {
    public static void count(String s) {
        Map<Character, Integer> m = new LinkedHashMap<>();
        for (char c : s.toCharArray()) m.put(c, m.getOrDefault(c, 0) + 1);
        for (Map.Entry<Character, Integer> e : m.entrySet()) System.out.print(e.getKey() + ":" +
e.getValue() + " ");
        System.out.println();
    }
    public static void main(String[] args) {
        count("techmahindra");
    }
}
```

DRY RUN WALKTHROUGH:

Counts tracked.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(K).

EDGE CASES: Spaces in string.

#12: Reverse a String

Strings

PROBLEM STATEMENT:

Reverse characters of string.

SAMPLE INPUT & OUTPUT:

```
Input: "Mahindra"
Output: "ardnihaM"
```

ALGORITHM & INTUITION:

Two pointer swap.

COMPLETE JAVA 21 SOLUTION:

```
public class TechMReverseString {
    public static String reverse(String s) {
        char[] ca = s.toCharArray();
        int l = 0, r = ca.length - 1;
        while (l < r) { char t = ca[l]; ca[l++] = ca[r]; ca[r--] = t; }
        return new String(ca);
    }
    public static void main(String[] args) {
        System.out.println(reverse("Mahindra")); // ardnihaM
    }
}
```

DRY RUN WALKTHROUGH:

Reversed.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Single char.

#13: Check Palindrome

Strings

PROBLEM STATEMENT:

Check if string is palindrome.

SAMPLE INPUT & OUTPUT:

```
Input: "racecar"
Output: true
```

ALGORITHM & INTUITION:

Two pointers check.

COMPLETE JAVA 21 SOLUTION:

```java
public class TechMPalindrome {
    public static boolean isPal(String s) {
        int l = 0, r = s.length() - 1;
        while (l < r) if (s.charAt(l++) != s.charAt(r--)) return false;
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isPal("racecar")); // true
    }
}
```

DRY RUN WALKTHROUGH:

racecar matches backwards.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Single char.

#14: Check Anagrams

Strings

PROBLEM STATEMENT:

Check whether two strings are anagrams.

SAMPLE INPUT & OUTPUT:

```
Input: "state", "taste"
Output: true
```

ALGORITHM & INTUITION:

26 bucket array.

COMPLETE JAVA 21 SOLUTION:

```java
public class TechMAnagram {
    public static boolean isAnagram(String s1, String s2) {
        if (s1.length() != s2.length()) return false;
        int[] f = new int[26];
        for (int i = 0; i < s1.length(); i++) {
            f[s1.charAt(i) - 'a']++; f[s2.charAt(i) - 'a']--;
        }
        for (int x : f) if (x != 0) return false;
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isAnagram("state", "taste")); // true
    }
}
```

DRY RUN WALKTHROUGH:

All frequencies 0.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Length mismatch.

#15: Remove Duplicate Characters

Strings

PROBLEM STATEMENT:

Remove duplicate characters from string.

SAMPLE INPUT & OUTPUT:

```
Input: "bananas"
Output: "bans"
```

ALGORITHM & INTUITION:

Boolean seen array.

COMPLETE JAVA 21 SOLUTION:

```java
public class TechMRemoveDups {
    public static String removeDups(String s) {
        boolean[] seen = new boolean[256];
        StringBuilder sb = new StringBuilder();
        for (char c : s.toCharArray()) {
            if (!seen[c]) { seen[c] = true; sb.append(c); }
        }
        return sb.toString();
    }
    public static void main(String[] args) {
        System.out.println(removeDups("bananas")); // bans
    }
}
```

DRY RUN WALKTHROUGH:

"bananas" -> "bans".

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: All identical.

#16: Binary Search

Searching

PROBLEM STATEMENT:

Find index of target in sorted array.

SAMPLE INPUT & OUTPUT:

```
Input: [2, 4, 6, 8, 10], target = 6
Output: 2
```

ALGORITHM & INTUITION:

Binary search divide and conquer.

COMPLETE JAVA 21 SOLUTION:

```java
public class TechMBinarySearch {
    public static int search(int[] a, int t) {
        int l = 0, h = a.length - 1;
        while (l <= h) {
            int m = l + (h - l) / 2;
            if (a[m] == t) return m;
            if (a[m] < t) l = m + 1; else h = m - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        System.out.println(search(new int[]{2, 4, 6, 8, 10}, 6)); // 2
    }
}
```

DRY RUN WALKTHROUGH:

Index 2.

TIME COMPLEXITY:
O(log N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Not found.

#17: Selection Sort

Sorting

PROBLEM STATEMENT:

Implement Selection Sort.

SAMPLE INPUT & OUTPUT:

```
Input: [29, 10, 14, 37, 13]
Output: [10, 13, 14, 29, 37]
```

ALGORITHM & INTUITION:

Find minimum in unsorted part, swap with current index.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class TechMSelectionSort {
    public static void selectionSort(int[] a) {
        int n = a.length;
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < n; j++) {
                if (a[j] < a[minIdx]) minIdx = j;
            }
            int t = a[minIdx]; a[minIdx] = a[i]; a[i] = t;
        }
    }
    public static void main(String[] args) {
        int[] a = {29, 10, 14, 37, 13};
        selectionSort(a);
        System.out.println(Arrays.toString(a)); // [10, 13, 14, 29, 37]
    }
}
```

DRY RUN WALKTHROUGH:

Sorts in O(N^2) swaps.

TIME COMPLEXITY:
O(N^2).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Already sorted.

#18: Merge Two Sorted Arrays

Arrays

PROBLEM STATEMENT:

Merge two sorted arrays.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 5, 9] and [2, 4, 6]
Output: [1, 2, 4, 5, 6, 9]
```

ALGORITHM & INTUITION:

Two pointers merge.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class TechMMergeSorted {
    public static int[] merge(int[] a, int[] b) {
        int[] res = new int[a.length + b.length];
        int i = 0, j = 0, k = 0;
        while (i < a.length && j < b.length) res[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
        while (i < a.length) res[k++] = a[i++];
        while (j < b.length) res[k++] = b[j++];
        return res;
    }
    public static void main(String[] args) {
        System.out.println(Arrays.toString(merge(new int[]{1, 5, 9}, new int[]{2, 4, 6})));
    }
}
```

DRY RUN WALKTHROUGH:

[1, 2, 4, 5, 6, 9].

TIME COMPLEXITY:
O(A + B).

SPACE COMPLEXITY:
O(A + B).

EDGE CASES: Empty array.

#19: Find the Middle of a Linked List

Linked List

PROBLEM STATEMENT:

Find middle node of linked list.

SAMPLE INPUT & OUTPUT:

```
Input: 1 -> 2 -> 3 -> 4 -> 5
Output: 3
```

ALGORITHM & INTUITION:

Fast (2 steps) and slow (1 step) pointers.

COMPLETE JAVA 21 SOLUTION:

```
public class TechMMiddleLL {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }
    public static int findMiddle(Node head) {
        Node slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next; fast = fast.next.next;
        }
        return slow != null ? slow.val : -1;
    }
    public static void main(String[] args) {
        Node h = new Node(1); h.next = new Node(2); h.next.next = new Node(3);
        h.next.next.next = new Node(4); h.next.next.next.next = new Node(5);
        System.out.println("Middle: " + findMiddle(h)); // 3
    }
}
```

DRY RUN WALKTHROUGH:

slow reaches 3.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Even length.

#20: Stack and Queue Implementation

Data Structures

PROBLEM STATEMENT:

Implement Stack using array and Queue using array.

SAMPLE INPUT & OUTPUT:

```
Stack LIFO: push(5), pop() -> 5; Queue FIFO: enqueue(5), dequeue() -> 5
```

ALGORITHM & INTUITION:

Stack pointer top; Queue circular indices front & rear.

COMPLETE JAVA 21 SOLUTION:

```
public class TechMStackQueue {
    static class Stack {
        int[] arr = new int[10]; int top = -1;
        void push(int x) { arr[++top] = x; }
        int pop() { return arr[top--]; }
    }
    static class Queue {
        int[] arr = new int[10]; int front = 0, rear = 0;
        void enqueue(int x) { arr[rear++] = x; }
        int dequeue() { return arr[front++]; }
    }
    public static void main(String[] args) {
        Stack s = new Stack(); s.push(5);
        System.out.println("Stack Pop: " + s.pop()); // 5
        Queue q = new Queue(); q.enqueue(5);
        System.out.println("Queue Dequeue: " + q.dequeue()); // 5
    }
}
```

DRY RUN WALKTHROUGH:

Basic LIFO & FIFO operations.

TIME COMPLEXITY:
O(1).

SPACE COMPLEXITY:
O(Capacity).

EDGE CASES: Overflow / underflow checks.

CHAPTER 10

The Final Master List — 40 Core Problems

The Ultimate Placement Preparation Checklist for Freshers

Target Focus: Universal Foundation for All Technical Interviews

If you have limited preparation time, solve these 40 problems first. They cover 85% of all recurring algorithmic patterns encountered across service and product company campus rounds.

Questions in this Section (40 Problems):

#1: Reverse a Number — Mathematics

PROBLEM STATEMENT:

Given an integer n, return the reversed number, handling negative numbers and preventing 32-bit overflow.

SAMPLE INPUT & OUTPUT:

```
Input: 1234 -> Output: 4321
```

ALGORITHM & INTUITION:

Extract last digit using pop = n % 10, check for overflow against Integer.MAX_VALUE / 10, then rev = rev * 10 + pop.

COMPLETE JAVA 21 SOLUTION:

```java
public class MasterReverseNumber {
    public static int reverse(int n) {
        int rev = 0;
        while (n != 0) {
            int pop = n % 10;
            n /= 10;
            if (rev > Integer.MAX_VALUE / 10 || (rev == Integer.MAX_VALUE / 10 && pop > 7)) return 0;
            if (rev < Integer.MIN_VALUE / 10 || (rev == Integer.MIN_VALUE / 10 && pop < -8)) return 0;
            rev = rev * 10 + pop;
        }
        return rev;
    }
    public static void main(String[] args) { System.out.println(reverse(1234)); }
}
```

DRY RUN WALKTHROUGH:

1234 -> 4321.

TIME COMPLEXITY:
O(log10 N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Overflow; trailing zeros.

#2: Palindrome Number

Mathematics

PROBLEM STATEMENT:

Given an integer x, return true if x is a palindrome integer without converting to string.

SAMPLE INPUT & OUTPUT:

```
Input: 121 -> Output: true; Input: -121 -> Output: false
```

ALGORITHM & INTUITION:

Negative numbers can never be palindromes (e.g. -121 reversed is 121-). Numbers ending in 0 (except 0 itself) cannot be palindromes. Reverse only the second half of the number: while x > revertedNumber, revertedNumber = revertedNumber * 10 + x % 10, x /= 10. Check if x == revertedNumber || x == revertedNumber / 10.

COMPLETE JAVA 21 SOLUTION:

```java
public class PalindromeNumber {
    public static boolean isPalindrome(int x) {
        if (x < 0 || (x % 10 == 0 && x != 0)) return false;
        int rev = 0;
        while (x > rev) {
            rev = rev * 10 + x % 10;
            x /= 10;
        }
        return x == rev || x == rev / 10;
    }
    public static void main(String[] args) {
        System.out.println(isPalindrome(121)); // true
        System.out.println(isPalindrome(-121)); // false
    }
}
```

DRY RUN WALKTHROUGH:

x = 121 -> rev = 1, x = 12 -> rev = 12, x = 1 -> x == rev/10 (1 == 1) -> true.

TIME COMPLEXITY:
O(log10 N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: 0 (true); negative numbers (false); ending in 0 (false).

#3: Prime Number

Mathematics

PROBLEM STATEMENT:

Check if n is prime in O(sqrt(N)).

SAMPLE INPUT & OUTPUT:

```
Input: 29 -> Output: true
```

ALGORITHM & INTUITION:

Check 2 and 3, then check 6k ± 1 up to sqrt(n).

COMPLETE JAVA 21 SOLUTION:

```java
public class MasterPrime {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;
        if (n <= 3) return true;
        if (n % 2 == 0 || n % 3 == 0) return false;
        for (int i = 5; i * i <= n; i += 6) {
            if (n % i == 0 || n % (i + 2) == 0) return false;
        }
        return true;
    }
    public static void main(String[] args) { System.out.println(isPrime(29)); }
}
```

DRY RUN WALKTHROUGH:

29 not divisible by 2, 3, 5, 7 -> prime.

TIME COMPLEXITY:
O(sqrt(N)).

SPACE COMPLEXITY:
O(1).

EDGE CASES: n <= 1.

#4: Armstrong Number

Mathematics

PROBLEM STATEMENT:

Verify if n equals sum of digits raised to power of digit count.

SAMPLE INPUT & OUTPUT:

```
Input: 371 -> Output: true (27 + 343 + 1 = 371)
```

ALGORITHM & INTUITION:

Count digits k, accumulate digit^k.

COMPLETE JAVA 21 SOLUTION:

```java
public class MasterArmstrong {
    public static boolean isArmstrong(int n) {
        int orig = n, k = String.valueOf(n).length();
        long sum = 0, t = n;
        while (t > 0) { sum += Math.round(Math.pow(t % 10, k)); t /= 10; }
        return sum == orig;
    }
    public static void main(String[] args) { System.out.println(isArmstrong(371)); }
}
```

DRY RUN WALKTHROUGH:

3^3 + 7^3 + 1^3 = 27 + 343 + 1 = 371.

TIME COMPLEXITY:
O(log10 N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Single digit numbers.

#5: Fibonacci Series

Mathematics

PROBLEM STATEMENT:

Generate first n Fibonacci numbers.

SAMPLE INPUT & OUTPUT:

```
Input: 5 -> Output: [0, 1, 1, 2, 3]
```

ALGORITHM & INTUITION:

Iterative sliding window.

COMPLETE JAVA 21 SOLUTION:

```java
public class MasterFib {
    public static long[] fib(int n) {
        long[] f = new long[n];
        if (n > 0) f[0] = 0;
        if (n > 1) f[1] = 1;
        for (int i = 2; i < n; i++) f[i] = f[i-1] + f[i-2];
        return f;
    }
    public static void main(String[] args) { System.out.println(java.util.Arrays.toString(fib(5))); }
}
```

DRY RUN WALKTHROUGH:

0 1 1 2 3.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: n = 1.

#6: GCD and LCM

Mathematics

PROBLEM STATEMENT:

Compute GCD and LCM of two numbers.

SAMPLE INPUT & OUTPUT:

```
Input: 12, 18 -> Output: GCD=6, LCM=36
```

ALGORITHM & INTUITION:

Euclidean algorithm + relation LCM = (a / gcd) * b.

COMPLETE JAVA 21 SOLUTION:

```java
public class MasterGcdLcm {
    public static long gcd(long a, long b) { return b == 0 ? a : gcd(b, a % b); }
    public static long lcm(long a, long b) { return (a / gcd(a, b)) * b; }
    public static void main(String[] args) { System.out.println("GCD: " + gcd(12, 18) + ", LCM: " +
lcm(12, 18)); }
}
```

DRY RUN WALKTHROUGH:

gcd(12, 18) = 6, lcm = (12/6)*18 = 36.

TIME COMPLEXITY:
O(log(min(A, B))).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Zero input.

#7: Second-Largest Element

Arrays

PROBLEM STATEMENT:

Find second-largest distinct element in array.

SAMPLE INPUT & OUTPUT:

```
Input: [12, 35, 1, 10, 34, 1] -> Output: 34
```

ALGORITHM & INTUITION:

Maintain largest and secondLargest in single pass.

COMPLETE JAVA 21 SOLUTION:

```
public class MasterSecondLargest {
    public static int secondLargest(int[] a) {
        int l = Integer.MIN_VALUE, s = Integer.MIN_VALUE;
        for (int x : a) {
            if (x > l) { s = l; l = x; }
            else if (x < l && x > s) { s = x; }
        }
        return s == Integer.MIN_VALUE ? -1 : s;
    }
    public static void main(String[] args) { System.out.println(secondLargest(new int[]{12, 35, 1, 10,
34, 1})); }
}
```

DRY RUN WALKTHROUGH:

34 is second-largest.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: All identical.

#8: Find the Missing Number

Arrays

PROBLEM STATEMENT:

Find missing integer in range 0 to n.

SAMPLE INPUT & OUTPUT:

```
Input: [3, 0, 1] -> Output: 2
```

ALGORITHM & INTUITION:

XOR index with values.

COMPLETE JAVA 21 SOLUTION:

```
public class MasterMissing {
    public static int missingNumber(int[] nums) {
        int x = 0;
        for (int i = 0; i <= nums.length; i++) x ^= i;
        for (int num : nums) x ^= num;
        return x;
    }
    public static void main(String[] args) { System.out.println(missingNumber(new int[]{3, 0, 1})); }
}
```

DRY RUN WALKTHROUGH:

(0^1^2^3) ^ (3^0^1) = 2.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Missing 0.

#9: Find Duplicates

Arrays

PROBLEM STATEMENT:

Find duplicate values in array.

SAMPLE INPUT & OUTPUT:
`Input: [4, 3, 2, 7, 8, 2, 3, 1] -> Output: [2, 3]`

ALGORITHM & INTUITION:

HashSet lookup for seen elements.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.*;

public class MasterDuplicates {
    public static List<Integer> findDuplicates(int[] nums) {
        Set<Integer> seen = new HashSet<>(), dups = new LinkedHashSet<>();
        for (int x : nums) if (!seen.add(x)) dups.add(x);
        return new ArrayList<>(dups);
    }
    public static void main(String[] args) { System.out.println(findDuplicates(new int[]{4, 3, 2, 7, 8,
2, 3, 1})); }
}
```

DRY RUN WALKTHROUGH:

[2, 3] duplicate.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: No duplicates.

#10: Move Zeros to the End

Arrays

PROBLEM STATEMENT:

Move all zeros to end in-place.

SAMPLE INPUT & OUTPUT:

```
Input: [0, 1, 0, 3, 12] -> Output: [1, 3, 12, 0, 0]
```

ALGORITHM & INTUITION:

Two-pointer write non-zeros to front, fill tail with zeros.

COMPLETE JAVA 21 SOLUTION:

```java
public class MasterMoveZeros {
    public static void moveZeroes(int[] nums) {
        int p = 0;
        for (int x : nums) if (x != 0) nums[p++] = x;
        while (p < nums.length) nums[p++] = 0;
    }
    public static void main(String[] args) {
        int[] a = {0, 1, 0, 3, 12}; moveZeroes(a);
        System.out.println(java.util.Arrays.toString(a));
    }
}
```

DRY RUN WALKTHROUGH:

[1, 3, 12, 0, 0].

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: All zeros.

#11: Rotate Array by K

Arrays

PROBLEM STATEMENT:

Rotate array to right by k steps.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 3, 4, 5, 6, 7], k = 3 -> Output: [5, 6, 7, 1, 2, 3, 4]
```

ALGORITHM & INTUITION:

Reverse all, reverse 0..k-1, reverse k..n-1.

COMPLETE JAVA 21 SOLUTION:

```
public class MasterRotate {
    public static void rotate(int[] a, int k) {
        k %= a.length;
        rev(a, 0, a.length - 1); rev(a, 0, k - 1); rev(a, k, a.length - 1);
    }
    private static void rev(int[] a, int l, int r) {
        while (l < r) { int t = a[l]; a[l++] = a[r]; a[r--] = t; }
    }
    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4, 5, 6, 7}; rotate(a, 3);
        System.out.println(java.util.Arrays.toString(a));
    }
}
```

DRY RUN WALKTHROUGH:

[5, 6, 7, 1, 2, 3, 4].

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: k = 0.

#12: Remove Duplicates from Sorted Array

Arrays

PROBLEM STATEMENT:

Remove duplicates in-place from sorted array, return unique count.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 1, 2] -> Output: 2, arr = [1, 2, ...]
```

ALGORITHM & INTUITION:

Two-pointer write.

COMPLETE JAVA 21 SOLUTION:

```
public class MasterRemoveDupsSorted {
    public static int removeDuplicates(int[] nums) {
        if (nums.length == 0) return 0;
        int idx = 0;
        for (int i = 1; i < nums.length; i++) if (nums[i] != nums[idx]) nums[++idx] = nums[i];
        return idx + 1;
    }
    public static void main(String[] args) {
        int[] a = {1, 1, 2};
        System.out.println(removeDuplicates(a)); // 2
    }
}
```

DRY RUN WALKTHROUGH:

Unique count 2.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Single element.

#13: Two Sum

Hashing

PROBLEM STATEMENT:

Return indices of two elements adding up to target.

SAMPLE INPUT & OUTPUT:

```
Input: nums = [2, 7, 11, 15], target = 9 -> Output: [0, 1]
```

ALGORITHM & INTUITION:

HashMap complement lookup.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class MasterTwoSum {
    public static int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int c = target - nums[i];
            if (map.containsKey(c)) return new int[]{map.get(c), i};
            map.put(nums[i], i);
        }
        return new int[0];
    }
    public static void main(String[] args) {
        System.out.println(Arrays.toString(twoSum(new int[]{2, 7, 11, 15}, 9))); // [0, 1]
    }
}
```

DRY RUN WALKTHROUGH:

Complement found at index 0.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Negative values.

#14: Maximum Subarray Sum

Kadane's algorithm

PROBLEM STATEMENT:

Find largest contiguous subarray sum.

SAMPLE INPUT & OUTPUT:

```
Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4] -> Output: 6
```

ALGORITHM & INTUITION:

Kadane's algorithm.

COMPLETE JAVA 21 SOLUTION:

```java
public class MasterKadane {
    public static int maxSubArray(int[] nums) {
        int c = nums[0], m = nums[0];
        for (int i = 1; i < nums.length; i++) {
            c = Math.max(nums[i], c + nums[i]);
            m = Math.max(m, c);
        }
        return m;
    }
    public static void main(String[] args) {
        System.out.println(maxSubArray(new int[]{-2, 1, -3, 4, -1, 2, 1, -5, 4})); // 6
    }
}
```

DRY RUN WALKTHROUGH:

Max sum is 6.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: All negative.

#15: Sort 0s, 1s and 2s

Two pointers

PROBLEM STATEMENT:

Sort array of 0, 1, 2 in-place in O(N).

SAMPLE INPUT & OUTPUT:
`Input: [2, 0, 2, 1, 1, 0] -> Output: [0, 0, 1, 1, 2, 2]`

ALGORITHM & INTUITION:

Dutch National Flag (low, mid, high).

COMPLETE JAVA 21 SOLUTION:

```java
public class MasterSortColors {
    public static void sortColors(int[] a) {
        int l = 0, m = 0, h = a.length - 1;
        while (m <= h) {
            if (a[m] == 0) { int t = a[l]; a[l++] = a[m]; a[m++] = t; }
            else if (a[m] == 1) m++;
            else { int t = a[m]; a[m] = a[h]; a[h--] = t; }
        }
    }
    public static void main(String[] args) {
        int[] a = {2, 0, 2, 1, 1, 0}; sortColors(a);
        System.out.println(java.util.Arrays.toString(a));
    }
}
```

DRY RUN WALKTHROUGH:

[0, 0, 1, 1, 2, 2].

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Already sorted.

#16: Merge Two Sorted Arrays

Arrays

PROBLEM STATEMENT:

Merge two sorted arrays into one.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 3] and [2, 5, 6] -> Output: [1, 2, 2, 3, 5, 6]
```

ALGORITHM & INTUITION:

Two pointers stepping through arrays.

COMPLETE JAVA 21 SOLUTION:

```java
public class MasterMergeSorted {
    public static int[] merge(int[] a, int[] b) {
        int[] res = new int[a.length + b.length];
        int i = 0, j = 0, k = 0;
        while (i < a.length && j < b.length) res[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
        while (i < a.length) res[k++] = a[i++];
        while (j < b.length) res[k++] = b[j++];
        return res;
    }
    public static void main(String[] args) {
        System.out.println(java.util.Arrays.toString(merge(new int[]{1, 2, 3}, new int[]{2, 5, 6})));
    }
}
```

DRY RUN WALKTHROUGH:

[1, 2, 2, 3, 5, 6].

TIME COMPLEXITY:
O(A + B).

SPACE COMPLEXITY:
O(A + B).

EDGE CASES: Empty array.

#17: Check If Array is Sorted

Arrays

PROBLEM STATEMENT:

Return true if array is non-decreasing.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 3, 4, 5] -> Output: true
```

ALGORITHM & INTUITION:

Linear scan verifying arr[i] <= arr[i+1].

COMPLETE JAVA 21 SOLUTION:

```
public class MasterCheckSorted {
    public static boolean isSorted(int[] a) {
        for (int i = 0; i < a.length - 1; i++) if (a[i] > a[i + 1]) return false;
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isSorted(new int[]{1, 2, 3, 4, 5})); // true
    }
}
```

DRY RUN WALKTHROUGH:

All adjacent elements in order.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Length 0 or 1.

#18: Reverse a String

Strings

PROBLEM STATEMENT:

Reverse characters of string in-place on char array.

SAMPLE INPUT & OUTPUT:

```
Input: "hello" -> Output: "olleh"
```

ALGORITHM & INTUITION:

Two-pointer swap.

COMPLETE JAVA 21 SOLUTION:

```
public class MasterRevString {
    public static String reverse(String s) {
        char[] ca = s.toCharArray();
        int l = 0, r = ca.length - 1;
        while (l < r) { char t = ca[l]; ca[l++] = ca[r]; ca[r--] = t; }
        return new String(ca);
    }
    public static void main(String[] args) { System.out.println(reverse("hello")); }
}
```

DRY RUN WALKTHROUGH:

"hello" -> "olleh".

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Single char.

#19: Check Anagrams

Strings

PROBLEM STATEMENT:

Determine if two strings are anagrams.

SAMPLE INPUT & OUTPUT:

```
Input: "listen", "silent" -> Output: true
```

ALGORITHM & INTUITION:

26 bucket frequency check.

COMPLETE JAVA 21 SOLUTION:

```java
public class MasterAnagram {
    public static boolean isAnagram(String a, String b) {
        if (a.length() != b.length()) return false;
        int[] f = new int[26];
        for (int i = 0; i < a.length(); i++) {
            f[a.charAt(i) - 'a']++; f[b.charAt(i) - 'a']--;
        }
        for (int x : f) if (x != 0) return false;
        return true;
    }
    public static void main(String[] args) { System.out.println(isAnagram("listen", "silent")); }
}
```

DRY RUN WALKTHROUGH:

Counts match.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Lengths differ.

#20: Reverse Words in a Sentence

Strings

PROBLEM STATEMENT:

Reverse word sequence in sentence.

SAMPLE INPUT & OUTPUT:

```
Input: "the sky is blue" -> Output: "blue is sky the"
```

ALGORITHM & INTUITION:

Split by space, traverse in reverse.

COMPLETE JAVA 21 SOLUTION:

```
public class MasterReverseWords {
    public static String reverseWords(String s) {
        String[] w = s.trim().split("\\s+");
        StringBuilder sb = new StringBuilder();
        for (int i = w.length - 1; i >= 0; i--) sb.append(w[i]).append(i == 0 ? "" : " ");
        return sb.toString();
    }
    public static void main(String[] args) { System.out.println(reverseWords("the sky is blue")); }
}
```

DRY RUN WALKTHROUGH:

Words reversed.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Multiple spaces.

#21: Count Character Frequencies

Hashing

PROBLEM STATEMENT:

Count frequency of every character.

SAMPLE INPUT & OUTPUT:
`Input: "banana" -> Output: b:1, a:3, n:2`

ALGORITHM & INTUITION:

LinkedHashMap for order preservation.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.*;

public class MasterCharFreq {
    public static void printFreq(String s) {
        Map<Character, Integer> m = new LinkedHashMap<>();
        for (char c : s.toCharArray()) m.put(c, m.getOrDefault(c, 0) + 1);
        for (Map.Entry<Character, Integer> e : m.entrySet()) System.out.print(e.getKey() + ":" +
e.getValue() + " ");
        System.out.println();
    }
    public static void main(String[] args) { printFreq("banana"); }
}
```

DRY RUN WALKTHROUGH:

b:1 a:3 n:2.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(K).

EDGE CASES: Special chars.

#22: First Non-Repeating Character

Hashing

PROBLEM STATEMENT:

Find index of first unique character in string.

SAMPLE INPUT & OUTPUT:

```
Input: "leetcode" -> Output: 0 ('l')
```

ALGORITHM & INTUITION:

Frequency count array + second scan.

COMPLETE JAVA 21 SOLUTION:

```java
public class MasterFirstUnique {
    public static int firstUniqChar(String s) {
        int[] f = new int[26];
        for (char c : s.toCharArray()) f[c - 'a']++;
        for (int i = 0; i < s.length(); i++) if (f[s.charAt(i) - 'a'] == 1) return i;
        return -1;
    }
    public static void main(String[] args) { System.out.println(firstUniqChar("leetcode")); } // 0
}
```

DRY RUN WALKTHROUGH:

'l' has frequency 1 at index 0.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: No unique characters.

#23: Longest Common Prefix

Strings

PROBLEM STATEMENT:

Find longest common prefix among array of strings.

SAMPLE INPUT & OUTPUT:

```
Input: ["flower", "flow", "flight"] -> Output: "fl"
```

ALGORITHM & INTUITION:

Horizontal scanning trimming prefix.

COMPLETE JAVA 21 SOLUTION:

```
public class MasterLCP {
    public static String longestCommonPrefix(String[] strs) {
        if (strs.length == 0) return "";
        String p = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (strs[i].indexOf(p) != 0) {
                p = p.substring(0, p.length() - 1);
                if (p.isEmpty()) return "";
            }
        }
        return p;
    }
    public static void main(String[] args) { System.out.println(longestCommonPrefix(new String[]
{"flower", "flow", "flight"})); }
}
```

DRY RUN WALKTHROUGH:

"flower" -> "flow" -> "fl".

TIME COMPLEXITY:
O(S).

SPACE COMPLEXITY:
O(1).

EDGE CASES: No common prefix.

#24: Binary Search

Searching

PROBLEM STATEMENT:

Search sorted array in O(log N).

SAMPLE INPUT & OUTPUT:

```
Input: [-1, 0, 3, 5, 9, 12], target = 9 -> Output: 4
```

ALGORITHM & INTUITION:

Divide and conquer mid calculation.

COMPLETE JAVA 21 SOLUTION:

```
public class MasterBinarySearch {
    public static int search(int[] a, int t) {
        int l = 0, h = a.length - 1;
        while (l <= h) {
            int m = l + (h - l) / 2;
            if (a[m] == t) return m;
            if (a[m] < t) l = m + 1; else h = m - 1;
        }
        return -1;
    }
    public static void main(String[] args) { System.out.println(search(new int[]{-1, 0, 3, 5, 9, 12},
9)); } // 4
}
```

DRY RUN WALKTHROUGH:

Binary search finds 9 at index 4.

TIME COMPLEXITY:
O(log N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Not found.

#25: First and Last Occurrence of Element in Sorted Array

Binary Search

PROBLEM STATEMENT:

Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value in O(log N) time. If target is not found, return [-1, -1].

SAMPLE INPUT & OUTPUT:

```
Input: nums = [5, 7, 7, 8, 8, 10], target = 8
Output: [3, 4]
```

ALGORITHM & INTUITION:

Two Binary Searches:
1. findBound(nums, target, true): When nums[mid] == target, save mid as candidate and search LEFT (high = mid - 1) to find first occurrence.
2. findBound(nums, target, false): When nums[mid] == target, save mid as candidate and search RIGHT (low = mid + 1) to find last occurrence.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Arrays;

public class FirstAndLastOccurrence {
    public static int[] searchRange(int[] nums, int target) {
        int first = findBound(nums, target, true);
        if (first == -1) return new int[]{-1, -1};
        int last = findBound(nums, target, false);
        return new int[]{first, last};
    }

    private static int findBound(int[] nums, int target, boolean isFirst) {
        int low = 0, high = nums.length - 1;
        int ans = -1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (nums[mid] == target) {
                ans = mid;
                if (isFirst) high = mid - 1; // Keep searching left
                else low = mid + 1;          // Keep searching right
            } else if (nums[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return ans;
    }

    public static void main(String[] args) {
        int[] nums = {5, 7, 7, 8, 8, 10};
        System.out.println("Range: " + Arrays.toString(searchRange(nums, 8))); // [3, 4]
    }
}
```

DRY RUN WALKTHROUGH:

target 8: first binary search stops at index 3; second stops at index 4 -> [3, 4].

TIME COMPLEXITY:
O(log N) — two logarithmic passes.

SPACE COMPLEXITY:
O(1) extra space.

EDGE CASES: Target not present [-1, -1]; target occurs once [i, i]; all elements match target [0, n-1].

#26: Merge Sort

Sorting

PROBLEM STATEMENT:

Implement Merge Sort.

SAMPLE INPUT & OUTPUT:

```
Input: [12, 11, 13, 5, 6, 7] -> Output: [5, 6, 7, 11, 12, 13]
```

ALGORITHM & INTUITION:

Divide at mid, conquer recursively, merge with buffer.

COMPLETE JAVA 21 SOLUTION:

```java
public class MasterMergeSort {
    public static void sort(int[] a, int l, int r) {
        if (l < r) {
            int m = l + (r - l) / 2;
            sort(a, l, m); sort(a, m + 1, r);
            merge(a, l, m, r);
        }
    }
    private static void merge(int[] a, int l, int m, int r) {
        int[] t = new int[r - l + 1];
        int i = l, j = m + 1, k = 0;
        while (i <= m && j <= r) t[k++] = a[i] <= a[j] ? a[i++] : a[j++];
        while (i <= m) t[k++] = a[i++];
        while (j <= r) t[k++] = a[j++];
        for (int p = 0; p < t.length; p++) a[l + p] = t[p];
    }
    public static void main(String[] args) {
        int[] a = {12, 11, 13, 5, 6, 7}; sort(a, 0, a.length - 1);
        System.out.println(java.util.Arrays.toString(a));
    }
}
```

DRY RUN WALKTHROUGH:

Sorted to [5, 6, 7, 11, 12, 13].

TIME COMPLEXITY:
O(N log N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Single element.

#27: Reverse a Linked List

Linked List

PROBLEM STATEMENT:

Reverse singly linked list in O(1) space.

SAMPLE INPUT & OUTPUT:

```
Input: 1 -> 2 -> 3 -> null -> Output: 3 -> 2 -> 1 -> null
```

ALGORITHM & INTUITION:

Pointer reversal (prev, curr, next).

COMPLETE JAVA 21 SOLUTION:

```
public class MasterReverseLL {
    static class Node { int v; Node n; Node(int v) { this.v = v; } }
    public static Node reverse(Node h) {
        Node p = null, c = h;
        while (c != null) { Node next = c.n; c.n = p; p = c; c = next; }
        return p;
    }
    public static void main(String[] args) {
        Node h = new Node(1); h.n = new Node(2); h.n.n = new Node(3);
        Node r = reverse(h);
        while (r != null) { System.out.print(r.v + " "); r = r.n; } // 3 2 1
    }
}
```

DRY RUN WALKTHROUGH:

Reversed.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Empty list.

#28: Find Middle of Linked List

Linked List

PROBLEM STATEMENT:

Return middle node of linked list.

SAMPLE INPUT & OUTPUT:
`Input: 1 -> 2 -> 3 -> 4 -> 5 -> Output: 3`

ALGORITHM & INTUITION:

Tortoise and Hare fast/slow pointers.

COMPLETE JAVA 21 SOLUTION:

```
public class MasterMiddleLL {
    static class Node { int v; Node n; Node(int v) { this.v = v; } }
    public static Node middle(Node h) {
        Node s = h, f = h;
        while (f != null && f.n != null) { s = s.n; f = f.n.n; }
        return s;
    }
    public static void main(String[] args) {
        Node h = new Node(1); h.n = new Node(2); h.n.n = new Node(3);
        System.out.println(middle(h).v); // 2
    }
}
```

DRY RUN WALKTHROUGH:

Middle node found.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Even number of nodes.

#29: Detect Linked-List Cycle

Linked List

PROBLEM STATEMENT:

Determine if linked list has a cycle.

SAMPLE INPUT & OUTPUT:

```
Input: 3 -> 2 -> 0 -> -4 (connects back to 2) -> Output: true
```

ALGORITHM & INTUITION:

Floyd's cycle detection (slow advances 1, fast advances 2).

COMPLETE JAVA 21 SOLUTION:

```java
public class MasterDetectCycle {
    static class Node { int v; Node n; Node(int v) { this.v = v; } }
    public static boolean hasCycle(Node h) {
        Node s = h, f = h;
        while (f != null && f.n != null) {
            s = s.n; f = f.n.n;
            if (s == f) return true;
        }
        return false;
    }
    public static void main(String[] args) {
        Node h = new Node(1); h.n = new Node(2); h.n.n = h;
        System.out.println(hasCycle(h)); // true
    }
}
```

DRY RUN WALKTHROUGH:

Fast catches slow -> true.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: No cycle.

#30: Valid Parentheses

Stack

PROBLEM STATEMENT:

Verify bracket nesting and closing.

SAMPLE INPUT & OUTPUT:

```
Input: "()[]{}" -> Output: true
```

ALGORITHM & INTUITION:

Stack push closing counterparts.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.Stack;

public class MasterValidParens {
    public static boolean isValid(String s) {
        Stack<Character> st = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(') st.push(')');
            else if (c == '{') st.push('}');
            else if (c == '[') st.push(']');
            else if (st.isEmpty() || st.pop() != c) return false;
        }
        return st.isEmpty();
    }
    public static void main(String[] args) { System.out.println(isValid("()[]{}")); }
}
```

DRY RUN WALKTHROUGH:

Valid.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Mismatched.

#31: Next Greater Element

Monotonic Stack

PROBLEM STATEMENT:

Find first greater element to right.

SAMPLE INPUT & OUTPUT:

```
Input: [4, 5, 2, 25] -> Output: [5, 25, 25, -1]
```

ALGORITHM & INTUITION:

Monotonic stack right to left.

COMPLETE JAVA 21 SOLUTION:

```
import java.util.*;

public class MasterNGE {
    public static int[] nge(int[] a) {
        int[] res = new int[a.length];
        Stack<Integer> st = new Stack<>();
        for (int i = a.length - 1; i >= 0; i--) {
            while (!st.isEmpty() && st.peek() <= a[i]) st.pop();
            res[i] = st.isEmpty() ? -1 : st.peek();
            st.push(a[i]);
        }
        return res;
    }
    public static void main(String[] args) { System.out.println(Arrays.toString(nge(new int[]{4, 5, 2,
25}))); }
}
```

DRY RUN WALKTHROUGH:

[5, 25, 25, -1].

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Decreasing sequence.

#32: Implement Stack Using Queues

Queue / Stack

PROBLEM STATEMENT:

Implement a last-in-first-out (LIFO) stack using only standard Queue operations (offer, poll, peek, isEmpty).

SAMPLE INPUT & OUTPUT:

```
MyStack s = new MyStack(); s.push(1); s.push(2); s.top(); // 2; s.pop(); // 2
```

ALGORITHM & INTUITION:

Single Queue Approach:
When pushing an element x into queue q:
1. Note current size = q.size().
2. Add x to q.
3. Rotate the preceding 'size' elements by dequeuing and enqueuing them back behind x. Now x is at the front of the queue, ready for O(1) pop and top!

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.LinkedList;
import java.util.Queue;

public class StackUsingQueues {
    static class MyStack {
        private Queue<Integer> q = new LinkedList<>();

        public void push(int x) {
            int size = q.size();
            q.add(x);
            for (int i = 0; i < size; i++) {
                q.add(q.poll());
            }
        }

        public int pop() { return q.poll(); }
        public int top() { return q.peek(); }
        public boolean empty() { return q.isEmpty(); }
    }

    public static void main(String[] args) {
        MyStack st = new MyStack();
        st.push(1); st.push(2);
        System.out.println("Top: " + st.top()); // 2
        System.out.println("Pop: " + st.pop()); // 2
        System.out.println("Empty? " + st.empty()); // false
    }
}
```

DRY RUN WALKTHROUGH:

push(1): q=[1]. push(2): q=[1, 2] -> rotate 1: q=[2, 1]. pop() yields 2 (LIFO).

TIME COMPLEXITY:
push: O(N); pop/top/empty: O(1).

SPACE COMPLEXITY:
O(N) memory.

EDGE CASES: Single element.

#33: Implement Queue Using Stacks

Queue / Stack

PROBLEM STATEMENT:

Implement FIFO queue using two stacks.

SAMPLE INPUT & OUTPUT:

```
push(1), push(2), pop() -> 1
```

ALGORITHM & INTUITION:

In-stack and out-stack with amortized O(1) transfer.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.Stack;

public class MasterQueueUsingStacks {
    static class Queue {
        Stack<Integer> in = new Stack<>(), out = new Stack<>();
        void push(int x) { in.push(x); }
        int pop() { peek(); return out.pop(); }
        int peek() {
            if (out.isEmpty()) while (!in.isEmpty()) out.push(in.pop());
            return out.peek();
        }
    }
    public static void main(String[] args) {
        Queue q = new Queue(); q.push(1); q.push(2);
        System.out.println(q.pop()); // 1
    }
}
```

DRY RUN WALKTHROUGH:

Elements reversed on transfer giving FIFO.

TIME COMPLEXITY:
Amortized O(1).

SPACE COMPLEXITY:
O(N).

EDGE CASES: Empty queue.

#34: Binary Tree Traversals

Binary Trees

PROBLEM STATEMENT:

Implement Inorder, Preorder, and Postorder tree traversals.

SAMPLE INPUT & OUTPUT:

```
Inorder: Left Root Right; Preorder: Root Left Right; Postorder: Left Right Root
```

ALGORITHM & INTUITION:

Recursive DFS tree functions.

COMPLETE JAVA 21 SOLUTION:

```
public class MasterTreeTraversals {
    static class Node { int v; Node l, r; Node(int v) { this.v = v; } }
    public static void inorder(Node root) {
        if (root == null) return;
        inorder(root.l); System.out.print(root.v + " "); inorder(root.r);
    }
    public static void main(String[] args) {
        Node r = new Node(1); r.r = new Node(2); r.r.l = new Node(3);
        System.out.print("Inorder: "); inorder(r); System.out.println(); // 1 3 2
    }
}
```

DRY RUN WALKTHROUGH:

Traversals execute recursively.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(H).

EDGE CASES: null tree.

#35: Height of Binary Tree

Binary Trees

PROBLEM STATEMENT:

Compute maximum depth of binary tree.

SAMPLE INPUT & OUTPUT:
`Input: tree of 3 levels -> Output: 3`

ALGORITHM & INTUITION:

height(root) = 1 + max(height(l), height(r)).

COMPLETE JAVA 21 SOLUTION:

```
public class MasterTreeHeight {
    static class Node { int v; Node l, r; Node(int v) { this.v = v; } }
    public static int height(Node root) {
        if (root == null) return 0;
        return 1 + Math.max(height(root.l), height(root.r));
    }
    public static void main(String[] args) {
        Node r = new Node(1); r.l = new Node(2); r.r = new Node(3);
        System.out.println("Height: " + height(r)); // 2
    }
}
```

DRY RUN WALKTHROUGH:

1 + max(1, 1) = 2.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(H).

EDGE CASES: Empty tree.

#36: BFS and DFS

Graphs

PROBLEM STATEMENT:

Implement graph BFS using Queue and DFS using recursion.

SAMPLE INPUT & OUTPUT:

```
BFS: level order; DFS: deep branch exploration.
```

ALGORITHM & INTUITION:

Queue + visited array for BFS; recursion + visited for DFS.

COMPLETE JAVA 21 SOLUTION:

```java
import java.util.*;

public class MasterGraphBFSDFS {
    public static void bfs(int s, List<List<Integer>> adj, int V) {
        boolean[] vis = new boolean[V];
        Queue<Integer> q = new LinkedList<>();
        vis[s] = true; q.add(s);
        System.out.print("BFS: ");
        while (!q.isEmpty()) {
            int u = q.poll(); System.out.print(u + " ");
            for (int v : adj.get(u)) if (!vis[v]) { vis[v] = true; q.add(v); }
        }
        System.out.println();
    }
    public static void main(String[] args) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < 3; i++) adj.add(new ArrayList<>());
        adj.get(0).add(1); adj.get(1).add(2);
        bfs(0, adj, 3); // 0 1 2
    }
}
```

DRY RUN WALKTHROUGH:

Queue visits layers.

TIME COMPLEXITY:
O(V + E).

SPACE COMPLEXITY:
O(V).

EDGE CASES: Disconnected.

#37: Climbing Stairs

Dynamic Programming

PROBLEM STATEMENT:

Count distinct ways to climb n stairs taking 1 or 2 steps.

SAMPLE INPUT & OUTPUT:

```
Input: 3 -> Output: 3 (1+1+1, 1+2, 2+1)
```

ALGORITHM & INTUITION:

Fibonacci recurrence in O(N) time and O(1) space.

COMPLETE JAVA 21 SOLUTION:

```
public class MasterClimbingStairs {
    public static int climbStairs(int n) {
        if (n <= 2) return n;
        int a = 1, b = 2;
        for (int i = 3; i <= n; i++) { int c = a + b; a = b; b = c; }
        return b;
    }
    public static void main(String[] args) { System.out.println(climbStairs(3)); } // 3
}
```

DRY RUN WALKTHROUGH:

n=3 -> 3.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: n = 1.

#38: Best Time to Buy and Sell Stock

Greedy

PROBLEM STATEMENT:

Find maximum profit from 1 buy and 1 sell.

SAMPLE INPUT & OUTPUT:

```
Input: [7, 1, 5, 3, 6, 4] -> Output: 5
```

ALGORITHM & INTUITION:

Single pass tracking minimum price and maximum profit.

COMPLETE JAVA 21 SOLUTION:

```
public class MasterStock {
    public static int maxProfit(int[] p) {
        int min = Integer.MAX_VALUE, max = 0;
        for (int x : p) {
            if (x < min) min = x;
            else if (x - min > max) max = x - min;
        }
        return max;
    }
    public static void main(String[] args) { System.out.println(maxProfit(new int[]{7, 1, 5, 3, 6,
4})); } // 5
}
```

DRY RUN WALKTHROUGH:

Buy at 1, sell at 6 = 5.

TIME COMPLEXITY:
O(N).

SPACE COMPLEXITY:
O(1).

EDGE CASES: Decreasing prices.

#39: Coin Change

Dynamic Programming

PROBLEM STATEMENT:

Find minimum coins to make amount.

SAMPLE INPUT & OUTPUT:

```
Input: [1, 2, 5], amount = 11 -> Output: 3
```

ALGORITHM & INTUITION:

1D DP: dp[i] = min(dp[i], 1 + dp[i-c]).

COMPLETE JAVA 21 SOLUTION:

```
import java.util.Arrays;

public class MasterCoinChange {
    public static int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;
        for (int i = 1; i <= amount; i++) {
            for (int c : coins) if (c <= i) dp[i] = Math.min(dp[i], 1 + dp[i - c]);
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
    public static void main(String[] args) { System.out.println(coinChange(new int[]{1, 2, 5}, 11)); } //
3
}
```

DRY RUN WALKTHROUGH:

3 coins.

TIME COMPLEXITY:
O(amount * coins.length).

SPACE COMPLEXITY:
O(amount).

EDGE CASES: amount = 0.

#40: Longest Common Subsequence

Dynamic Programming

PROBLEM STATEMENT:

Compute length of longest common subsequence between two strings.

SAMPLE INPUT & OUTPUT:

```
Input: "abcde", "ace" -> Output: 3
```

ALGORITHM & INTUITION:

2D DP matrix table.

COMPLETE JAVA 21 SOLUTION:

```
public class MasterLCS {
    public static int lcs(String s1, String s2) {
        int m = s1.length(), n = s2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (s1.charAt(i - 1) == s2.charAt(j - 1)) dp[i][j] = 1 + dp[i - 1][j - 1];
                else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
        return dp[m][n];
    }
    public static void main(String[] args) { System.out.println(lcs("abcde", "ace")); } // 3
}
```

DRY RUN WALKTHROUGH:

"ace" of length 3.

TIME COMPLEXITY:
O(M * N).

SPACE COMPLEXITY:
O(M * N).

EDGE CASES: Empty strings.

Complexity Reference Sheet (Big-O Cheat Sheet)

Quick revision table for Data Structure and Algorithm time & space complexities.

1. Data Structure Complexities

Data Structure	Access	Search	Insertion	Deletion	Space
Array (Static / Dynamic)	O(1)	O(N)	O(N) / O(1) amortized	O(N)	O(N)
Singly Linked List	O(N)	O(N)	O(1) at head / O(N)	O(1) / O(N)	O(N)
Stack (Array / List)	O(N)	O(N)	O(1) push	O(1) pop	O(N)
Queue (FIFO)	O(N)	O(N)	O(1) enqueue	O(1) dequeue	O(N)
Hash Table / HashMap	N/A	O(1) avg / O(N) worst	O(1) avg	O(1) avg	O(N)
Binary Search Tree (Balanced)	O(log N)	O(log N)	O(log N)	O(log N)	O(N)
Binary Heap (PriorityQueue)	O(1) peek	O(N)	O(log N)	O(log N) poll	O(N)
Graph (V vertices, E edges)	Adjacency List	BFS/DFS O(V+E)	O(1)	O(E)	O(V+E)

2. Algorithmic Patterns & Sorting/Searching Complexities

Algorithm	Best Time	Average Time	Worst Time	Space
Binary Search	O(1)	O(log N)	O(log N)	O(1)
Kadane's Subarray Sum	O(N)	O(N)	O(N)	O(1)
Dutch National Flag (0,1,2)	O(N)	O(N)	O(N)	O(1)
Merge Sort	O(N log N)	O(N log N)	O(N log N)	O(N)
Quick Sort	O(N log N)	O(N log N)	O(N^2)	O(log N)
Dijkstra's Shortest Path	O((V+E) log V)	O((V+E) log V)	O((V+E) log V)	O(V+E)
0/1 Knapsack (DP)	O(N * W)	O(N * W)	O(N * W)	O(W)
Longest Common Subsequence	O(M * N)	O(M * N)	O(M * N)	O(M * N)

Final 60-Day Placement Preparation Roadmap

Structured 4-phase preparation guide for technical placement exams & interviews.

Phase 1 (Days 1–15): Foundations & Core Patterns

Focus Area: Arrays, Two Pointers, Sliding Window & Hashing

- Master in-place manipulation: Move Zeros, Rotate Array, Dutch National Flag.
- Hashing patterns: Two Sum, Find Duplicates, Frequency Counting, Anagrams.

Phase 2 (Days 16–30): Essential Linear Data Structures & Strings

Focus Area: Linked Lists, Stacks, Queues & String Algorithms

- Linked List master patterns: Reverse Linked List, Middle of List, Floyd's Cycle Detection.
- Stack mechanics: Balanced Parentheses, Next Greater Element, Min-Stack.

Phase 3 (Days 31–45): Trees, Graphs & Recursion

Focus Area: Binary Trees, BFS/DFS & Backtracking

- Binary Tree fundamentals: Inorder/Preorder/Postorder traversals, Maximum Height.
- Tree ancestors: Lowest Common Ancestor (LCA) and Diameter of Binary Tree.

Phase 4 (Days 46–60): Dynamic Programming & Mock Rounds

Focus Area: 1D/2D Dynamic Programming & Timed Company Mocks

- Core DP paradigms: Climbing Stairs, Coin Change, Subset Sum, 0/1 Knapsack.
- String DP: Longest Common Subsequence (LCS), Longest Increasing Subsequence (LIS).

5 LIVE TECHNICAL INTERVIEW GOLDEN RULES

1. Clarify Constraints First: Ask the interviewer about input sizes (e.g. N <= 10^5 indicates an O(N) or O(N log N) solution is expected; N <= 20 permits exponential backtracking).
2. State the Brute Force First: Never jump silently to the optimal code. Explain the naive approach, state its O(N^2) complexity, and then explain where redundant work occurs.
3. Name the Pattern: Demonstrate technical maturity by explicitly mentioning algorithmic patterns: 'We can optimize this from O(N^2) to O(N) using a two-pointer sliding window.'
4. Test with Edge Cases: Before declaring your code finished, test mentally with null/empty inputs, single-element arrays, all duplicate values, and negative integers.
5. Write Clean, Modular Code: Use descriptive identifiers (e.g., 'insertPos', 'currentSum') rather than cryptic single letters, and separate helper functions cleanly.

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified

© 2026 CONCEPTIFYYY • All Solutions Java 21 Verified